

Detección de poses de agarre 2D/2.5D para brazos robóticos en entornos no Estructurados con CNNs.

14 abril 2026



Universidad
Internacional
de Valencia

Titulación:
Máster en Inteligencia Artificial

Curso académico
2025 – 2026

Alumno/a:
Lozano Rodríguez, Jesús
D.N.I: 52927628M

Director/a de TFM:
Jesús Marcey García

Convocatoria:
Segunda

De:
 Planeta Formación y Universidades

Índice

Resumen	9
Abstract.....	10
1. Introducción	11
1.1. Contextualización y motivación	11
1.2. Enfoque del trabajo y visión de integración robótica	12
2. Objetivos y alcance	14
2.1. Objetivo general	14
2.2. Objetivos específicos	14
2.3. Alcances y limitaciones	15
3. Estado del Arte y Marco teórico	18
3.1. Introducción y alcance del capítulo	18
3.2. Avances recientes en detección de poses de agarre en RGB-D	19
3.2.1. Métodos 2D/2.5D basados en mapas densos de agarre	19
3.2.2. Métodos 2D/2.5D por regresión de parámetros.....	20
3.3. Métodos 6-DoF basados en nubes de puntos.....	21
3.4. Robustez en escenas complejas y fondos ruidosos.....	22
3.5. Avances recientes en modelos y enfoques de aprendizaje	23
3.5.1. Atención y Transformers en agarre robótico	23
3.5.2. Integración visión-control y aprendizaje por refuerzo.....	23
3.6. Discusión y posicionamiento del presente trabajo	23
3.7. Datasets de referencia y protocolos de evaluación.....	25
3.7.1. Datasets para detección 2D/2.5D.....	25
3.7.2. Datasets para agarre 6-DoF	25
3.7.3. Protocolos de particionado: por imagen vs por objeto (image-wise vs object-wise)	26
3.8. Marco teórico.....	27
3.8.1. Enfoque de aprendizaje supervisado	27
3.8.2. Representación del problema: entradas RGB/RGB-D y salida paramétrica	27
3.8.3. CNN y backbones residuales (ResNet) para regresión de agarres	29
3.8.4. Métricas geométricas tipo Cornell.....	31
3.8.5. Métricas de eficiencia computacional	32
4. Metodología	33
4.1. Objetivo del capítulo	33

4.2.	Enfoque general y fases del trabajo	33
4.3.	Dataset y preparación de datos	36
4.3.1.	Cornell Grasping Dataset.....	36
4.3.2.	Particionado object-wise.....	37
4.4.	Auditoría y filtrado del dataset	38
4.5.	Preprocesamiento y data augmentation	39
4.5.1.	Normalización y redimensionado	39
4.5.2.	Transformación de anotaciones	39
4.5.3.	Aumento de datos	39
4.6.	Modelos propuestos	40
4.6.1.	ResNet18Grasp	41
4.6.2.	SimpleGraspCNN.....	42
4.7.	Protocolo de entrenamiento y reproducibilidad.....	44
4.7.1.	Entrenamiento supervisado.....	44
4.7.2.	Selección de mejor época	45
4.7.3.	Control de semillas y determinismo	45
4.8.	Evaluación cuantitativa, cualitativa y de eficiencia.....	46
4.8.1.	Métrica principal: grasp success tipo Cornell.....	46
4.8.2.	Evaluación cualitativa.....	48
4.8.3.	Métricas de eficiencia computacional	48
4.9.	Diseño de integración robótica (ROS 2 + Gazebo)	49
4.9.1.	Interfaz de percepción: entradas y salidas	50
4.9.2.	Sincronización temporal y reproducibilidad en ROS 2	51
4.9.3.	Proyección imagen→escena: calibración y transformación geométrica	51
4.9.4.	Planificación y ejecución.....	52
4.9.5.	Evidencias verificables en el entorno simulado	52
4.10.	Herramientas software y hardware	53
4.10.1.	Entorno software.....	53
4.10.2.	Infraestructura hardware.....	53
4.10.3.	Artefactos generados y estructura reproducible	53
4.11.	Resumen del capítulo y garantías de reproducibilidad.....	54
5.	Resultados y discusión	56
5.1.	Resultados de entrenamiento	56
5.1.1.	Curvas de pérdida y métricas de validación	56
5.2.	Análisis cualitativo (aciertos y fallos)	64

5.2.1.	Tipos de acierto observados	64
5.2.2.	Tipos de fallo (tipología)	65
5.2.3.	Comparación cualitativa entre modelos	67
5.3.	Eficiencia computacional	68
5.4.	Resultados de integración en ROS 2 + Gazebo	70
5.4.1.	Escena simulada y adquisición de imagen	70
5.4.2.	Modelo integrado y generación de la hipótesis de agarre	71
5.4.3.	Publicación en ROS 2 y consumo por MoveIt	72
5.4.4.	Alcance de la validación y nivel de evidencia.....	73
5.4.5.	Discusión	73
5.5.	Comparación con el estado del arte	73
5.6.	Discusión: causas probables del rendimiento modesto y plan de mejora.....	75
5.7.	Conclusiones parciales del capítulo.....	75
6.	Conclusiones y trabajos futuros	77
6.1.	Síntesis del trabajo realizado	77
6.2.	Consideración sobre el uso de un manipulador 6-DoF.....	78
6.3.	Conclusiones por objetivo específico	78
6.4.	Discusión crítica de resultados y limitaciones.....	80
6.5.	Aportaciones principales	81
6.6.	Líneas de trabajo futuro	82
6.7.	Cierre del capítulo	82
7.	Referencias bibliográficas	83
ANEXO A.	Repositorio y trazabilidad experimental.....	85
ANEXO B.	Modelo de Declaración del uso de sistemas de IA	86



A mi familia, por su apoyo incondicional durante el Máster, y a quienes creen que la inteligencia artificial puede mejorar la vida de las personas, muy especialmente a mi hijo Jesús y a mi niña Sofía, porque vuestra curiosidad y vuestra sonrisa dan sentido a cada esfuerzo y me recuerdan que la verdadera razón de este trabajo es que la IA sirva para construir un mundo más justo, humano y bonito, hecho a vuestra medida.

Y, de manera muy especial, en el recuerdo de Pili, mi suegra, que siempre estuvo ahí para ayudar, cuidar y darlo todo por su familia, especialmente por sus nietos. Su cariño, su generosidad y su entrega permanecerán siempre en nuestros corazones. Este trabajo quiere ser también un pequeño homenaje a su memoria.

Índice de ilustraciones

Ilustración 1-1. Ejemplos de escenarios no estructurados para detección de poses de agarre.	11
Ilustración 1-2. Representación 2D/2.5D del agarre como rectángulo orientado.	12
Ilustración 1-3. Diagrama general del pipeline del trabajo e integración.	13
Ilustración 2-1. Mapa de trazabilidad del pipeline (dataset → modelo → métricas → ROS 2/Gazebo).	15
Ilustración 3-1. Taxonomía del problema de agarre: 2D/2.5D vs 6-DoF.	19
Ilustración 3-2. Ejemplo conceptual de mapas densos (calidad, ángulo, apertura) estilo GG-CNN.	20
Ilustración 3-3. Ejemplos de degradación por oclusión/fondo: tipología de fallos.	22
Ilustración 3-4. Mapa de posicionamiento: “estado del arte → decisiones del trabajo”.	24
Ilustración 3-5. Parámetros del rectángulo de agarre (C_x , C_y , w , h , θ) sobre una imagen.	28
Ilustración 3-6. Interpretación geométrica de IoU y $\Delta\theta$ en rectángulos orientados.	32
Ilustración 4-1. Visión global del pipeline experimental.	33
Ilustración 4-2. Ejemplo de anotaciones Cornell sobre una imagen.	37
Ilustración 4-3. Flujo del pipeline de preparación del Cornell Grasping Dataset y generación de la partición object-wise reproducible.	38
Ilustración 4-4. Ejemplo de evaluación tipo Cornell con fallo por incumplimiento de umbrales de IoU y/o $\Delta\theta$	47
Ilustración 4-5. Galería cualitativa: 4 aciertos + 4 fallos con explicación del tipo de fallo.	48
Ilustración 4-6. Arquitectura ROS 2 del entorno simulado (nodos y tópicos).	50
Ilustración 4-7. Entorno de emulación ROS 2/Gazebo: ejemplo de consistencia visual (overlay) y consumo de la hipótesis de agarre en la escena table-top.	52
Ilustración 5-1. Curvas de pérdida (train_loss y val_loss) para EXP1 (SimpleGraspCNN, RGB).	57
Ilustración 5-2. Curvas de pérdida (train_loss y val_loss) para EXP2 (SimpleGraspCNN, RGB-D con aumento moderado).	57
Ilustración 5-3. Curvas de pérdida (train_loss y val_loss) para EXP3 (ResNet18Grasp, RGB + augmentation).	57
Ilustración 5-4. Curvas de pérdida (train_loss y val_loss) para EXP4 (ResNet18Grasp, RGB-D).	57
Ilustración 5-5. Evolución del éxito de agarre en validación (val_success) por época para las cuatro configuraciones experimentales.	58
Ilustración 5-6. Evolución del IoU medio en validación por época para las cuatro configuraciones experimentales.	59
Ilustración 5-7. Evolución del error angular medio ($\Delta\theta$, en grados) en validación por época para las cuatro configuraciones experimentales.	59
Ilustración 5-8. Comparativa de val_success por ejecución (seed) y experimento, tomando el valor en la mejor época (best_epoch) seleccionada por máximo val_success.	61
Ilustración 5-9. Comparativa de val_loss por ejecución (seed) y experimento, evaluado en best_epoch (seleccionado por máximo val_success).	61

Ilustración 5-10. Éxito final de agarre en validación (val_success), agregado por experimento (media $\pm$ desviación estándar sobre semillas), seleccionando en cada ejecución best_epoch por máximo val_success.....	63
Ilustración 5-11. IoU medio final en validación, agregado por experimento (media $\pm$ desviación estándar sobre semillas) bajo el criterio de selección de best_epoch.	63
Ilustración 5-12. Error angular medio final ($\Delta\theta$ en grados) en validación, agregado por experimento (media $\pm$ desviación estándar sobre semillas), bajo el criterio de best_epoch.	63
Ilustración 5-13. Aciertos representativos en validación para el modelo EXP3_RESNET18_RGB_AUGMENT (seed 0), con superposición entre el GT seleccionado (verde discontinuo) y la predicción (rojo continuo).	65
Ilustración 5-14. Fallos cualitativos representativos en validación, organizados por tipología: E1 (error angular), E2 (bajo solapamiento), E3 (tamaño o apertura no plausibles) y E4 (confusión por fondo o región no funcional).	66
Ilustración 5-15. Caso límite de falso negativo por IoU insuficiente (EXP2_SIMPLE_RGBD, seed 0). La predicción mantiene una orientación compatible con la referencia ($\Delta\theta = 17,18^\circ$), pero no alcanza el umbral Cornell de solapamiento ($\text{IoU} = 0,248 < 0,25$), por lo que se clasifica como fallo	68
Ilustración 5-16. Evidencia funcional del pipeline percepción–publicación–consumo en ROS 2, donde la hipótesis de agarre generada por el detector se transforma en una pose ejecutable y es procesada por el subsistema de planificación a través de la interfaz definida para la integración con MoveIt.....	70
Ilustración 5-17. Resultado de inferencia del modelo EXP3_RESNET18_RGB_AUGMENT sobre una imagen simulada, mostrando la hipótesis de agarre estimada y su representación visual en el panel de integración. En este ejemplo, la predicción se sitúa próxima a la referencia de agarre, lo que permite visualizar de forma cualitativa el comportamiento del sistema.....	71
Ilustración 5-18. Evidencia funcional del pipeline percepción–publicación–consumo en ROS 2, en el que la hipótesis de agarre generada por el detector se publica para su procesamiento por el subsistema de planificación, verificando la coherencia de la integración entre percepción, publicación y consumo dentro del entorno simulado.....	72

Índice de tablas

Tabla 2-1. Trazabilidad entre objetivos específicos y evidencias en el documento.....	15
Tabla 2-2. Fuera de alcance vs. extensiones futuras.....	17
Tabla 3-1. Comparativa sintética de enfoques 2D/2.5D: mapas densos vs regresión paramétrica.	21
Tabla 3-2. Datasets relevantes: Cornell/Jacquard (2D/2.5D) vs GraspNet/ACRONYM (6-DoF).	25
Tabla 4-1. Fases del trabajo, entradas, salidas y evidencias de verificación.	35
Tabla 4-2. Estadísticas del split utilizado en el conjunto experimental final.	37
Tabla 4-3. Transformaciones de data augmentation y cómo se actualizan las etiquetas Cx, Cy, w, h, θ	40
Tabla 4-4. Comparativa estructural de los modelos (SimpleGraspCNN vs ResNet18Grasp).	44
Tabla 4-5. Configuración experimental por experimento.	45
Tabla 4-6. Hiperparámetros de entrenamiento por experimento (según YAML asociado).	46
Tabla 4-7. Definición de métricas de evaluación (Cornell): IoU, $\Delta\theta$ y grasp success ...	47
Tabla 4-8. Protocolo de medición de latencia.	49
Tabla 4-9. Especificaciones de hardware utilizadas.	53
Tabla 5-1. Resultados agregados en validación (best_epoch por ejecución) bajo split object-wise.	62
Tabla 5-2. Resumen de métricas finales por experimento en validación (media $\pm$ desviación estándar sobre $n = 3$ semillas).	62
Tabla 5-3. Medición de latencia de inferencia por experimento y dispositivo, incluyendo parámetros del protocolo y métricas agregadas de tiempo de ejecución.	69
Tabla 5-4. Comparativa por modalidad entre SimpleGraspCNN y ResNet18Grasp derivada de la Tabla 5-1.	76
Tabla 6-1. Síntesis de cumplimiento de objetivos.	78

Resumen

Este trabajo desarrolla un *pipeline* reproducible para la detección de agarres 2D/2.5D en entornos no estructurados de tipo *table-top*, orientado a la ejecución con pinza paralela a partir de entradas RGB y RGB-D. La formulación adoptada representa cada agarre como un rectángulo orientado en el plano de imagen, parametrizado por su centro (C_x, C_y), dimensiones (w, h) y ángulo θ , siguiendo el esquema clásico del *Cornell Grasping Dataset*. Sobre esta representación se construye un flujo completo de percepción que incluye preprocesado, inferencia del modelo, selección de la hipótesis de agarre y publicación del resultado para su consumo por módulos robóticos.

La evaluación se realiza sobre Cornell mediante un particionado *object-wise*, de manera que los objetos presentes en validación no aparecen en entrenamiento, aproximando un escenario de generalización a objetos no vistos. Para garantizar trazabilidad y reproducibilidad, se emplea una partición reproducible materializada en los archivos *train.csv* y *val.csv* bajo protocolo *object-wise*. En la versión efectiva utilizada por el pipeline final, el conjunto de entrenamiento contiene 3542 pares imagen- anotación y el conjunto de validación 1569 pares imagen- anotación. Dado que una misma imagen puede disponer de múltiples anotaciones válidas de agarre, durante validación cada predicción se compara contra todas las anotaciones *Ground Truth* asociadas a su imagen, seleccionando el mejor emparejamiento válido. En consecuencia, aunque *val.csv* contiene 1569 filas, estas corresponden a 250 imágenes únicas con múltiples rectángulos de agarre anotados.

Se comparan dos modelos con la misma salida paramétrica: SimpleGraspCNN, diseñado para ser ligero y eficiente, y ResNet18Grasp, basado en una arquitectura residual de referencia. Ambos se entrenan y evalúan en modalidades RGB y RGB-D, estudiando el impacto de incorporar profundidad como canal adicional y del aumento de datos sobre la robustez. Los resultados finales se agregan sobre $n = 3$ semillas por experimento. Bajo este protocolo, SimpleGraspCNN alcanza un *val_success* medio de $27,41 \pm 4,40$ % en RGB y $38,05 \pm 0,51$ % en RGB-D, mientras que ResNet18Grasp obtiene $68,01 \pm 2,45$ % en RGB con aumento de datos y $64,92 \pm 1,02$ % en RGB-D. En conjunto, los resultados muestran una superioridad clara de la arquitectura residual frente al modelo ligero y sitúan a las configuraciones basadas en ResNet18Grasp como las más adecuadas dentro del marco experimental evaluado.

Finalmente, se valida la integración robótica en un entorno de emulación basado en ROS 2 y Gazebo. En este escenario, el sistema recibe las imágenes generadas por la cámara del simulador, calcula una hipótesis de agarre a partir de la información visual y la transmite al módulo encargado de planificar y ejecutar el movimiento del brazo robótico. Con ello, se comprueba el funcionamiento conjunto de las etapas de percepción, estimación del agarre y ejecución dentro de una arquitectura estándar de ROS 2.

Palabras clave: agarre robótico; detección de agarres; pose de agarre; RGB-D; aprendizaje profundo; redes convolucionales; eficiencia computacional; Cornell Grasping Dataset; ROS 2; Gazebo.

Abstract

This thesis develops a reproducible deep-learning *pipeline* for 2D/2.5D grasp detection in non-structured table-top environments, targeting execution with a parallel-jaw gripper from RGB and RGB-D inputs. Grasps are modeled using the Cornell formulation as oriented rectangles parameterized by center (C_x, C_y), dimensions (w, h), and rotation θ . The perception stack covers preprocessing, model inference, grasp hypothesis selection, and publication of the output for downstream robotic modules.

Evaluation is carried out on Cornell using an object-wise split, so that objects present in validation do not appear in training, approximating a generalization scenario to unseen objects. To ensure traceability and reproducibility, the final pipeline uses a reproducible split materialized in the `train.csv` and `val.csv` files under the object-wise protocol. In the effective version used for the final experiments, the training set contains 3542 image-annotation pairs and the validation set contains 1569 image-annotation pairs. Since a single image may have multiple valid grasp annotations, during validation each prediction is compared against all ground-truth annotations associated with its image, selecting the best valid match. Consequently, although `val.csv` contains 1569 rows, these correspond to 250 unique images with multiple annotated grasp rectangles.

Two models with the same parametric output are compared: SimpleGraspCNN, designed to be lightweight and efficient, and ResNet18Grasp, based on a reference residual architecture. Both are trained and evaluated under RGB and RGB-D settings, analyzing the impact of incorporating depth as an additional channel and of data augmentation on robustness. Final results are aggregated over $n = 3$ random seeds per experiment. Under this protocol, SimpleGraspCNN achieves a mean `val_success` of $27.41 \pm 4.40\%$ in RGB and $38.05 \pm 0.51\%$ in RGB-D, while ResNet18Grasp obtains $68.01 \pm 2.45\%$ in RGB with data augmentation and $64.92 \pm 1.02\%$ in RGB-D. Overall, the results show a clear superiority of the residual architecture over the lightweight model and position the ResNet18Grasp-based configurations as the most suitable within the evaluated experimental framework.

Finally, the robotic integration is validated in an emulation environment based on ROS 2 and Gazebo. In this scenario, the system processes the images generated by the simulated camera, computes a grasp hypothesis from the visual information, and sends it to the module responsible for planning and executing the robotic arm motion. This makes it possible to verify the end-to-end operation of perception, grasp estimation, and execution within a standard ROS 2 architecture.

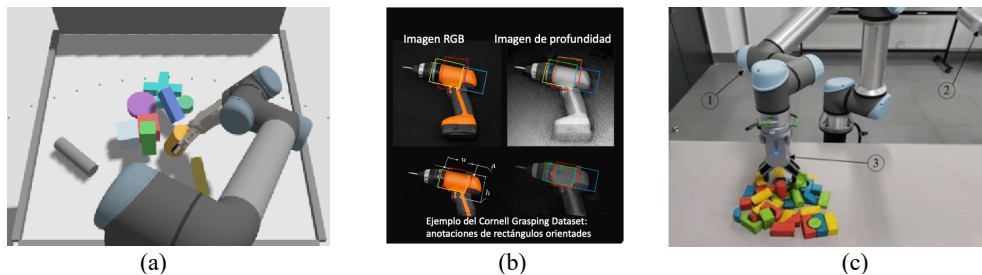
Keywords: robotic grasping; grasp detection; grasp pose; RGB-D; deep learning; convolutional neural networks; computational efficiency; Cornell Grasping Dataset; ROS 2; Gazebo.

1. Introducción

1.1. Contextualización y motivación

La manipulación robótica robusta en entornos reales sigue siendo uno de los retos más relevantes de la robótica moderna (Russell & Norvig, 2021), especialmente fuera de escenarios industriales altamente estructurados (Kleeberger et al., 2020). En entornos cotidianos —hogares, hospitales, almacenes o comercios— se combinan variaciones de iluminación, oclusiones parciales y una elevada diversidad de geometrías y materiales, lo que incrementa la incertidumbre y exige sistemas de percepción con capacidad de generalización (Xie et al., 2023).

La Ilustración 1-1 recoge tres ejemplos ilustrativos de escenarios de manipulación en los que se plantea el problema de detección de poses de agarre: (a) una captura del escenario sobre la mesa *table-top* en Gazebo empleado en este trabajo; (b) un ejemplo representativo del *Cornell Grasping Dataset*, donde las poses de agarre se anotan como rectángulos orientados sobre observaciones RGB-D; y (c) una escena real de manipulación en un entorno con desorden (objetos apilados o desordenados).



En los últimos años, el avance del aprendizaje profundo, junto con la disponibilidad de sensores RGB-D asequibles, ha impulsado enfoques de agarre guiado por visión capaces de inferir configuraciones de agarre directamente a partir de observaciones visuales (Platt, 2023). Estas propuestas se apoyan principalmente en redes convolucionales y, más recientemente, en modelos basados en atención, con el objetivo de generalizar a objetos no vistos durante el entrenamiento (Hou et al., 2025). Sin embargo, una parte de la literatura prioriza el rendimiento predictivo sin analizar con el mismo nivel de detalle el coste computacional asociado, lo que limita el despliegue en plataformas con recursos restringidos o con requisitos de respuesta cercanos al tiempo real (Morrison et al., 2020).

Dentro de este contexto, la detección de poses de agarre a partir de imágenes RGB-D constituye un componente clave para la manipulación robótica. En una formulación ampliamente utilizada, la tarea consiste en estimar una o varias configuraciones de agarre representadas como rectángulos orientados en el plano de imagen (agarre 2D/2.5D), que aproximan la acción de una pinza paralela sobre la escena observada (Jiang et al., 2011).

La disponibilidad de conjuntos de datos de referencia, como el *Cornell Grasping Dataset*, ha contribuido a estandarizar esta tarea y a facilitar comparaciones entre métodos mediante métricas ampliamente aceptadas en la comunidad (Jiang et al., 2011). No obstante, el rendimiento obtenido en condiciones relativamente controladas no siempre se traslada de forma directa a escenarios con mayor variabilidad, lo que refuerza la necesidad de *pipelines* reproducibles y de evaluaciones críticas bajo configuraciones exigentes (Kleeberger et al., 2020).

Para concretar el marco experimental adoptado en este trabajo, a continuación, se describe la representación de las poses de agarre empleada en la evaluación. La Ilustración 1-2 muestra la representación 2D/2.5D empleada, superpuesta sobre una observación RGB o de profundidad. Esta formulación es especialmente adecuada para escenarios “*table-top*” con pinza paralela y es compatible con el protocolo de evaluación tipo Cornell (IoU y error angular).

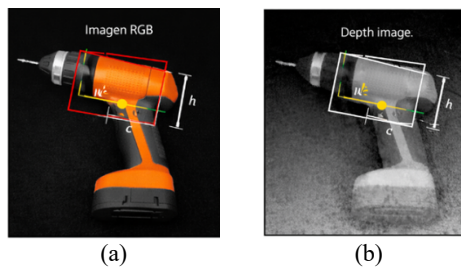


Ilustración 1-2. Representación 2D/2.5D del agarre como rectángulo orientado.

(a) Ejemplo sobre imagen RGB. (b) Ejemplo sobre imagen de profundidad (canal D). (Jiang et al., 2011). (Elaboración propia a partir de una muestra del *Cornell Grasping Dataset*).

Desde la perspectiva del Máster en Inteligencia Artificial, este problema resulta especialmente adecuado para integrar aprendizaje profundo, visión artificial, diseño experimental y análisis cuantitativo, además de permitir situar la percepción como parte de un flujo robótico completo (Russell & Norvig, 2021).

1.2. Enfoque del trabajo y visión de integración robótica

En este marco, este trabajo se centra en el diseño, implementación y evaluación de un sistema de detección de poses de agarre mediante aprendizaje profundo sobre imágenes RGB-D, tomando como referencia el *Cornell Grasping Dataset* y considerando explícitamente el equilibrio entre rendimiento y eficiencia computacional (Jiang et al., 2011; Morrison et al., 2020). El alcance experimental se formula en términos de detección 2D/2.5D de rectángulos de agarre para pinza paralela, un planteamiento adecuado para tareas de manipulación principalmente planares y coherente con la evaluación estándar utilizada en Cornell (Jiang et al., 2011).

El sistema se materializa en dos modelos basados en redes convolucionales:

- Modelo A (ResNet18Grasp): modelo de referencia basado en ResNet-18 pre-entrenado, representativo de arquitecturas de uso extendido y adaptado a la estimación de rectángulos de agarre en RGB y RGB-D (He et al., 2015).

- **Modelo B (SimpleGraspCNN):** modelo ligero diseñado específicamente para este trabajo, orientado a ejecución eficiente y evaluado en modalidad RGB y RGB-D.

Este enfoque se adopta de forma deliberada para poder comparar ResNet18Grasp y SimpleGraspCNN con un protocolo reproducible y con métricas geométricas establecidas, priorizando claridad experimental frente a formulaciones 6-DoF de mayor complejidad.

Además del estudio perceptivo, el trabajo contempla la integración del detector como nodo de percepción dentro de un *pipeline* robótico basado en ROS 2 y su validación inicial en simulación con Gazebo. En el entorno de emulación, un manipulador de 6 grados de libertad (6-DoF) opera sobre una mesa con objetos y recibe observaciones de una cámara RGB-D simulada. Las poses estimadas por el modelo se convierten en objetivos consumibles por el sistema robótico para su inspección y validación cualitativa dentro del entorno de simulación.

Es importante destacar que la integración con un brazo 6-DoF se utiliza como escenario de demostración y trazabilidad (percepción → propuesta de agarre → publicación/consumo en *pipeline* robótico), mientras que la estimación y ejecución completa de agarres plenamente espaciales 6-DoF se considera fuera del alcance experimental principal y se plantea como extensión futura (Platt, 2023; Sundermeyer et al., 2021).

La Ilustración 1-3 resume el flujo completo abordado: adquisición RGB/RGB-D, preprocesado, inferencia mediante los modelos propuestos, evaluación con métrica tipo Cornell y, como validación de la integración en un entorno de emulación, publicación/consumo de la propuesta de agarre dentro de un *pipeline* robótico en ROS 2 y su visualización en Gazebo.

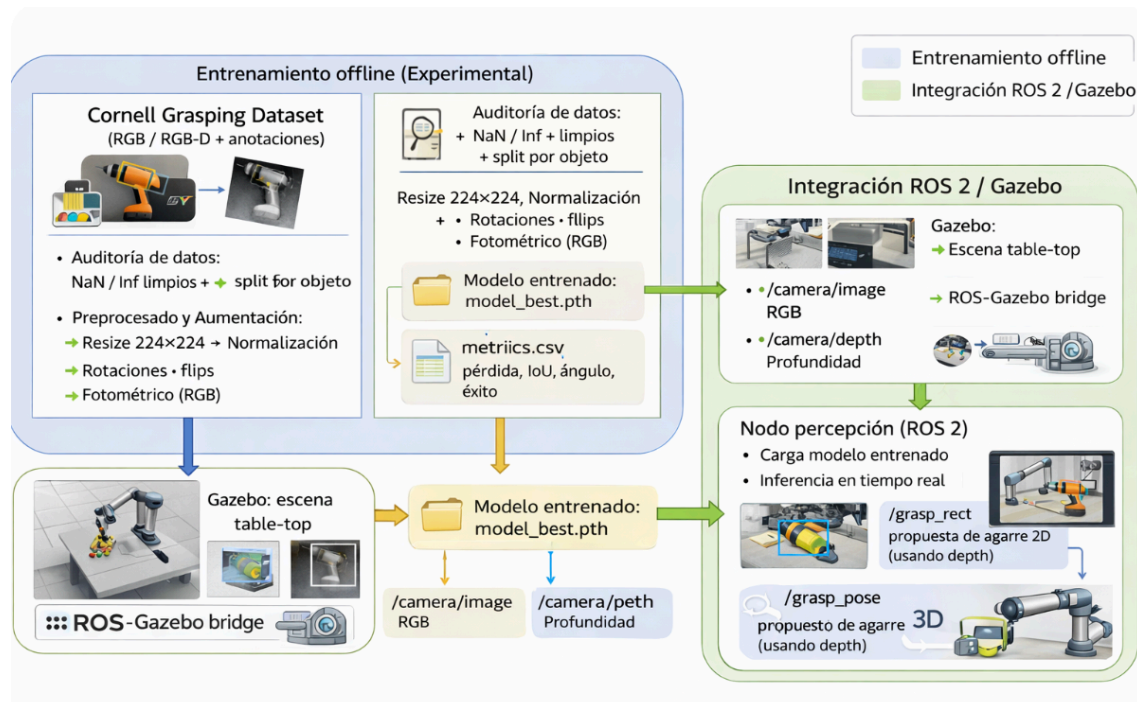


Ilustración 1-3. Diagrama general del pipeline del trabajo e integración.

2. Objetivos y alcance

En coherencia con la motivación expuesta en la introducción, en este capítulo se definen el objetivo general del trabajo y los objetivos específicos que lo desglosan en tareas concretas, verificables y evaluables.

2.1. Objetivo general

Desarrollar un sistema reproducible de detección de poses de agarre 2D/2.5D basado en aprendizaje profundo sobre imágenes RGB-D, entrenado y validado principalmente con el *Cornell Grasping Dataset*, incluyendo la comparación entre un modelo ligero diseñado desde cero y un modelo de referencia basado en ResNet-18 preentrenado, así como la evaluación de su rendimiento, eficiencia computacional e integración como bloque de percepción en un *pipeline* robótico basado en ROS 2 y Gazebo.

2.2. Objetivos específicos

A partir del objetivo general, se plantean los siguientes objetivos específicos:

1. Analizar el problema de la detección de poses de agarre 2D/2.5D en escenarios *table-top*, incluyendo su encaje en un *pipeline* robótico reproducible
2. Preparar el *Cornell Grasping Dataset* y el *pipeline* de preprocesamiento/augmentación para entrenamiento y evaluación reproducibles.
3. Implementar el modelo ResNet18Grasp en modalidades RGB y RGB-D, cuantificando rendimiento y coste computacional.
4. Desarrollar el modelo SimpleGraspCNN como alternativa ligera a ResNet18Grasp, caracterizando su rendimiento y coste computacional.
5. Definir un protocolo experimental reproducible (semillas, particionado, métricas y registros) para comparar modelos y configuraciones.
6. Evaluar los modelos de detección de agarres en términos de rendimiento predictivo y eficiencia computacional, mediante un protocolo reproducible.
7. Demostrar la integración del detector en un entorno de emulación ROS 2 + Gazebo como nodo de percepción, verificando la publicación/consumo de poses estimadas.

La Ilustración 2-1 sintetiza el mapa de trazabilidad del trabajo, conectando los objetivos específicos con el flujo metodológico completo: (i) preparación y auditoría del conjunto de datos (*dataset*), (ii) diseño y entrenamiento de los modelos en modalidades RGB y RGB-D, (iii) evaluación cuantitativa y cualitativa mediante métrica tipo Cornell, (iv) análisis de eficiencia computacional y (v) validación de la integración en un entorno de emulación basado en ROS 2 y Gazebo.

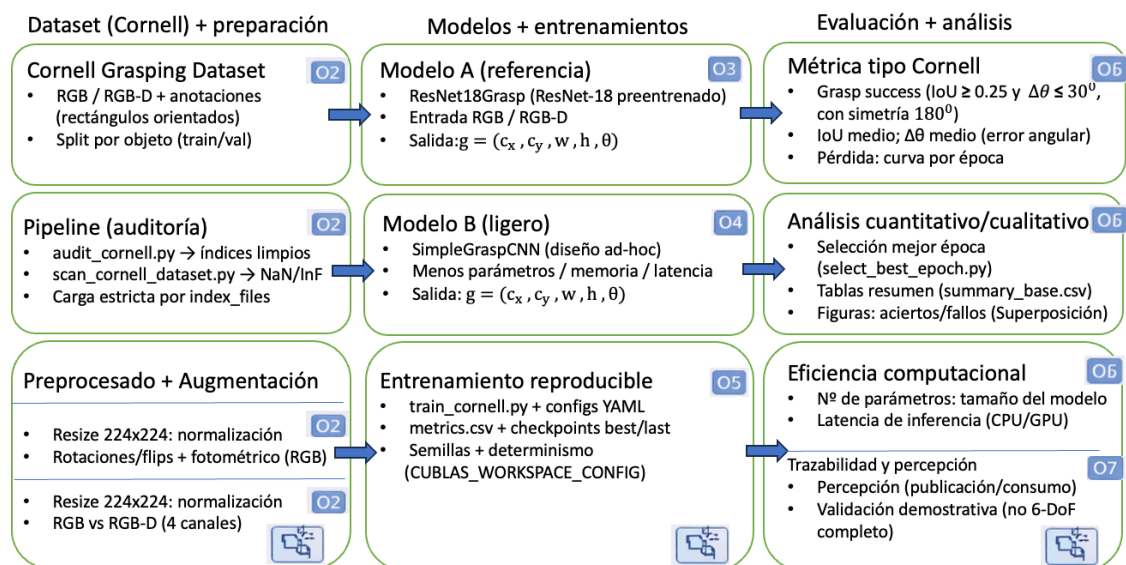


Ilustración 2-1. Mapa de trazabilidad del pipeline (dataset → modelo → métricas → ROS 2/Gazebo). Diagrama de trazabilidad elaborado para este trabajo. Muestra la conexión entre los objetivos específicos (O1–O7) y el flujo metodológico: preparación del Cornell Grasping Dataset (incl. auditoría con índices limpios), preprocesado y aumento de datos; entrenamiento de los modelos (ResNet18Grasp y SimpleGraspCNN) con registro de métricas; evaluación tipo Cornell (IoU y error angular), análisis cualitativo y de eficiencia; e integración demostrativa mediante un nodo de percepción en ROS 2 con visualización en Gazebo.

La Tabla 2-1 presenta la trazabilidad entre cada objetivo específico y las evidencias asociadas en el documento (Capítulos, Ilustraciones/tablas de resultados). Su finalidad es facilitar la verificación del cumplimiento de los objetivos:

ID.	Objetivo (resumen)	Capítulo donde se cumple	Evidencias recomendadas (figuras/tablas)
O1	Estado del arte	3	Tabla comparativa 2D/2.5D vs. 6-DoF (3) y citas completas
O2	Preparación de Cornell particionado por objeto (object-wise) y auditoría	4.2–4.3	Tabla de particionado y recuentos + referencia a scripts
O3	Modelo A: ResNet18Grasp	4.6.1	Ilustración de arquitectura + tabla de hiperparámetros
O4	Modelo B: SimpleGraspCNN	4.6.2	Ilustración de arquitectura + tabla de parámetros
O5	Protocolo y métricas tipo Cornell	4.7–4.8	Tablas 4-5, 4-6 y 4-7 + figuras/tablas del Cap. 5
O6	Evaluación cuantitativa y cualitativa	5	Tablas 5-1, 5-2 y 5-3 + galería cualitativa
O7	Integración ROS 2 + Gazebo	4.9	Diagrama de nodos + captura en Gazebo

Tabla 2-1. Trazabilidad entre objetivos específicos y evidencias en el documento.

2.3. Alcances y limitaciones

Este trabajo se centra de forma deliberada en la percepción visual para manipulación, concretamente en la detección de poses de agarre a partir de imágenes RGB-D. Con el fin de mantener un alcance evaluable, el estudio se formula experimentalmente como detección de poses de agarres 2D/2.5D mediante rectángulos orientados en el plano imagen, compatibles con una pinza paralela y apropiados para escenarios predominantemente planares (por ejemplo, manipulación sobre mesa) (Jiang et al., 2011). La integración con un brazo 6-DoF se aborda como un demostrador de arquitectura (percepción → publicación ROS 2 → consumo en *pipeline*), mientras que la estimación y ejecución completa de agarres 6-DoF queda planteada como extensión futura (Platt, 2023).

En consecuencia, el trabajo no incluye experimentación sobre:

- control de bajo nivel del manipulador,
- planificación de trayectorias del efector final,
- ejecución física de agarres en un robot real.

Cuando se haga referencia a estos elementos, se hará desde una perspectiva de integración conceptual y validación en simulación, no como evaluación completa en hardware físico.

Desde el punto de vista de los datos, el estudio se apoya principalmente en el *Cornell Grasping Dataset* por su uso extendido y por disponer de métricas estandarizadas (Jiang et al., 2011). Aunque existen *datasets* de mayor escala y orientados a agarres 6-DoF (por ejemplo, basados en nubes de puntos), su inclusión se considera fuera del alcance por el coste de ingeniería y cómputo asociado, y se reserva para trabajo futuro (Kleeberger et al., 2020; Xie et al., 2023).

Finalmente, el número de experimentos y combinaciones de hiperparámetros se encuentra condicionado por el presupuesto de cómputo disponible. Por ello, se prioriza un *pipeline* reproducible y una comparación controlada entre modelos frente a una búsqueda exhaustiva. Aun así, el análisis experimental se plantea con criterios rigurosos, apoyándose en métricas estándar, resultados cualitativos y discusión crítica de limitaciones y vías de mejora (Platt, 2023).

La Tabla 2-2 delimita el alcance del trabajo, diferenciando los componentes que quedan explícitamente fuera del estudio (p. ej., control, planificación y validación en robot real) de las extensiones futuras consideradas (p. ej., ampliación del entrenamiento, optimización de hiperparámetros, cuantización, uso de *datasets* más complejos y transición gradual hacia agarres 6-DoF). Esta separación permite clarificar las fronteras del trabajo y contextualizar correctamente los resultados.

Área / componente	Fuera de alcance (qué NO se evalúa)	Extensiones futuras (qué se podría abordar)
Ejecución física en robot real	No se realiza experimentación en hardware físico (agarres reales, calibración real, fallos mecánicos, seguridad).	Validación en robot real (p. ej., UR5/UR5e) con protocolo de seguridad, repetibilidad y evaluación de éxito en el mundo real.
Control de bajo nivel	No se implementa control de motores, pinza ni lazo de control del efector final.	Integración con controladores y estrategias de control (control por impedancia/admitancia, servo-visión, y control en lazo cerrado con realimentación)
Planificación de trayectorias	No se desarrolla planificación completa (pre-agarre, aproximación, retirada, evitación de colisiones).	Integración con planificadores (p. ej., MoveIt 2) y validación en escenas con colisiones, trayectorias y restricciones cinemáticas/dinámicas.
Transformación geométrica completa a 6-DoF	La salida del modelo es 2D/2.5D (rectángulo en plano imagen); no se resuelve el problema completo 6-DoF.	Estimación 6-DoF a partir de RGB-D/nube de puntos, generación de candidatos 6-DoF y verificación de colisiones (p. ej., enfoques tipo Contact-GraspNet (6-DoF)).
Datasets fuera de Cornell	El entrenamiento y la validación se centran en Cornell (dataset de referencias/benchmark 2D/2.5D).	Incorporación de datasets más complejos o a gran escala (Jacquard para 2D; GraspNet/ACRONYM para 6-DoF) y evaluación cruzada cambio de dominio (domain shift)
Búsqueda exhaustiva de hiperparámetros	No se realiza exploración amplia por coste computacional/tiempo; se prioriza una comparación A/B controlada.	Búsqueda sistemática (rejilla/aleatoria/bayesiana), análisis de sensibilidad y escalado a más semillas/experimentos con control estadístico.
Robustez avanzada y escenas desordenadas (clutter) complejos	No se evalúa de forma extensiva robustez ante fondos muy ruidosos, oclusiones severas o escenarios multiobjeto complejos (más allá del demostrador).	Protocolos específicos de robustez (escenas desordenadas (clutter) complejos, iluminación variable, oclusiones) y evaluación en bancos de pruebas (benchmarks) orientados a fondos ruidosos.
Optimización para despliegue (plataformas embebidas (en el borde, edge))	No se implementa un paquete de despliegue altamente optimizado (aunque se analiza eficiencia).	Cuantización (INT8), poda y destilación, exportación a ONNX/TensorRT y bancos de prueba (benchmarks) estandarizados de latencia (CPU/GPU).
Integración robótica: demostrador vs. sistema completo	La integración en ROS 2/Gazebo se utiliza como demostrador de arquitectura y trazabilidad, no como solución completa de extremo a extremo.	Pipeline completo percepción → planificación → ejecución, con métricas de sistema (tiempo total, tasa de éxito final, fallos por módulo).

Tabla 2-2. Fuera de alcance vs. extensiones futuras.

La delimitación de alcance prioriza un estudio reproducible y una comparación controlada entre ResNet18Grasp y SimpleGraspCNN del detector de agarres 2D/2.5D sobre RGB y RGB-D (principalmente en Cornell), junto con una integración demostrativa en ROS 2 y Gazebo. Las extensiones futuras se orientan a ampliar datos y robustez, optimizar el despliegue en hardware con recursos limitados y avanzar hacia agarres 6-DoF y validación en robot real.

3. Estado del Arte y Marco teórico

3.1. Introducción y alcance del capítulo

En este capítulo se presenta el estado del arte y el marco teórico relacionados con la detección de poses de agarre mediante aprendizaje profundo, con especial atención a observaciones RGB-D en entornos no estructurados. El objetivo es situar este trabajo en las líneas de investigación actuales, identificando: (i) tendencias dominantes, (ii) datasets y protocolos de evaluación relevantes, (iii) familias de modelos representativas, y (iv) el papel de la eficiencia computacional cuando se requiere operar con latencias reducidas o bajo restricciones de recursos (Kleeberger et al., 2020; Xie et al., 2023; Platt, 2023).

La literatura reciente muestra que el aprendizaje profundo ha desplazado progresivamente enfoques clásicos basados en reglas o geometría explícita en el componente perceptivo del agarre. Sin embargo, persisten retos como la generalización a objetos no vistos, la dependencia de datos etiquetados y la transferencia entre simulación y entorno real (Kleeberger et al., 2020; Platt, 2023). Además, múltiples revisiones subrayan que el despliegue en sistemas reales exige considerar coste computacional, latencia y memoria, dimensiones que no siempre se reportan con la misma profundidad que el rendimiento predictivo (Xie et al., 2023; Platt, 2023).

A efectos de este trabajo, resulta clave distinguir dos formulaciones:

- Detección 2D/2.5D: el agarre se representa en el plano imagen mediante un rectángulo orientado (posición, orientación y apertura), aproximación ampliamente utilizada en *benchmarks* como Cornell (Jiang et al., 2011).
- Detección 6-DoF: el agarre se expresa como una pose espacial completa del efector final, habitualmente a partir de nubes de puntos u otras representaciones 3D (Sundermeyer et al., 2021). Esta segunda formulación se incluye únicamente como contexto para situar el campo; el alcance experimental del presente trabajo se centra en 2D/2.5D con observaciones RGB-D.

Este trabajo se enmarca en la detección 2D/2.5D en imágenes RGB-D como módulo perceptivo, adoptando Cornell como dataset de referencia y la métrica tipo Cornell como protocolo de evaluación. El compromiso entre rendimiento y coste se aborda mediante la comparación entre un modelo de referencia basado en ResNet-18 y un modelo ligero diseñado desde cero. La integración en un *pipeline* robótico basado en ROS 2 y validación en Gazebo se utiliza como demostrador de arquitectura (percepción → publicación → consumo), mientras que la extensión hacia agarres plenamente 6-DoF queda como trabajo futuro (Jiang et al., 2011; Morrison et al., 2020).

La Ilustración 3-1 presenta una taxonomía sintética de las dos formulaciones dominantes en detección de agarres con percepción RGB-D: (i) agarre 2D/2.5D, donde la salida se parametriza como un rectángulo orientado en el plano imagen (centro, dimensiones y ángulo), y (ii) agarre 6-DoF, donde la salida se expresa como una pose completa en $SE(3)$ (*Special Euclidean group* o grupo euclídeo especial en 3D; posición y orientación) del

efector final, normalmente inferida a partir de representaciones 3D (p. ej., nubes de puntos derivadas de profundidad). Esta distinción no es solo representacional: suele implicar diferencias en datasets de referencia, protocolos de evaluación y requisitos de cómputo, lo que condiciona el diseño del sistema cuando se busca operar con latencias reducidas o bajo restricciones de recursos (Jiang et al., 2011; Kleeberger et al., 2020; Platt, 2023).

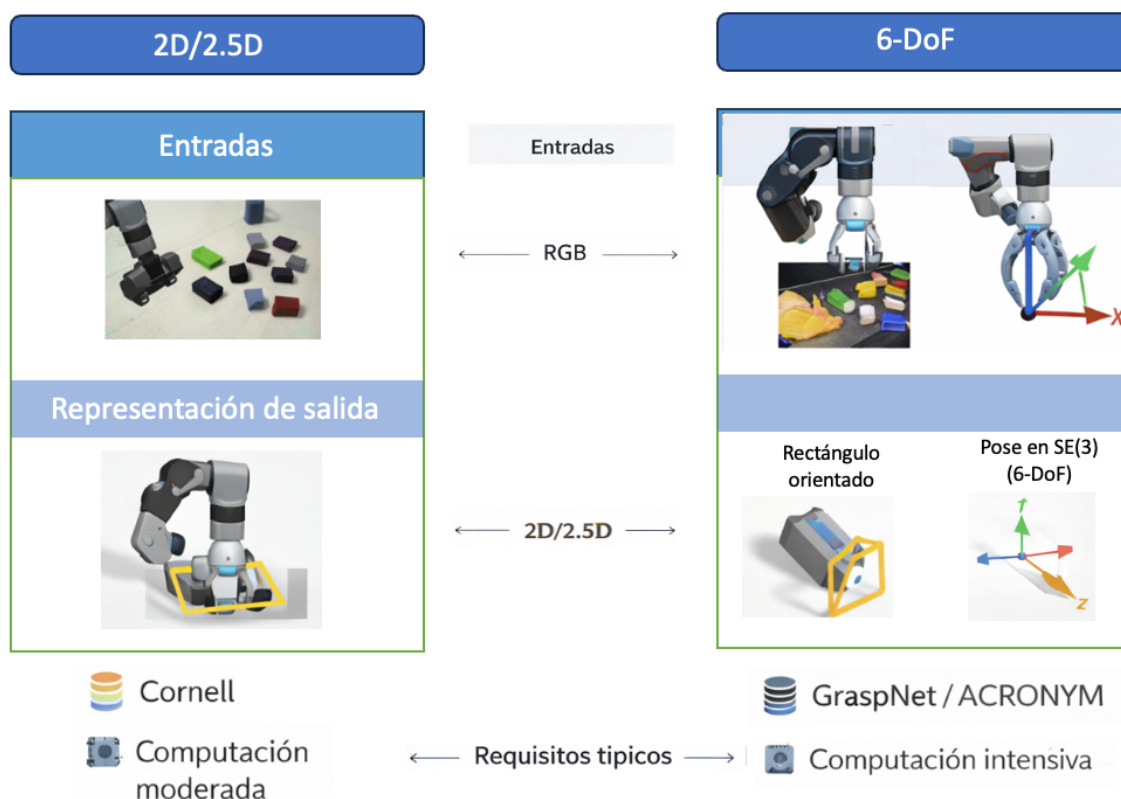


Ilustración 3-1. Taxonomía del problema de agarre: 2D/2.5D vs 6-DoF.

Comparación conceptual entre ambas formulaciones a partir de observaciones RGB-D: en 2D/2.5D la salida se representa como rectángulo orientado en el plano imagen (compatible con protocolos tipo Cornell), mientras que en 6-DoF la salida se representa como pose SE(3) del efector final, típicamente apoyada en geometría 3D y verificación de colisiones. En términos generales, los enfoques 6-DoF tienden a requerir mayor volumen de datos y mayor coste computacional (p. ej., benchmarks a gran escala y modelos sobre nubes de puntos), mientras que 2D/2.5D facilita el estudio del compromiso precisión-eficiencia en escenarios “table-top”. (Elaboración propia basada en Jiang et al., 2011; Fang et al., 2020; Eppner et al., 2020; Sundermeyer et al., 2021; Platt, 2023).

3.2. Avances recientes en detección de poses de agarre en RGB-D

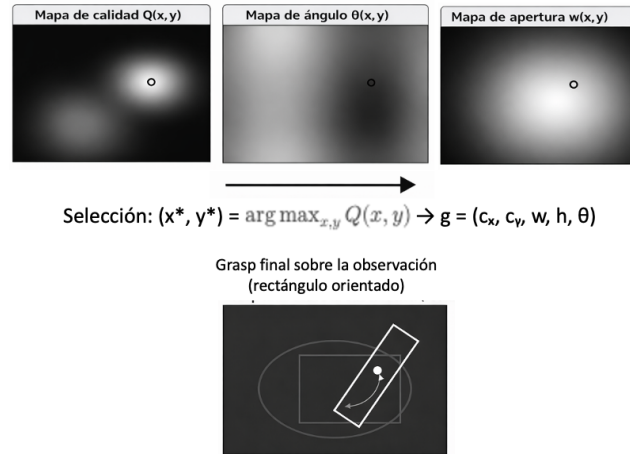
3.2.1. Métodos 2D/2.5D basados en mapas densos de agarre

Una línea consolidada formula la predicción de agarres 2D/2.5D como un problema *pixel-wise*, donde se generan mapas densos que parametrizan para cada píxel la calidad del agarre, la orientación y la apertura de la pinza. A partir de estos mapas se seleccionan los candidatos más prometedores (Morrison et al., 2020).

Un trabajo representativo es GG-CNN, que aprende una correspondencia directa entre la observación de profundidad y mapas densos de agarre, con el objetivo explícito de reducir latencia y habilitar control reactivo en lazo cerrado (Morrison et al., 2020). Esta formulación es especialmente relevante cuando se pretende operar con restricciones de cómputo y requisitos cercanos al tiempo real.

$$(x^*, y^*) = \arg \max_{x,y} Q(x, y) \quad (3.1)$$

La Ilustración 3-2 muestra una representación conceptual del enfoque de predicción densa popularizado por arquitecturas tipo GG-CNN (Morrison et al., 2020), en el que la red estima, para cada píxel de la imagen (RGB o profundidad), un conjunto de mapas: calidad de agarre $Q(x, y)$, orientación $\theta(x, y)$ y apertura $w(x, y)$. A partir de estos mapas se obtiene una propuesta de agarre seleccionando el máximo de Q y recuperando en esa posición los parámetros (C_x, C_y, w, h, θ) , lo que permite derivar un rectángulo orientado compatible con evaluaciones tipo Cornell.



Esquema conceptual (no datos reales): predicción densa + selección por máximo en calidad.

Ilustración 3-2. Ejemplo conceptual de mapas densos (calidad, ángulo, apertura) estilo GG-CNN.

Esquema conceptual (no basado en datos reales) para ilustrar la idea de predicción densa: (a) mapa de calidad $Q(x, y)$, (b) mapa de ángulo $\theta(x, y)$ y (c) mapa de apertura $w(x, y)$. El agarre final se selecciona como $(x^*, y^*) = \arg \max_{x,y} Q(x, y)$ y se recuperan en esa posición los parámetros (C_x, C_y, w, h, θ) para construir el rectángulo orientado. (Elaboración propia, basada en Morrison et al. (2020)).

3.2.2. Métodos 2D/2.5D por regresión de parámetros

Además de mapas densos, existen enfoques 2D/2.5D que combinan: (i) extracción de características con *backbones* convolucionales, (ii) cabezas de regresión para parámetros del rectángulo de agarre y (iii) estrategias de fusión RGB-D a diferentes niveles. En esta familia se sitúan aproximaciones basadas en CNN para regresión de parámetros del agarre, evaluadas mediante métricas tipo Cornell (Jiang et al., 2011; Kumra & Kanan, 2017). Aunque hay extensiones posteriores, la literatura reciente tiende a priorizar predicción densa y agarre 6-DoF; por ello aquí se citan trabajos fundacionales y se remite a revisiones actuales (Kleeberger et al., 2020; Xie et al., 2023; Platt, 2023).

En conjunto, los métodos 2D/2.5D resultan atractivos aquí por tres motivos: (1) compatibilidad con Cornell y su evaluación estandarizada, (2) claridad para estudiar el compromiso precisión–eficiencia, y (3) adecuación a escenarios con fuerte componente planar como tareas *table-top* (Jiang et al., 2011; Platt, 2023). Para sintetizar estas dos familias principales (predicción densa mediante mapas frente a regresión paramétrica), la Tabla 3-1 resume su representación, salida típica, ventajas, limitaciones y requisitos de cómputo, con el fin de justificar el enfoque adoptado en este trabajo.

Enfoque	Representación	Salida típica	Ventajas	Limitaciones	Requisitos de cómputo	Ejemplos
Mapas densos (pixel-wise)	La escena se modela como un campo por píxel sobre RGB/Depth; el grasp se define como máximo sobre un mapa	Mapas $Q(x, y)$ (calidad), $\theta(x, y)$ (orientación), $w(x, y)$ (apertura). → selección (argmax Q) y reconstrucción del grasp.	Muy adecuado para <i>table-top</i> ; propone grasps locales y es fácil “ver” dónde el modelo cree que hay agarre; permite múltiples candidatos de forma natural	Requiere postprocesado (argmax, <i>non-max suppression</i> , umbrales); sensible a ruido/artefactos locales; la salida densa puede dificultar una comparación modelos A/B si cambian heurísticas	Inferencia tipo segmentación/fully-conv: coste moderado-alto según resolución; memoria mayor si mapas a alta resolución	GG-CNN y variantes (Morrison et al., 2020)
Regresión paramétrica (por imagen / por región)	El grasp se parametriza directamente como rectángulo orientado (o un conjunto pequeño de parámetros)	(c_x, c_y, w, h, θ) o $(c_x, c_y, w, h, \sin \theta)$ por imagen (o por ROI).	Pipeline simple y reproducible; facilita comparación modelos A/B (misma salida y pérdida); evaluación directa con métrica tipo Cornell; control más explícito del “qué predice” el modelo	Normalmente predice 1 grasp por imagen (si no se extiende); puede perder soluciones alternativas; si la escena es muy “clutter”, un único grasp puede ser insuficiente	Inferencia más ligera: backbone + capas FC / head pequeño; menor memoria que mapas densos	Enfoques tipo Cornell/rectángulo orientado y variantes con CNN/ResNet; regresión directa en RGB o RGB-D (Jiang et al., 2011; Kleeberger et al., 2020)

Tabla 3-1. Comparativa sintética de enfoques 2D/2.5D: mapas densos vs regresión paramétrica. Ambos enfoques producen representaciones 2D/2.5D compatibles con pinza paralela en escenarios *table-top*. En este trabajo se adopta regresión paramétrica para favorecer una comparación entre SimpleGraspCNN y ResNet18Grasp controlada entre arquitecturas y una evaluación homogénea mediante métrica tipo Cornell (IoU y error angular).

3.3. Métodos 6-DoF basados en nubes de puntos

En paralelo, una parte creciente de la investigación se orienta a generar agarres 6-DoF como pose completa del efector final, típicamente a partir de nubes de puntos derivadas de sensores RGB-D. Un ejemplo influyente es Contact-GraspNet, que propone generación eficiente de agarres 6-DoF en escenas con aglomeración/escenas desordenadas (*clutter*) e incorpora criterios relacionados con colisiones en la selección de candidatos (Sundermeyer et al., 2021).

Este avance ha sido impulsado por *benchmarks* a gran escala como GraspNet-1Billion, que proporciona escenas RGB-D y un volumen masivo de agarres asociados para entrenamiento y comparación (Fang et al., 2020). Asimismo, datasets sintéticos como ACRONYM permiten generar colecciones amplias de ejemplos mediante simulación física para pinzas paralelas, favoreciendo diversidad y escalabilidad (Eppner et al., 2020).

Aunque el foco experimental de este trabajo es 2D/2.5D, la línea 6-DoF se incluye como referencia por marcar la dirección hacia representaciones espaciales más completas y por evidenciar el incremento de requisitos de datos e infraestructura asociado (Fang et al., 2020; Platt, 2023).

3.4. Robustez en escenas complejas y fondos ruidosos

Un reto central en entornos no estructurados es la degradación del rendimiento ante fondos complejos, oclusiones, variabilidad de iluminación y presencia de múltiples objetos. Parte de la literatura advierte que evaluar únicamente en escenarios controlados puede sobreestimar la transferencia a contextos reales (Xie et al., 2023).

En esta línea, Cao et al., proponen NBMOD para evaluar explícitamente detección de agarres en fondos ruidosos y escenas multiobjeto, acompañando el dataset con propuestas orientadas a robustez y eficiencia de inferencia (Cao et al., 2023). Este tipo de trabajos refuerza la necesidad de considerar robustez y coste computacional como criterios de diseño, además de la precisión.

En coherencia con estas limitaciones y con el objetivo de interpretar de forma sistemática las degradaciones observadas en condiciones adversas, para anticipar y estructurar la interpretación de los resultados, se presenta un catálogo de fallos esperados que relaciona condiciones visuales adversas con degradaciones típicas de la predicción: incremento del error angular ($\Delta\theta$), disminución del solapamiento (IoU) entre el agarre predicho y la referencia, y confusión en la selección de región/objeto. En este trabajo, dicho catálogo se concreta en cuatro situaciones típicas, resumidas en la Ilustración 3-3, que se retomarán en el capítulo de Resultados para contextualizar tanto las métricas cuantitativas como la evidencia cualitativa (Morrison et al., 2020; Hou et al., 2025).

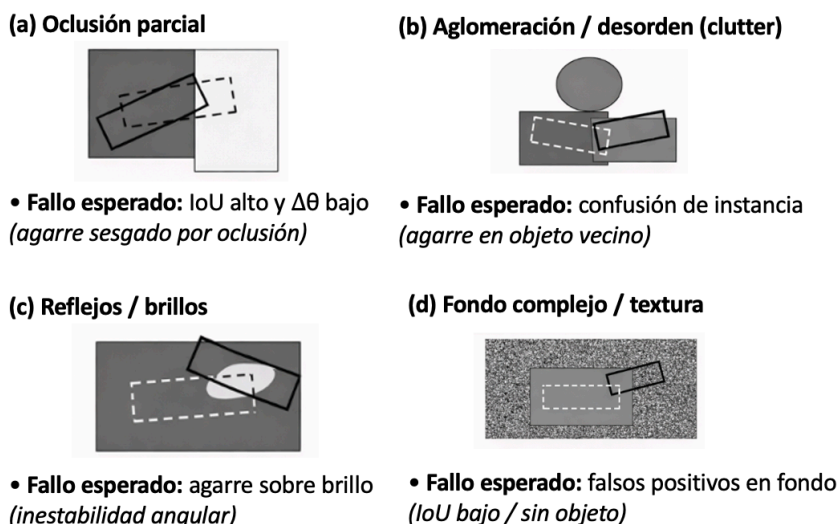


Ilustración 3-3. Ejemplos de degradación por oclusión/fondo: tipología de fallos. Panel con cuatro situaciones típicas: (a) oclusión parcial, (b) clutter/aglomeración, (c) reflejos o brillos especulares, y (d) fondo complejo o altamente texturizado. En cada caso se indica el tipo de fallo esperado en la predicción del agarre (p. ej., $\Delta\theta$ elevado, IoU bajo o confusión al seleccionar un objeto/región incorrecta). La línea discontinua representa el agarre de referencia y la línea continua la predicción ilustrativa. (Elaboración propia, basada en Morrison et al. (2020), Cao et al. (2023), Xie et al. (2023) y Hou et al. (2025))

3.5. Avances recientes en modelos y enfoques de aprendizaje

3.5.1. Atención y Transformers en agarre robótico

En los últimos años se observa un aumento de propuestas que incorporan mecanismos de atención y arquitecturas inspiradas en Transformers para capturar dependencias de mayor alcance y mitigar interferencias del fondo. Como ejemplo, *GraspFormer* propone un enfoque jerárquico guiado por información para mejorar la detección en escenas complejas (Hou et al., 2025). No obstante, estas arquitecturas pueden incrementar el coste computacional, lo que introduce un compromiso explícito entre mejora de rendimiento y latencia/memoria (Song et al., 2025).

En este trabajo se adopta una estrategia alineada con el alcance: comparar un *backbone* moderado (ResNet-18) con un modelo ligero diseñado desde cero, dejando como línea futura la incorporación de módulos de atención ligeros si el coste es compatible con el objetivo de eficiencia (Platt, 2023).

3.5.2. Integración visión–control y aprendizaje por refuerzo

Otra tendencia es integrar percepción y control mediante aprendizaje por refuerzo (RL), especialmente en tareas dinámicas donde la percepción alimenta decisiones de control en tiempo real. En este contexto, *Eye-on-Hand Reinforcement Learner* (EARL) aborda el agarre dinámico combinando estimación activa de pose con una política aprendida en configuración “*eye-on-hand*” (Huang et al., 2023). En general, el aprendizaje por refuerzo aporta un marco sólido para toma de decisiones secuencial bajo incertidumbre, aunque suele ampliar el alcance experimental por los requisitos de simulación y entrenamiento (Sutton & Barto, 2018).

En este trabajo se mantiene la detección de agarres como módulo independiente y evaluable, y la integración con control/RL se plantea como extensión futura (Huang et al., 2023).

3.6. Discusión y posicionamiento del presente trabajo

A partir de los trabajos revisados, pueden destacarse los siguientes puntos para posicionar este trabajo:

- Los enfoques 2D/2.5D basados en rectángulos orientados y mapas densos siguen siendo relevantes por su relación rendimiento–coste y su compatibilidad con evaluación tipo Cornell; GG-CNN es una referencia orientada a latencia reducida (Jiang et al., 2011; Morrison et al., 2020).
- La investigación 6-DoF avanza con fuerza gracias a modelos basados en nubes de puntos y datasets a gran escala (p. ej., GraspNet-1Billion, ACRONYM), pero con

requisitos de datos e infraestructura mayores (Eppner et al., 2020; Fang et al., 2020; Sundermeyer et al., 2021).

- La robustez ante fondos ruidosos y escenarios multiobjeto es un reto reconocido; propuestas como NBMOD enfatizan la importancia de evaluar fuera de escenarios “limpios” (Cao et al., 2023; Xie et al., 2023).
- La atención y los *Transformers* pueden aportar mejoras en escenas complejas, pero introducen un coste adicional que debe considerarse respecto al objetivo de eficiencia (Hou et al., 2025; Song et al., 2025).
- La integración visión–control vía RL es prometedora para manipulación dinámica, pero incrementa el alcance experimental y se mantiene como extensión (Huang et al., 2023; Sutton & Barto, 2018).

En síntesis, este trabajo se sitúa en la intersección entre (i) detección 2D/2.5D en RGB-D con evaluación estándar y (ii) la necesidad práctica de estudiar eficiencia y reproducibilidad. La contribución se centra en comparar un modelo de referencia basado en ResNet-18 con un modelo ligero propio, analizar rendimiento y coste computacional bajo un protocolo reproducible, y definir una vía de integración en un *pipeline* robótico con validación en simulación (Morrison et al., 2020).

Para reforzar la coherencia global del trabajo y facilitar su defensa, se presenta un mapa de posicionamiento que conecta, en una sola vista, las decisiones metodológicas con el marco de referencia (Ilustración 3-4). El diagrama resume cómo el estado del arte (uso de Cornell como benchmark, formulaciones 2D/2.5D y restricciones de eficiencia para despliegue) conduce al diseño comparativo entre ResNet18Grasp y SimpleGraspCNN, se valida mediante métricas estándar Cornell, y culmina con una integración demostrativa en ROS 2 + Gazebo. Esta figura actúa como “puente” entre motivación, metodología, evaluación e implementación, y permite justificar de forma explícita por qué las decisiones del trabajo son consistentes con la literatura y con los objetivos del demostrador.

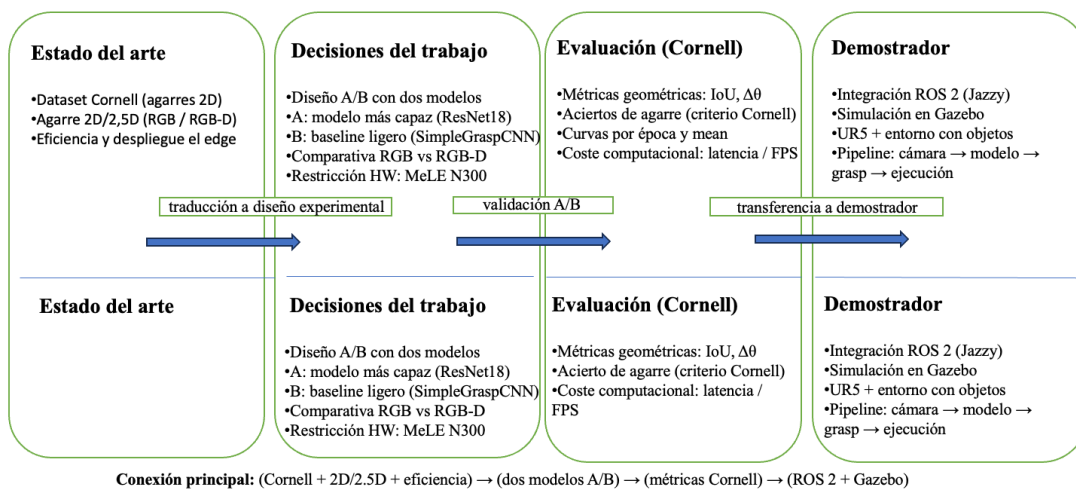


Ilustración 3-4. Mapa de posicionamiento: “estado del arte → decisiones del trabajo”.

Diagrama de trazabilidad que enlaza: (Cornell + formulación 2D/2.5D + eficiencia) → (comparativa entre ResNet18Grasp y SimpleGraspCNN) → (métricas Cornell: IoU, error angular y criterio de acierto) → (integración ROS 2 + Gazebo como demostrador). La figura sintetiza la lógica de diseño experimental y su transferencia a un entorno de validación robótica reproducible.

3.7. Datasets de referencia y protocolos de evaluación

El rendimiento reportado en detección de agarres depende de forma crítica tanto del dataset como del protocolo de particionados empleados, ya que ambos condicionan la representatividad del entrenamiento, el nivel de generalización exigido y la comparabilidad con trabajos previos. En este trabajo se utilizan datasets de referencia para detección 2D/2.5D mediante rectángulos orientados y métrica tipo Cornell; adicionalmente, se citan datasets 6-DoF como contexto del estado del arte.

3.7.1. Datasets para detección 2D/2.5D

El progreso en detección de agarres mediante aprendizaje profundo está estrechamente ligado a datasets estandarizados y métricas comparables. En agarre robótico, la estandarización es especialmente importante porque el rendimiento depende de la representación (2D/2.5D vs 6-DoF), del tipo de sensor y del protocolo de particionado (Kleeberger et al., 2020; Platt, 2023). Los datasets más utilizados y referenciados en la literatura (tanto 2D/2.5D como 6-DoF) se resumen en la Tabla 3-2, siendo los más relevantes para este trabajo:

Dataset	Tipo	Modalidad	Tamaño aproximado	Tipo de anotación	Evaluación típica	Objetivo
Cornell Grasping Dataset	Real	RGB-D	~885 imágenes, 240 objetos, ~8019 agarres	Rectángulos orientados (agarre 2D/2.5D)	Grasp success tipo Cornell (IoU y error angular)	2D/2.5D
Jacquard	Sintético	RGB-D	~54k imágenes, ~11k objetos, ~1.1M agarres	Rectángulos orientados (agarre 2D/2.5D)	Métrica tipo Cornell (IoU/ $\Delta\theta$) y comparativas en detección	2D/2.5D
GraspNet-1Billion	Real	RGB-D + nube de puntos	97,280 imágenes RGB-D y >1B poses de agarre	Poses de agarre (6-DoF) + sistema de evaluación unificado	Evaluación de éxito de agarre mediante cómputo analítico (<i>success/fail</i>)	6-DoF
ACRONYM	Sintético	Mallas/3D (simulación)	~8,872 objetos y ~60M agarres	Agarres 6-DoF generados por simulación (con "calidad" asociada)	Métricas/criterios de calidad en simulación (según configuración)	6-DoF

Tabla 3-2. Datasets relevantes: Cornell/Jacquard (2D/2.5D) vs GraspNet/ACRONYM (6-DoF).

Los tamaños se reportan de forma aproximada según la descripción publicada de cada dataset y pueden variar ligeramente según versión, filtrado o particionado utilizado en cada estudio

- **Cornell Grasping Dataset:** conjunto de referencia para detección 2D/2.5D en RGB-D, ligado a la representación de rectángulos orientados y a la evaluación con umbrales de solape y orientación (Jiang et al., 2011).
- **Jacquard:** dataset sintético a gran escala propuesto para superar limitaciones de tamaño en datasets reales, manteniendo anotación mediante rectángulos 2D (Depierre et al., 2018).

3.7.2. Datasets para agarre 6-DoF

Los datasets 6-DoF incluidos en la Tabla 3-2 se emplean aquí como referencia del estado del arte. Evidencian que aumentar realismo y diversidad suele exigir más datos e infraestructura, pero permite representar mejor la complejidad espacial de agarres completos (Platt, 2023). Como referencia se consideran:

- GraspNet-1Billion: benchmark a gran escala para detección 6-DoF en escenas con clutter (Fang et al., 2020).
- ACRONYM: dataset sintético generado mediante simulación física centrado en pinza paralela, escalable en número de ejemplos (Eppner et al., 2020).

3.7.3. Protocolos de particionado: por imagen vs por objeto (*image-wise* vs *object-wise*)

Una cuestión crítica para interpretar resultados es el tipo de particionado (*split*), ya que determina el grado de independencia entre entrenamiento y validación y, por tanto, el nivel real de generalización que se está evaluando. En el *Cornell Grasping Dataset* se emplean principalmente dos estrategias: particionado *image-wise* y particionado *object-wise*. En Cornell, el particionado *image-wise* permite que aparezcan objetos comunes entre entrenamiento y validación, mientras que el particionado *object-wise* evalúa de forma más estricta la generalización a objetos no vistos. Por ello, al comparar resultados con el estado del arte es imprescindible indicar el protocolo utilizado (Platt, 2023; Xie et al., 2023).

En el particionado *image-wise*, las imágenes se asignan a entrenamiento y validación de forma independiente, de modo que un mismo objeto físico puede estar presente en ambos subconjuntos, aunque observado desde poses distintas o bajo variaciones de iluminación y ruido. Este esquema evalúa principalmente la capacidad del modelo para interpolar variaciones de observación sobre objetos ya conocidos (cambios de pose, pequeñas oclusiones, ruido del sensor). Como consecuencia, suele producir estimaciones de rendimiento más optimistas, ya que el modelo puede beneficiarse de regularidades específicas de objetos vistos durante el entrenamiento (forma global, textura o contornos característicos).

Por el contrario, el particionado *object-wise* separa por identidad de objeto, evitando que los objetos presentes en validación aparezcan en el conjunto de entrenamiento. Este protocolo es más exigente y aproxima mejor el escenario objetivo de manipulación en entornos no estructurados, donde el sistema debe proponer agarres sobre objetos previamente no vistos. En este caso, el rendimiento refleja en mayor medida la capacidad de extraer representaciones geométricas transferibles (p. ej., bordes, simetrías locales o indicios de estabilidad de la pinza), en lugar de apoyarse en la repetición de instancias ya observadas.

Estas diferencias tienen implicaciones directas al comparar con la literatura, porque el valor numérico de la métrica puede depender fuertemente del protocolo. En la práctica, numerosos trabajos reportan resultados en ambos escenarios y muestran que el rendimiento en *object-wise* suele ser igual o inferior al *image-wise*, precisamente por el salto de dificultad asociado a generalizar a objetos no vistos. Por ejemplo, Redmon y Angelova (2015) reportan un desempeño en torno al 85% tanto en *image-wise* como en *object-wise* en Cornell bajo el criterio de rectángulos (métrica tipo Cornell). En trabajos más recientes, también es habitual reportar ambos valores y observar diferencias pequeñas o moderadas entre ellos; por ejemplo, un método en Cornell informa 98.9% (*image-wise*) y 98.3% (*object-wise*), discutiendo además aspectos de eficiencia. De forma

complementaria, revisiones recientes subrayan que estos resultados “de laboratorio” pueden no trasladarse de manera directa a escenarios reales o desordenados, lo que refuerza la necesidad de protocolos bien especificados y comparaciones cuidadosas (Platt, 2023; Xie et al., 2023).

Además, la elección del particionado se relaciona con la propia métrica de Cornell: la evaluación estándar suele considerar una predicción correcta si el rectángulo estimado cumple un umbral mínimo de solape (IoU) y un error angular máximo ($\Delta\theta$), típicamente $\text{IoU} \geq 25\%$ y $\Delta\theta \leq 30^\circ$, aunque existen variantes. Dado que la métrica es relativamente “tolerante” y el particionado puede introducir solapes de identidad de objeto (en *image-wise*), es posible obtener valores altos sin que ello garantice robustez ante objetos realmente nuevos o configuraciones más complejas. En consecuencia, para una comparación rigurosa con el estado del arte, no basta con reportar $\text{IoU}/\Delta\theta$: es imprescindible detallar cómo se construyó el conjunto de validación.

Por todo lo anterior, en este trabajo se considera esencial documentar explícitamente: (i) el tipo de particionado (*image-wise* u *object-wise*), (ii) el procedimiento exacto de construcción del *split* (criterio, semilla y/o lista de índices), y (iii) cualquier filtrado adicional aplicado (p. ej., exclusión de muestras corruptas) para garantizar reproducibilidad e interpretación rigurosa, tal y como recomiendan revisiones del área (Platt, 2023; Xie et al., 2023).

3.8. Marco teórico

3.8.1. Enfoque de aprendizaje supervisado

Este trabajo se formula como un problema de aprendizaje profundo supervisado aplicado a visión por computador: a partir de una observación visual de la escena (RGB o RGB-D), el modelo aprende a predecir parámetros de una pose de agarre, ajustando pesos mediante optimización basada en gradiente (Goodfellow et al., 2016). Esta formulación es coherente con la representación de rectángulos orientados y la evaluación estándar asociada a Cornell (Jiang et al., 2011).

Los modelos se desarrollan y entrenan con PyTorch, por su soporte de GPU, modularidad y utilidades de entrenamiento y checkpointing (Paszke et al., 2019).

3.8.2. Representación del problema: entradas RGB/RGB-D y salida paramétrica

Una imagen RGB-D combina color (R, G, B) con un canal de profundidad $D(u, v)$ que proporciona la profundidad por píxel (relacionada con la geometría de la cámara). La profundidad aporta información geométrica útil, especialmente en escenarios no estructurados (Kleeberger et al., 2020). En este trabajo se consideran dos modalidades de entrada: RGB (3 canales) y RGB-D (4 canales).

La salida adopta una representación paramétrica compacta del agarre como rectángulo orientado:

$$(C_x, C_y, w, h, \theta) \quad (3.2)$$

donde (C_x, C_y) , es el centro del rectángulo en coordenadas de imagen, w y h representan dimensiones asociadas a apertura y longitud útil de contacto, y θ es el ángulo de orientación. Esta representación es estándar en Cornell (Jiang et al., 2011).

En este trabajo, la salida del modelo se representa mediante un rectángulo orientado en el plano imagen, lo que constituye una aproximación 2D/2.5D adecuada para escenarios *table-top* (por ejemplo, cámara cenital y aproximación casi perpendicular al plano de la mesa). Sin embargo, esta parametrización no describe por sí sola un agarre 6-DoF completo. En la integración con manipuladores 6-DoF, el rectángulo debe interpretarse como una hipótesis inicial en el plano imagen, que requiere transformación al sistema de referencia del robot (vía calibración y proyección, típicamente usando profundidad) y enriquecimiento geométrico (estimación de *pose/approach* y restricciones de colisión) para su ejecución segura (Platt, 2023).

Para que el lector pueda interpretar correctamente la salida del modelo y, posteriormente, las métricas y gráficos asociados (p. ej., IoU y error angular), se explicita la parametrización empleada. La Ilustración 3-5 modela el agarre como un rectángulo orientado definido por su centro (C_x, C_y) , sus dimensiones (w, h) y su orientación θ respecto al eje horizontal de la imagen. Esta representación coincide con el formato habitual del dataset Cornell y permite evaluar la calidad geométrica de la predicción mediante medidas de solapamiento y de desviación angular.

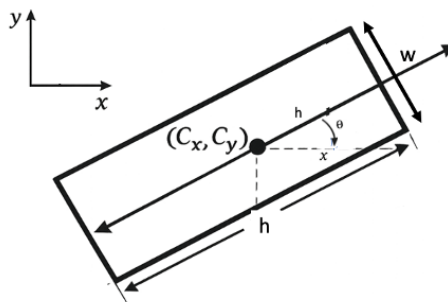


Ilustración 3-5. Parámetros del rectángulo de agarre (C_x, C_y, w, h, θ) sobre una imagen.

El agarre se representa como un rectángulo orientado en el plano imagen: (C_x, C_y) denota el centro en píxeles, w la dimensión perpendicular al eje principal (apertura efectiva en imagen), h la dimensión a lo largo del eje principal (longitud útil de contacto) y θ el ángulo de orientación del eje principal (asociado a h) respecto al eje x de la imagen. (Elaboración propia, basada en Platt, 2023)

3.8.3. CNN y backbones residuales (ResNet) para regresión de agarres

Una red neuronal convolucional (CNN) es un modelo especialmente adecuado para imágenes porque explota dos propiedades estructurales: localidad espacial (los patrones visuales relevantes suelen ser locales) y compartición de pesos (el mismo filtro se aplica a toda la imagen). Esto induce invariancias útiles ante traslaciones pequeñas y reduce el número de parámetros respecto a una red totalmente conectada, favoreciendo la generalización (LeCun et al., 2015; Goodfellow et al., 2016). Los elementos más importantes para considerar en este trabajo son:

Convolución y mapas de características: Dada una entrada X (por ejemplo, RGB o RGB-D apilado como canales), una capa convolucional calcula mapas de características aplicando filtros aprendibles K . Para un canal de salida j , una formulación habitual es:

$$Y_j(u, v) = b_j + \sum_i \sum_m \sum_n K_{j,i}(m, n) X_i(u + m, v + n) \quad (3.3)$$

donde i indexa los canales de entrada, (m, n) recorre el soporte espacial del filtro y (u, v) denota la posición espacial en el mapa de salida. Tras la convolución se aplica una función de activación no lineal:

$$Z = \phi(Y) \quad (3.4)$$

(p.ej. ReLU), que permite aproximar funciones no lineales y construir jerarquías de características: bordes y texturas en capas tempranas, y estructuras más complejas en capas profundas (Goodfellow et al., 2016; LeCun et al., 2015).

Además, mediante *stride* y/o *pooling* se reduce la resolución espacial, lo que aumenta el campo receptivo efectivo (la región de la imagen que influye en una activación), permitiendo capturar contexto relevante para decidir un agarre (posición y orientación relativas en la escena). En entradas RGB-D, la profundidad aporta información geométrica local que puede ayudar a distinguir superficies y bordes útiles para el agarre sin resolver una pose completa en SE(3).

Parametrización del agarre y objetivo de regresión: En este trabajo se adopta la representación planar de agarre como rectángulo orientado:

$$g = (c_x, c_y, w, h, \theta) \quad (3.5)$$

donde (c_x, c_y) es el centro en la imagen, (w, h) capturan apertura/tamaño del rectángulo y θ la orientación. Esta parametrización es compatible con el protocolo tipo Cornell (Jiang et al., 2011) y permite evaluar de forma homogénea configuraciones 2D y 2.5D manteniendo la misma salida g . En términos operativos:

- 2D: la red estima g a partir de RGB.
- 2.5D: la red estima g a partir de RGB-D (profundidad como canal adicional o entrada equivalente), manteniendo la misma salida planar g . La diferencia está en la observación (entrada), no en el espacio de acciones/predicción, que sigue siendo un grasp planar evaluable como rectángulo (Jiang et al., 2011; Platt, 2023).

Backbone residual (ResNet) y motivación: Recordando que en este trabajo se entrenan dos modelos (A y B), para el modelo A (ResNet18Grasp) se utiliza un *backbone* residual (ResNet-18) como extractor de características. La idea central de ResNet es aprender una corrección residual sobre la identidad:

$$y = \mathcal{F}(x; W) + x \quad (3.6)$$

donde x es la entrada del bloque, $\mathcal{F}$ una composición de capas convolucionales (y típicamente normalización y no linealidades), y y la salida. Esta conexión de salto facilita la optimización de redes profundas al mejorar el flujo del gradiente y permite entrenar modelos más profundos y expresivos de forma estable (He et al., 2015).

En el contexto de predicción de agarres, un *backbone* residual ayuda a extraer representaciones robustas ante variaciones visuales, iluminación y textura, lo cual es relevante en escenarios no estructurados o con *clutter* (Kleeberger et al., 2020; Xie et al., 2023). Aun así, mayor capacidad suele implicar mayor coste computacional.

Transfer learning con backbone preentrenado: Se emplea *transfer learning*: se inicializa el *backbone* con pesos preentrenados y se adapta al dominio del agarre mediante ajuste fino (*fine-tuning*). La motivación es que filtros tempranos aprendidos en tareas visuales genéricas capturan primitivas útiles (bordes, texturas) que pueden reutilizarse, reduciendo el riesgo de sobreajuste cuando el dataset es relativamente pequeño (Goodfellow et al., 2016). En términos prácticos, hay dos estrategias típicas:

1. Congelar el *backbone* y entrenar solo la “cabeza” de regresión.
2. Ajuste fino parcial o total: entrenar *backbone* y cabeza con una tasa de aprendizaje menor en el *backbone*.

El ajuste elegido debe justificarse por estabilidad del entrenamiento y rendimiento en validación (Goodfellow et al., 2016).

Cabeza de regresión y comparación A/B con un modelo ligero: Tras el *backbone*, se añade una cabeza de regresión que mapea las características a g . El Modelo B es una CNN

compacta diseñada para ser más eficiente (menos parámetros y menor coste de cómputo), entrenada desde inicialización aleatoria. Manteniendo la misma salida g y el mismo protocolo de evaluación, la comparación A/B mide el intercambio entre:

- Capacidad/precisión (ResNet-18 suele ser más expresiva),
- Eficiencia/latencia (CNN ligera puede ser más rápida y apropiada para control en lazo cerrado o despliegue con restricciones).

Este análisis es especialmente pertinente cuando se busca viabilidad en tiempo real y robustez práctica (Morrison et al., 2020; Platt, 2023).

3.8.4. Métricas geométricas tipo Cornell

La evaluación se apoya en métricas estándar del protocolo tipo Cornell (Jiang et al., 2011): IoU (Intersección sobre Unión), error angular y criterio de acierto (*grasp success*). Estas métricas permiten cuantificar la calidad geométrica de un agarre predicho como rectángulo orientado, comparándolo con los rectángulos de referencia (*ground truth*) anotados.

Definición formal de IoU. Sea R_{pred} el rectángulo orientado predicho y R_{gt} un rectángulo orientado de referencia. La métrica Intersection over Union se define como:

$$\text{IoU}(R_{\text{pred}}, R_{\text{gt}}) = \frac{A(R_{\text{pred}} \cap R_{\text{gt}})}{A(R_{\text{pred}} \cup R_{\text{gt}})} \quad (3.7)$$

donde $A(\cdot)$ representa área. Equivalentemente:

$$\text{IoU} = \frac{A_{\cap}}{A_{\text{pred}} + A_{\text{gt}} - A_{\cap}} \quad (3.8)$$

con $A_{\cap} = A(R_{\text{pred}} \cap R_{\text{gt}})$, $A_{\text{pred}} = A(R_{\text{pred}})$ y $A_{\text{gt}} = A(R_{\text{gt}})$. Para rectángulos orientados, ambos se interpretan como polígonos convexos de cuatro vértices (sus esquinas), y el área de intersección $A_{\cap}$ se obtiene calculando el polígono intersección y su área, de modo que IoU capture simultáneamente discrepancias de posición, escala y orientación.

Definición formal del error angular $\Delta\theta$ con simetría de 180° . Dado que una pinza paralela presenta simetría a 180° en el plano, dos orientaciones separadas por 180° representan el mismo agarre. Por tanto, el error angular se normaliza como:

$$\Delta\theta = \min(|\theta_{\text{pred}} - \theta_{\text{gt}}|, 180^\circ - |\theta_{\text{pred}} - \theta_{\text{gt}}|) \quad (3.9)$$

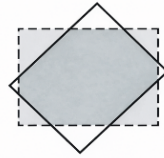
(si se trabaja en radianes, se sustituye 180° por π). Así, $\Delta\theta \in [0^\circ, 90^\circ]$ y cuantifica la discrepancia de orientación de forma consistente con el protocolo tipo Cornell.

Criterio de acierto (*Grasp success*). Una predicción se considera correcta si existe al menos un rectángulo de referencia en la imagen tal que se cumplan simultáneamente:

$$\text{IoU}(R_{\text{pred}}, R_{\text{gt}}) \geq 0,25 \wedge \Delta\theta(R_{\text{pred}}, R_{\text{gt}}) \leq 30^\circ \quad (3.10)$$

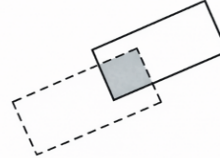
Interpretación conjunta IoU– $\Delta\theta$. IoU y $\Delta\theta$ capturan aspectos complementarios de la calidad geométrica del agarre. IoU mide el grado de solapamiento entre el rectángulo predicho y el de referencia, mientras que $\Delta\theta$ cuantifica la discrepancia de orientación. Es posible obtener un IoU alto con $\Delta\theta$ alto (buen solape, mala orientación) o un IoU bajo con $\Delta\theta$ bajo (buena orientación, bajo solape por desplazamiento). Esta relación se ilustra en la Ilustración 3-6.

(a) Buen solape, mala orientación



IoU = 0.68 (alto)
 $\Delta\theta = +45^\circ$ (alto)

(b) Buena orientación, bajo solape



IoU = 0.13 (bajo)
 $\Delta\theta = 0^\circ$ (bajo)

Criterio Cornell: éxito si $\text{IoU} \geq 0,25$ y $\Delta\theta \leq 30^\circ$ (considerando simetría 180°).

Rectángulo discontinuo: GT
Intersección (IoU): sombreado

Rectángulo continuo: predicción
Intersección (IoU): sombreado

Ilustración 3-6. Interpretación geométrica de IoU y $\Delta\theta$ en rectángulos orientados.

En ambos paneles se compara el rectángulo de referencia (ground truth, GT, línea discontinua) frente al rectángulo predicho (predicción, línea continua). El área sombreada representa la intersección usada para el cálculo de IoU.

(a) Ejemplo con IoU alto, pero $\Delta\theta$ alto (buen solape, mala orientación). (b) Ejemplo con IoU bajo, pero $\Delta\theta$ bajo (buena orientación, bajo solape por desplazamiento). (Elaboración propia).

3.8.5. Métricas de eficiencia computacional

Dado el objetivo de valorar idoneidad para despliegue, la comparación no debe limitarse al rendimiento: se consideran número de parámetros, tamaño del modelo, latencia de inferencia y memoria aproximada durante inferencia (Morrison et al., 2020; Platt, 2023).

En este trabajo, estas magnitudes se reportan junto con las métricas geométricas para comparar el compromiso precisión–eficiencia bajo un protocolo reproducible.

4. Metodología

4.1. Objetivo del capítulo

En este capítulo se describe la metodología seguida para abordar la detección de poses de agarre a partir de imágenes RGB-D mediante aprendizaje profundo. El objetivo es proporcionar una descripción suficientemente detallada para asegurar la reproducibilidad del estudio y ofrecer un marco claro para interpretar los resultados, tanto en términos de rendimiento (métrica tipo Cornell; Jiang et al., 2011) como de eficiencia computacional. Asimismo, se detalla el planteamiento de integración del detector dentro de un pipeline robótico basado en ROS 2 y su validación en un entorno de emulación basado en Gazebo.

Este capítulo se organiza siguiendo el *pipeline* experimental completo, de manera que cada paso deje artefactos verificables (particiones, configuraciones, logs y métricas) y el estudio pueda reproducirse con facilidad (Ilustración 4-1). El flujo parte del *Cornell Grasping Dataset*, incorpora una etapa de auditoría para detectar anomalías (p. ej., etiquetas no finitas) y generar una partición reproducible bajo protocolo *object-wise*, materializada en los archivos *train.csv* y *val.csv*; luego, aplica preprocesado y augmentación controlados, entrena los modelos bajo un diseño comparativo entre ResNet18Grasp y SimpleGraspCNN, y evalúa el rendimiento desde tres perspectivas: métricas estándar Cornell (IoU, $\Delta\theta$ y criterio de acierto), análisis cualitativo mediante superposiciones y catálogo de fallos, y eficiencia computacional (latencia/FPS). Finalmente, se integra el resultado en un entorno de emulación basado en ROS 2 y Gazebo, cerrando el ciclo “percepción → hipótesis de agarre → consumo en el entorno de emulación” (Platt, 2023; Morrison et al., 2020).

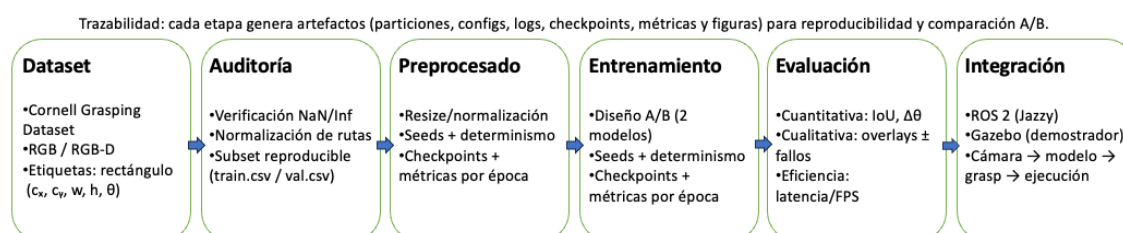


Ilustración 4-1. Visión global del pipeline experimental.

Diagrama de bloques del flujo metodológico: dataset → auditoría → partición *object-wise* reproducible → preprocesado/augmentación → entrenamiento → evaluación cuantitativa, cualitativa y de eficiencia → integración en ROS 2 y Gazebo como demostrador. Cada etapa genera artefactos (particiones, configuraciones, logs, checkpoints, métricas y figuras) que garantizan trazabilidad y reproducibilidad en la comparación entre ResNet18Grasp y SimpleGraspCNN.

4.2. Enfoque general y fases del trabajo

El trabajo se estructura en fases consecutivas y parcialmente solapadas, con el objetivo de mantener un flujo de desarrollo controlado y reproducible:

1. **Revisión bibliográfica y delimitación del alcance.** Se revisa el estado del arte en agarre robótico basado en aprendizaje profundo, distinguiendo detección

- 2D/2.5D sobre RGB-D y enfoques 6-DoF basados en nubes de puntos (Kleeberger et al., 2020; Platt, 2023; Xie et al., 2023). A partir de esta revisión se adopta la formulación 2D/2.5D (rectángulos orientados) por su estandarización en Cornell y por su idoneidad en escenarios “*table-top*”, manteniendo la integración con un brazo 6-DoF como demostrador de arquitectura (Jiang et al., 2011; Platt, 2023).
2. **Selección y estudio del dataset.** Se utiliza el *Cornell Grasping Dataset* como dataset principal, por su uso extendido y por su protocolo de evaluación estandarizado (Jiang et al., 2011). Se adopta particionado *object-wise* para evaluar generalización a objetos no vistos (Platt, 2023; Xie et al., 2023).
 3. **Auditoría del dataset y control de calidad.** Se incorpora un proceso explícito para detectar etiquetas no finitas (NaN/Inf), verificar la integridad de las anotaciones y asegurar que el conjunto utilizado en los experimentos no incorpore muestras corruptas. (Paszke et al., 2019).
 4. **Diseño e implementación de modelos.** Se implementan dos modelos con la misma salida paramétrica para comparar eficiencia frente a capacidad:
 - ResNet18Grasp: arquitectura de referencia basada en ResNet-18 preentrenado, de complejidad moderada.
 - SimpleGraspCNN: CNN ligera diseñada desde cero, orientada a entornos con recursos de cómputo limitados.
 5. **Entrenamiento, registro y evaluación.** Se entrena en régimen supervisado, registrando métricas por época y seleccionando la mejor época a posteriori con scripts de análisis (Goodfellow et al., 2016).
 6. **Validación de la integración robótica.** Se plantea una arquitectura ROS 2 donde el detector actúa como nodo de percepción que consume RGB-D, ejecuta inferencia y publica la pose estimada. Se valida en Gazebo como demostrador funcional del flujo percepción -> publicación -> consumo.

La Tabla 4-1 estructura el capítulo en fases verificables, indicando para cada una de las fases: entrada, salida, script/artefacto responsable y evidencias observables (informes, logs, ficheros generados, experimentos y figuras). Esta organización refuerza la trazabilidad del pipeline (Dataset → auditoría → partición object-wise reproducible → entrenamiento A/B → evaluación → integración) y permite demostrar reproducibilidad: un tercero puede repetir cada fase a partir de los artefactos almacenados (reports/, agarre_inteligente/config/, agarre_inteligente/experiments/ y logs del entorno de emulación ROS 2/Gazebo). El código fuente y los artefactos citados están disponibles en el repositorio del trabajo, TFM_VIU_09MIAR, alojado en GitHub. La versión exacta utilizada en este trabajo, identificada mediante su correspondiente *commit* o etiqueta, se especifica en el *Anexo A. Repositorio y trazabilidad experimental*, con el fin de facilitar la reproducción de los resultados.

Fase	Entrada	Salida	Artefacto / Script	Evidencia verificable
1. Dataset operativo	Cornell raw	Dataset accesible y consistente	agarre_inteligente/data/raw/cornell/	Captura/árbol de directorios + recuento de muestras (mencionado en texto)
2. Auditoría de integridad	agarre_inteligente/data/raw/cornell/ + config de auditoría	Informe de calidad + listados de incidencias	agarre_inteligente/src/graspnet/train/audit_cornell.py	Artefactos de auditoría del dataset generados durante la fase de limpieza y validación de integridad (listados de incidencias y resúmenes JSON)
3. Preparación del split reproducible	Cornell raw + protocolo object-wise	Partición reproducible materializada en train.csv y val.csv	agarre_inteligente/scripts/prepare_cornell_csv.py	agarre_inteligente/data/processed/cornell/splits/object_wise/train.csv y val.csv
4. Configuración reproducible del dataset	YAML experimental + rutas train.csv/val.csv	Dataloaders estables (RGB / RGB-D)	agarre_inteligente/config/*.yaml	Logs/configs del entrenamiento mostrando split object-wise, rutas CSV y tamaños train/val
5. Preprocesado y augmentación	Imágenes + YAML	Pipeline de transformaciones consistente	Definido en agarre_inteligente/config/*.yaml + transformaciones del dataset	Evidencia indirecta: cambios/variación en metrics.csv + descripción metodológica + Tabla 4-3
6. Entrenamiento Modelo B (SimpleGraspCNN RGB)	train.csv + config B	Checkpoints + métricas por época	agarre_inteligente/scripts/train.py + agarre_inteligente/scripts/run_seeds.sh	Experimento EXP1_SIMPLE_RGB: agarre_inteligente/experiments/EXP1_SIMPLE_RGB/metrics.csv, checkpoints/best.pth, last.pth
7. Entrenamiento Modelo B (SimpleGraspCNN RGB-D)	train.csv + config B RGB-D	Checkpoints + métricas por época	Igual que fase 6	Experimento EXP2_SIMPLE_RGBD: agarre_inteligente/experiments/EXP2_SIMPLE_RGBD/metrics.csv, checkpoints/best.pth, last.pth
8. Entrenamiento Modelo A (ResNet18Grasp)	train.csv + config A	Checkpoints + métricas por época	Igual que fase 6	Experimentos EXP3_RESNET18_RGB_AUGMENT y EXP4_RESNET18_RGBD: metrics.csv, best.pth, last.pth
9. Control de determinismo	Entorno + seeds	Ejecuciones más reproducibles	Variable CUBLAS_WORKSPACE_CONFIG + flags deterministas	Log de arranque mostrando seed/modo determinista y ausencia de errores de CuBLAS cuando procede
10. Selección de mejor época y resumen	experiments/*/metrics.csv	Resumen comparativo por experimento	agarre_inteligente/scripts/select_best_epoch.py	reports/metrics/aggregated/chapter5_best_epoch_runs_all_seeds.csv (+ referencia en Resultados)
11. Evaluación cuantitativa (Cornell)	Predicciones + val.csv	IoU, $\Delta\theta$ y "success Cornell"	Evaluador en pipeline + extracción de métricas	Tablas del capítulo de Resultados (incluye IoU/ $\Delta\theta$ /success) + figuras asociadas
12. Evaluación cualitativa (overlays)	Imágenes + GT + pred	Figuras de aciertos/fallos	Script de overlays/galería (generación automática de ejemplos cualitativos desde checkpoints y GT)	Figuras en agarre_inteligente/experiments/figures_* + catálogo cualitativo + Ilustraciones del capítulo 5
13. Evaluación de eficiencia (latencia/FPS)	Modelo exportado/cargado	Latencia media, p95, FPS	Benchmark de latencia (ejecución batch=1 con warm-up; salida a reports/tables/cap5/Tabla_5-3_medicion_de_latencia_de_inferencia_por_experimento_y_dispositivo.csv)	Tabla/figura de latencia y procedimiento + logs de ejecución
14. Validación en entorno de emulación ROS 2 y Gazebo	Modelo + escena simulada	Pipeline robótico operativo	agarre_ros2_ws/src/ur5_qt_panel/ur5_qt_panel/main_panel.py + mundos/modelos Gazebo	Capturas del panel/RViz + logs ros_gz_bridge + tópicos activos (cámara, /clock)
15. Calibración / transformación mesa-robot	Poses Gazebo + cámara	Coherencia píxel→mesa/TF	Calibración leyendo /world/<world>/pose/info (gz topic JSON)	Evidencia: objetos listados con centro en píxeles + proyección estable + selección/overlay coherentes
16. Verificación end-to-end	Todo lo anterior	Validación en entorno de emulación reproducible	Secuencia START ALL + calibración + ejecución	Vídeo/capturas + logs + mención explícita conectando con objetivo específico 7

Tabla 4-1. Fases del trabajo, entradas, salidas y evidencias de verificación.

Nota 1. Los archivos `clean_idx_train*.txt` y `clean_idx_val*.txt` se conservan como artefactos de auditoría y trazabilidad del proceso de depuración del dataset, pero no constituyen el mecanismo activo de carga empleado por el pipeline final de entrenamiento y evaluación.

Nota 2. En la versión efectiva utilizada por los experimentos finales del Capítulo 5, la partición object-wise queda materializada en `agarre_inteligente/data/processed/cornell/splits/object_wise/train.csv` y `val.csv`, con 3542 pares imagen- anotación en entrenamiento y 1569 pares imagen- anotación en validación.

4.3. Dataset y preparación de datos

4.3.1. Cornell Grasping Dataset

El *Cornell Grasping Dataset* está compuesto por imágenes RGB-D de objetos cotidianos capturadas desde distintos puntos de vista. Cada imagen contiene múltiples anotaciones de agarre expresadas como rectángulos orientados, que representan agarres viables para una pinza paralela (Jiang et al., 2011).

Cada agarre puede representarse mediante los parámetros:

$$(C_x, C_y, w, h, \theta) \quad (4.1)$$

donde (C_x, C_y) es el centro del rectángulo, w la apertura asociada a la pinza, h la longitud útil de contacto y θ la orientación. Para pinzas simétricas se considera equivalencia angular por 180° (Jiang et al., 2011).

Para visualizar con claridad la tarea que se aborda en este capítulo, la Ilustración 4-2 muestra un ejemplo del tipo de anotación empleada en Cornell: una misma imagen puede contener varias anotaciones GT (*ground truth*) que representan diferentes agarres válidos sobre el objeto. Cada anotación se expresa como un rectángulo orientado en el plano imagen (C_x, C_y, w, h, θ) , que aproxima la región y orientación donde una pinza paralela podría cerrar con éxito. Esta representación es la base de las métricas (IoU y $\Delta\theta$) y de la evaluación del acierto según el criterio Cornell. En este trabajo, la unidad de muestra para entrenamiento/validación es el par imagen–anotación, no la imagen completa.

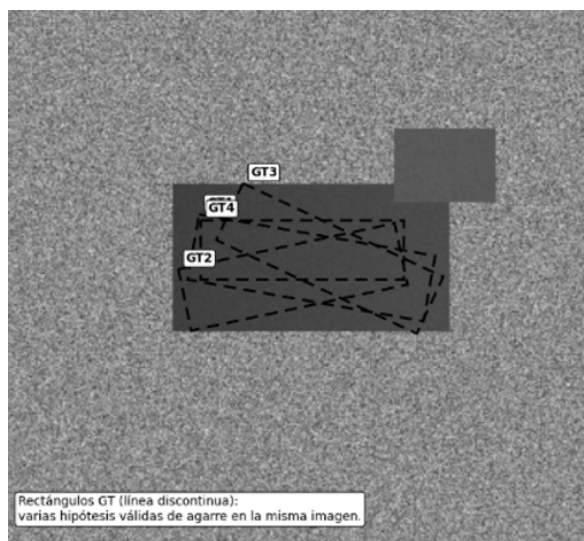


Ilustración 4-2. Ejemplo de anotaciones Cornell sobre una imagen.

Se ilustran varias anotaciones GT (rectángulos orientados) para una misma muestra, reflejando que pueden existir múltiples hipótesis de agarre correctas. Esta configuración define el objetivo de aprendizaje: predecir un rectángulo que sea geoméricamente consistente con alguna de las anotaciones GT (según IoU y $\Delta\theta$). (Elaboración propia).

4.3.2. Particionado object-wise

Para evaluar generalización a objetos no vistos, se adopta el protocolo *object-wise*, en el que los objetos del conjunto de validación no aparecen en entrenamiento. Este protocolo es más exigente que *image-wise* y debe indicarse explícitamente al comparar con el estado del arte (Platt, 2023; Xie et al., 2023).

Por otra parte, en la Tabla 4-2 se muestra el *split* utilizado e indica la cantidad de muestras de cada conjunto. En este trabajo, el criterio de conteo se establece a nivel de muestra del *pipeline*: cada ejemplo del *dataloader* corresponde a un par (imagen, anotación de agarre). Por tanto, los tamaños N reportados en el *split* se refieren al número de pares imagen–agarre, y no al número de imágenes u objetos únicos.

Split	N.º muestras (pares imagen–agarre)	% del total
Train	3542	69,30%
Val	1569	30,70%
Total	5111	100%

Tabla 4-2. Estadísticas del *split* utilizado en el conjunto experimental final.

Los tamaños reportados corresponden al conjunto experimental final estructurado en *train.csv* y *val.csv*, contando pares imagen–agarre.

La trazabilidad experimental se establece explícitamente mediante los ficheros *train.csv* y *val.csv*, generados a partir de un particionado object-wise del *Cornell Grasping Dataset*. Este diseño fija de forma reproducible qué pares imagen–agarre pertenecen a cada partición y evita ambigüedades entre conteos por imagen y conteos por anotación. En el conjunto final utilizado en este trabajo, *train.csv* contiene 3542 muestras y *val.csv* contiene 1569, manteniendo la separación por objetos entre entrenamiento y validación.

4.4. Auditoría y filtrado del dataset

Para asegurar la reproducibilidad del estudio y reducir el riesgo de inestabilidades numéricas durante el entrenamiento, se aplicó un proceso previo de auditoría y saneamiento sobre el *Cornell Grasping Dataset*. Este procedimiento tuvo como finalidad verificar la integridad de las anotaciones, detectar etiquetas no finitas o inconsistentes y dejar trazabilidad explícita de las incidencias encontradas antes de la fase experimental.

La auditoría se planteó como un mecanismo de control de calidad del dato. En particular, se comprobó la validez numérica de las anotaciones de agarre, la consistencia de los ficheros asociados a cada muestra y la ausencia de valores no finitos (NaN o Inf) en los campos empleados por el *pipeline* de entrenamiento y evaluación. Este paso resulta relevante en un contexto de aprendizaje profundo, ya que errores silenciosos en las etiquetas pueden propagarse al entrenamiento y degradar tanto la convergencia como la interpretación posterior de los resultados. Como resultado de este proceso, se generaron artefactos verificables de auditoría, incluyendo listados de incidencias y registros de muestras problemáticas, conservados como evidencia del control de calidad aplicado sobre el conjunto de datos. Estos artefactos permiten justificar que el dataset utilizado en el trabajo fue revisado de forma explícita antes de su explotación experimental y facilitan la repetibilidad del procedimiento sobre el mismo entorno de trabajo.

Conviene señalar que la auditoría y el filtrado constituyen una fase de saneamiento previa, pero no equivalen por sí mismos a la definición final del conjunto experimental. La estructuración reproducible de entrenamiento y validación empleada en este trabajo se estableció posteriormente mediante los ficheros *train.csv* y *val.csv*, generados a partir de un particionado *object-wise* del *Cornell Grasping Dataset*. De este modo, la auditoría actúa como salvaguarda de integridad del dato, mientras que los ficheros de partición constituyen la referencia final utilizada por el *pipeline* experimental.

La Ilustración 4-3 resume este flujo de trabajo: auditoría inicial del dataset, detección y registro de incidencias, y posterior definición del conjunto experimental reproducible. En conjunto, esta estrategia refuerza la robustez metodológica del trabajo, mejora la trazabilidad de los datos empleados y reduce la probabilidad de obtener resultados afectados por muestras corruptas o inconsistencias en las etiquetas.

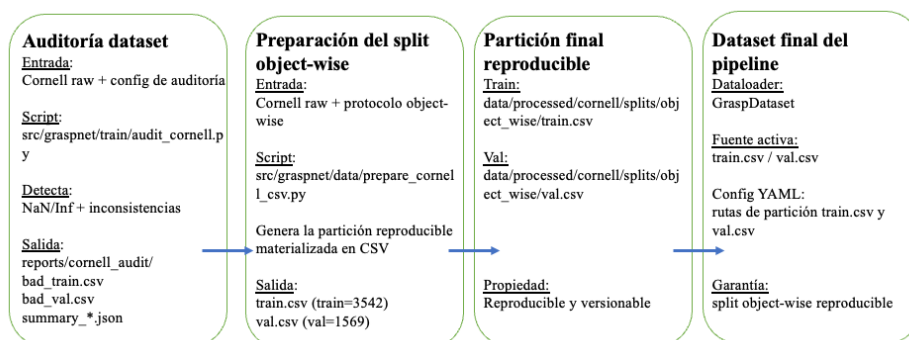


Ilustración 4-3. Flujo del pipeline de preparación del Cornell Grasping Dataset y generación de la partición *object-wise* reproducible.

4.5. Preprocesamiento y data augmentation

4.5.1. Normalización y redimensionado

Las entradas se normalizan y redimensionan para entrenar de forma consistente:

- normalización de RGB a $[0,1]$ y/o estandarización por canal,
- redimensionado de RGB y profundidad a tamaño fijo a 224×224 ,
- construcción de tensores de entrada: RGB (3 canales) y RGB-D (4 canales) (Goodfellow et al., 2016).

4.5.2. Transformación de anotaciones

Las anotaciones se transforman de forma coherente con el redimensionado:

- adaptación de coordenadas al nuevo tamaño,
- conversión desde la representación original (vértices del rectángulo) a (C_x, C_y, w, h, θ) , compatible con la salida de los modelos (Jiang et al., 2011).

4.5.3. Aumento de datos

Se aplican transformaciones en entrenamiento para aumentar diversidad y reducir sobreajuste:

- rotaciones discretas ($0^\circ, 90^\circ, 180^\circ, 270^\circ$) actualizando anotaciones,
- *flips* horizontales/verticales cuando proceda,
- pequeñas traslaciones y escalados,
- ajustes fotométricos moderados (brillo/contraste) sobre RGB.

La configuración de *augmentation* se controla por experimento para medir su impacto de forma aislada. El agarre está definido por un rectángulo orientado en coordenadas de imagen. Para actualizar las etiquetas, es útil expresar el rectángulo mediante centro $\mathbf{c} = [c_x, c_y]^T$, dimensiones (w, h) y orientación θ . En transformaciones geométricas, la actualización se obtiene aplicando la misma transformación a la geometría del rectángulo (idealmente a sus 4 vértices) y re-estimando (C_x, C_y, w, h, θ) .

La Tabla 4-3 detalla las transformaciones y la forma en que se actualizan las etiquetas del agarre representadas como rectángulo orientado. En particular, transformaciones geométricas requieren actualizar centro y orientación, mientras que las fotométricas conservan la geometría y no modifican la etiqueta. Esta explicitación evita ambigüedades metodológicas y ayuda a asegurar que la mejora por *augmentation* no proviene de inconsistencias entre imagen y anotación (Goodfellow et al., 2016). El aumento de datos se aplica de forma online durante el entrenamiento, por lo que no se materializa un nuevo dataset en disco. En consecuencia, los tamaños nominales del conjunto experimental final permanecen fijados en 3542 muestras para entrenamiento y 1569 para validación. Como referencia

conceptual, si solo se considerasen las cuatro rotaciones discretas (0° , 90° , 180° y 270°), el número de vistas efectivas podría multiplicarse, sin contar flips, traslaciones, escalados ni ajustes fotométricos, lo que implica que el número real de variantes observadas durante entrenamiento no sea fijo.

Transformación	Qué hace sobre la imagen	Actualización de (C_x, C_y, w, h, θ)	Nota de verificación / rigor
Flip horizontal	Espeja en eje vertical	$c'_x = (W - 1) - c_x$; $c'_y = c_y$; $w' = w$; $h' = h$; $\theta' = \pi - \theta$ (normalizar)	Normalizar θ' al rango elegido y manejar simetría 180° .
Flip vertical	Espeja en eje horizontal	$c'_x = c_x$; $c'_y = (H - 1) - c_y$; $w' = w$; $h' = h$; $\theta' = -\theta$ (normalizar)	Igual: normalizar θ' y simetría 180° .
Rotación (α) sobre centro	Rota la imagen α grados	$c' = c_0 + R(\alpha)(c - c_0)$; $w' \approx w$; $h' \approx h$; $\theta' = \theta + \alpha$ (normalizar)	Si hay crop/padding al rotar, lo riguroso es transformar los 4 vértices del rectángulo y reajustar.
Traslación (t_x, t_y)	Desplaza en píxeles	$c'_x = c_x + t_x$; $c'_y = c_y + t_y$; $w' = w$; $h' = h$; $\theta' = \theta$	Si recorta y el rectángulo queda fuera, definir política: descartar o recortar etiqueta.
Escalado isotrópico (s)	Zoom in/out	$c' = sc$, $w' = sw$, $h' = sh$, $\theta' = \theta$	Si luego hay resize a tamaño fijo, combinar escalas (evitar doble conteo).
Escalado anisotrópico (s_x, s_y)	Escala distinta en x e y	Recomendado: transformar 4 vértices y reajustar; θ no se conserva en general	Evitar aproximaciones: mejor método por vértices (más correcto).
Crop (recorte) con offset (x_0, y_0)	Recorta una ventana	$c'_x = c_x - x_0$; $c'_y = c_y - y_0$; $w' = w$; $h' = h$; $\theta' = \theta$ (si no hay escalado posterior)	Si el rectángulo queda parcialmente fuera: definir política (descartar o recortar etiqueta).
Resize a (W', H')	Ajusta a tamaño fijo	$c'_x = c_x \cdot (W'/W)$; $c'_y = c_y \cdot (H'/H)$; $w' = w \cdot (W'/W)$; $h' = h \cdot (H'/H)$; $\theta' = \theta$ solo si escala isotrópica	Transformación "siempre presente" si el pipeline fija tamaño. Si $W'/W \neq H'/H$, mejor por vértices.
Brightness/Contrast/Gamma	Cambios fotométricos	No cambia (etiquetas geométricas invariantes)	Útil para robustez a iluminación sin tocar geometría.
Ruido (Gauss/Salt)	Perturba intensidades	No cambia	Igual que fotométricas
Blur	Suaviza, reduce bordes	No cambia	Afecta detección, no etiqueta
Cutout / Oclusión sintética	Ocluye un parche	No cambia, pero puede degradar la validez del grasp	Definir política: descartar o tratar como oclusión sintética y registrarlo en cualitativo.

Tabla 4-3. Transformaciones de data augmentation y cómo se actualizan las etiquetas (C_x, C_y, w, h, θ) .

4.6. Modelos propuestos

Se evalúan configuraciones alineadas con el plan experimental del trabajo:

- EXP1 — SimpleGraspCNN (RGB, línea base): modelo ligero diseñado *ad hoc*, con entrada RGB (3 canales). Se entrena sin aumento de datos o con aumento mínimo, según la configuración definida en la Tabla 4-5.
- EXP2 — SimpleGraspCNN (RGB-D): misma arquitectura ligera, con entrada RGB-D (4 canales) mediante concatenación de la profundidad como canal adicional tras el preprocesado.
- EXP3 — ResNet18Grasp (referencia, RGB + aumento): modelo de referencia basado en ResNet-18 preentrenado y adaptado a regresión de agarres. Se evalúa en modalidad RGB incorporando aumento de datos, con el objetivo de estudiar su efecto en esta modalidad (Tabla 4-5).
- EXP4 — ResNet18Grasp (referencia, RGB-D): misma arquitectura de referencia, evaluada en modalidad RGB-D bajo la configuración indicada en la Tabla 4-5,

para analizar el efecto de incorporar profundidad manteniendo el protocolo experimental.

En conjunto, estos experimentos permiten una comparación controlada entre: (i) modalidad de entrada (RGB vs RGB-D), (ii) uso de aumento de datos (según configuración) y (iii) capacidad del modelo (ligero vs referencia), manteniendo constante la parametrización de salida (C_x, C_y, w, h, θ) y el criterio de evaluación.

4.6.1. ResNet18Grasp

ResNet18Grasp utiliza un *backbone* ResNet-18 preentrenado y una cabeza de regresión para estimar una única hipótesis de agarre por imagen con salida paramétrica:

$$\hat{g} = (\hat{c}_x, \hat{c}_y, \hat{w}, \hat{h}, \hat{\theta}) \quad (4.2)$$

Arquitectura. El extractor de características es ResNet-18 (He et al., 2015) preentrenado en *ImageNet*, está compuesto por una convolución inicial (conv1, 7×7 , stride 2) y *max-pooling*, y cuatro etapas residuales con configuración [2,2,2,2] de bloques *BasicBlock*, con tamaños de canal 64, 128, 256 y 512. A la salida del *backbone* se aplica global average pooling, obteniendo un *embedding* de 512 características.

A diferencia de una formulación con MLP intermedia, en este trabajo se emplea un cabezal de regresión directo, formado por una única capa *fully-connected* que mapea: Linear (512→5). Por tanto, no existe capa FC intermedia (H no aplica) y la regresión es directa 512→5, lo que simplifica la arquitectura y reduce riesgo de sobreajuste en un *dataset* relativamente pequeño.

Entrada del modelo (RGB vs RGB-D). La imagen se re-escala a un tamaño fijo $S = 224 \times 224$ y se normaliza antes de entrar en la red. En la modalidad RGB, la entrada es un tensor $x \in R^{3 \times S \times S}$. En la modalidad RGB-D, la profundidad se incorpora como un cuarto canal, $x \in R^{4 \times S \times S}$. Para soportar 4 canales, se adapta la primera convolución (*conv1*) para aceptar $C=4$ en lugar de $C=3$, manteniendo el resto del *backbone* idéntico. De este modo, la comparación entre modalidades se realiza sin modificar la arquitectura posterior al primer bloque convolucional.

Transfer learning: Se adopta transfer learning, inicializando ResNet-18 con pesos preentrenados (*ImageNet*) y sustituyendo la capa final de clasificación por la capa de regresión Linear (512→5). Esto permite reutilizar características visuales generales aprendidas previamente y concentrar el ajuste en la tarea de regresión de parámetros de agarre. El modelo mantiene aproximadamente 11,2M parámetros totales (orden de magnitud de ResNet-18).

Variantes experimentales: En este trabajo se emplea una única variante estructural de ResNet18Grasp. Las diferencias experimentales se limitan a: (i) modalidad de entrada (RGB vs RGB-D) y (ii) uso de aumento de datos según configuración experimental.

Tratamiento del ángulo y función de pérdida. Aunque en la literatura se propone codificar la orientación mediante $(\sin \theta, \cos \theta)$ para evitar discontinuidades (Goodfellow et al., 2016), en la implementación de este trabajo se mantiene una salida escalar $\hat{\theta}$ y se modela el error angular de forma coherente con la métrica tipo Cornell (simetría a 180°). En particular, se define primero la distancia angular directa:

$$d = | \hat{\theta} - \theta | \quad (4.3)$$

y se utiliza el error angular envuelto:

$$\Delta\theta = \min(d, 180^\circ - d) \quad (4.4)$$

(si se opera en radianes, sustituir 180° por π). La función objetivo se define como una pérdida de regresión sobre parámetros geométricos y orientación:

$$\mathcal{L} = \lambda_c \ell(\hat{c}_x - c_x, \hat{c}_y - c_y) + \lambda_s \ell(\hat{w} - w, \hat{h} - h) + \lambda_\theta \ell(\Delta\theta) \quad (4.5)$$

donde $\ell(\cdot)$ es una pérdida de regresión estándar (p. ej., L_1 o L_2) y $\lambda_c, \lambda_s, \lambda_\theta$ ponderan contribuciones para equilibrar escalas. Como extensión futura, puede sustituirse la regresión directa de θ por una codificación $(\sin \theta, \cos \theta)$ para reforzar continuidad en el espacio de salida.

4.6.2. SimpleGraspCNN

SimpleGraspCNN se diseña con objetivo de eficiencia computacional y simplicidad arquitectónica, manteniendo la misma salida paramétrica que ResNet18Grasp para permitir una comparación directa entre SimpleGraspCNN y ResNet18Grasp. Esta decisión facilita el análisis controlado de las diferencias estructurales entre ambas arquitecturas sin introducir cambios en la representación de salida:

$$\hat{g} = (\hat{c}_x, \hat{c}_y, \hat{w}, \hat{h}, \hat{\theta}). \quad (4.6)$$

Arquitectura. SimpleGraspCNN es una CNN compacta entrenada desde cero (sin transfer learning), con entrada re-escalada a tamaño fijo $S = 224 \times 224$ y un extractor de características formado por $L = 3$ bloques convolucionales. Cada bloque aplica una convolución seguida de normalización y no linealidad, con reducción de resolución mediante max-pooling en los dos primeros bloques. En concreto, los tres bloques emplean 32, 64 y 128 filtros, respectivamente:

- Bloque 1: Conv 7×7 , *stride* 2, $C \rightarrow 32$ + BatchNorm + ReLU + MaxPool(2)
- Bloque 2: Conv 3×3 , *stride* 1, $32 \rightarrow 64$ + BatchNorm + ReLU + MaxPool(2)
- Bloque 3: Conv 3×3 , *stride* 1, $64 \rightarrow 128$ + BatchNorm + ReLU

A la salida del extractor se aplica un pool adaptativo (*AdaptiveAvgPool2d*) a tamaño fijo 7×7 , obteniendo un tensor $[B, 128, 7, 7]$, que se aplana para alimentar una cabeza de regresión con dos capas fully-connected: una capa intermedia de $H = 256$ neuronas con ReLU y una capa final $256 \rightarrow 5$. De forma explícita:

$$\text{Flatten}(128 \times 7 \times 7 = 6272) \rightarrow \text{Linear}(6272, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 5)$$

Este diseño, de menor capacidad representacional que un *backbone* residual preentrenado, reduce memoria y coste de inferencia, lo que resulta relevante en escenarios con hardware limitado (Morrison et al., 2020). El modelo tiene aproximadamente 2,6M parámetros totales.

Con esta configuración, la resolución se reduce de 224×224 a 56×56 tras el bloque 1 (stride 2 + MaxPool), a 28×28 tras el bloque 2 (MaxPool) y se fija a 7×7 mediante *AdaptiveAvgPool2d*.

Entrada del modelo (RGB vs RGB-D). La entrada se re-escala a un tamaño fijo $S = 224 \times 224$ y se normaliza. En modalidad RGB, el modelo recibe $x \in \mathbb{R}^{3 \times S \times S}$. En modalidad RGB-D, la profundidad se incorpora como un cuarto canal, $x \in \mathbb{R}^{4 \times S \times S}$. A diferencia del Modelo A (donde se adapta conv1 por preentrenamiento), en SimpleGraspCNN el primer bloque convolucional se define directamente con $C=3$ o $C=4$ canales de entrada, manteniéndose el resto de la arquitectura sin cambios. De este modo, la comparación entre SimpleGraspCNN y ResNet18Grasp permite aislar el efecto de (i) la capacidad del extractor (compacto vs residual preentrenado) y (ii) la modalidad sensorial (RGB vs RGB-D).

Variantes experimentales: En este trabajo se emplea una única variante estructural de SimpleGraspCNN. Las diferencias experimentales se limitan a: (i) modalidad de entrada (RGB vs RGB-D) y (ii) el uso de aumento de datos según la configuración experimental.

Función de pérdida y tratamiento angular. Para asegurar comparabilidad con el Modelo A (ResNet18Grasp), se utiliza la misma parametrización de salida y el mismo planteamiento de pérdida de regresión sobre (C_x, C_y, w, h, θ) , incluyendo el tratamiento del error angular envuelto $\Delta\theta$ definido en (4.6.1). Con ello, las diferencias observadas en rendimiento y latencia pueden atribuirse principalmente a la arquitectura y a la modalidad de entrada.

La Tabla 4-4 sintetiza las diferencias estructurales entre los modelos SimpleGraspCNN y ResNet18Grasp y su impacto práctico. Al mantener constante la salida, la pérdida y el protocolo de evaluación, la comparación se centra en el efecto del *backbone* (residual preentrenado vs CNN compacta) y de la modalidad de entrada (RGB vs RGB-D).

Aspecto	SimpleGraspCNN	ResNet18Grasp	Implicación práctica
Objetivo	Eficiencia (CPU/GPU modesta)	Máxima capacidad predictiva	Compromiso rendimiento vs coste
Backbone	CNN compacta (diseño ad hoc)	ResNet-18 preentrenada + head de regresión	Transferencia mejora generalización
Capacidad representacional	Baja-media	Media-alta	ResNet18Grasp tiende a capturar mejor variabilidad/estructura; SimpleGraspCNN puede quedarse corto en escenas complejas
Datos necesarios	Más sensible a “poco entrenamiento”	Más estable por preentrenamiento	Con pocas épocas/datos, SimpleGraspCNN puede infra ajustar; ResNet18Grasp suele converger de forma más estable
Entrada	RGB o RGB-D (según configuración)	RGB o RGB-D (según configuración)	Permite comparar “modalidad” (RGB vs RGB-D) manteniendo el resto constante
Salida	(C_x, C_y, w, h, θ)	(C_x, C_y, w, h, θ)	Comparación directa sin cambiar métrica
Entrenamiento	Más rápido	Más costoso	Impacta en tiempo total y consumo
Inferencia	Menor latencia típica	Mayor latencia típica	Relevante para despliegue en edge y tiempo real
Robustez / generalización	Moderada	Mayor (habitual)	En general, ResNet18Grasp presenta una mayor capacidad de generalización, favorecida por el uso de preentrenamiento, mientras que SimpleGraspCNN puede mostrar una mayor sensibilidad a cambios de fondo o iluminación si estos no se compensan mediante técnicas de augmentation.
Despliegue	Muy apto para hardware limitado	Recomendable GPU; en CPU aumenta latencia	Define escenarios de uso y requisitos de cómputo

Tabla 4-4. Comparativa estructural de los modelos (SimpleGraspCNN vs ResNet18Grasp).

4.7. Protocolo de entrenamiento y reproducibilidad

4.7.1. Entrenamiento supervisado

El entrenamiento sigue un esquema estándar de aprendizaje supervisado con optimización por gradiente: se emplea el optimizador Adam, con implementación estándar en PyTorch, y se realiza validación al final de cada época. Este planteamiento es consistente con la práctica habitual en entrenamiento de redes profundas y con el uso de frameworks imperativos reproducibles. En cada época se registran métricas en ficheros metrics.csv, que se usan posteriormente para la selección determinista del checkpoint y el resumen agregado por semilla (Capítulo 4.7.2).

Hiperparámetros y configuraciones experimentales. Para evitar decisiones ad hoc y asegurar trazabilidad, los hiperparámetros de entrenamiento (p. ej., tasa de aprendizaje, tamaño de lote, número de épocas, regularización y nivel de aumento de datos) se fijan por configuración experimental y se documentan en los ficheros YAML asociados, junto con el identificador del experimento. La Tabla 4-5 resume las configuraciones empleadas (modelo, modalidad RGB/RGB-D, aumento de datos, número de épocas, semillas y condiciones de determinismo) y enlaza el YAML que reproduce cada ejecución. En este trabajo no se realiza una búsqueda automática de hiperparámetros; se utilizan los valores definidos en dichas configuraciones para mantener una comparabilidad directa entre ResNet18Grasp y SimpleGraspCNN, así como la reproducibilidad del estudio.

4.7.2. Selección de mejor época

La selección del *checkpoint* final se realiza exclusivamente con métricas de validación, para asegurar comparabilidad homogénea entre modelos y evitar decisiones ad-hoc. Sea $s(e)$ la métrica principal de validación (Success tipo Cornell) en la época e . Se define la mejor época e^* como:

$$e^* = \arg \max_e s(e) \quad (4.7)$$

y se guarda como `best.pth` el modelo correspondiente a e^* . En caso de empate (varias épocas con el mismo máximo), se selecciona la primera época que alcanza el valor máximo, manteniendo un criterio determinista.

4.7.3. Control de semillas y determinismo

Se fijan semillas pseudoaleatorias y, en ejecuciones con CUDA, se activa el modo determinista cuando se requiere reproducibilidad estricta. En particular, al activar `torch.use_deterministic_algorithms (True)` se documenta la configuración necesaria para evitar errores asociados a CuBLAS, exportando `CUBLAS_WORKSPACE_CONFIG=:4096:8`. Esta configuración permite replicar resultados a costa de posibles penalizaciones de rendimiento, por lo que se emplea de forma controlada en el protocolo.

La Tabla 4-5 presenta, para cada experimento, la combinación de modelo (SimpleGraspCNN/ResNet18Grasp), modalidad (RGB/RGB-D), nivel de aumento de datos, número de épocas y semillas utilizadas, junto con el fichero YAML que fija los hiperparámetros de entrenamiento. Esta tabla se utiliza como referencia única para reproducir exactamente cada ejecución y para interpretar los resultados comparativos del Capítulo 5.

Configuración	Modalidad	Modelo	Augment	Épocas	LR	Batch	Semillas	Determinismo	YAML asociado
EXP1 SIMPLE_RGB	RGB	Simple-GraspCNN	No / mínimo	10	Según YAML	Según YAML	0,1,2	Sí (si se activó + CUBLAS)	exp1_simple_rgb.yaml
EXP2 SIMPLE_RGBD	RGB-D	Simple-GraspCNN	Sí (moderado)	10	Según YAML	Según YAML	0,1,2	Sí (si se activó + CUBLAS)	exp2_simple_rgbd.yaml
EXP RES-NET18_RGB AUG	RGB	Res-Net18Grasp	Sí (moderado/fuerte)	50	Según YAML	Según YAML	0,1,2	Sí (si se activó + CUBLAS)	exp3_resnet18_rgb_augment.yaml
EXP4 RES-NET18_RGBD	RGB-D	Res-Net18Grasp	No	50	Según YAML	Según YAML	0,1,2	Sí (si se activó + CUBLAS)	exp4_resnet18_rgbd.yaml

Tabla 4-5. Configuración experimental por experimento.

Para completar la trazabilidad de la Tabla 4-5, a continuación, en la Tabla 4-6, se resumen los hiperparámetros de entrenamiento comunes (tasa de aprendizaje, tamaño de lote, regularización por decaimiento L2 y planificador de LR) utilizados en cada configuración experimental, conforme a los ficheros de configuración asociados.

Configuración	LR	Batch	Weight decay	Scheduler
EXP1_SIMPLE_RGB	0.0005	32	0.0001	None (LR constante)
EXP2_SIMPLE_RGBD	0.0010	32	0.0000	None (LR constante)
EXP3_RESNET18_RGB_AUG	0.0005	32	0.0001	None (LR constante)
EXP4_RESNET18_RGBD	0.0005	32	0.0001	None (LR constante)

Tabla 4-6. Hiperparámetros de entrenamiento por experimento (según YAML asociado).

Estos valores se fijan en el YAML asociado de cada experimento y se mantienen constantes dentro de la configuración correspondiente; no se realizó un proceso de optimización automática de hiperparámetros, con el fin de preservar comparabilidad directa entre experimentos.

4.8. Evaluación cuantitativa, cualitativa y de eficiencia

En presencia de múltiples anotaciones de agarre válidas para una misma imagen, la evaluación no se realiza contra una única referencia arbitraria, sino contra el conjunto completo de ground truth asociados a dicha imagen. En consecuencia, una predicción se considera correcta si existe al menos una anotación GT válida que satisfaga simultáneamente los umbrales establecidos de solapamiento geométrico y error angular. Este criterio de mejor emparejamiento (*best-match*) reproduce de forma fiel el protocolo tipo Cornell en escenarios con varias soluciones de agarre plausibles por imagen.

4.8.1. Métrica principal: grasp success tipo Cornell

Para cada imagen evaluada:

- se genera un agarre predicho (C_x, C_y, w, h, θ),
- se compara con anotaciones válidas,
- se considera acierto si existe al menos una anotación que cumpla simultáneamente:
 - $IoU \geq 0,25$
 - $\Delta\theta \leq 30^\circ$
- en caso contrario, se contabiliza fallo (0).

La métrica *grasp success* del conjunto se define como la proporción de imágenes con acierto sobre el total de imágenes evaluadas.

Se reportan adicionalmente IoU medio, error angular medio, pérdida de validación y curvas por época de pérdida y grasp success para analizar la convergencia (Platt, 2023). Para garantizar reproducibilidad y comparabilidad entre configuraciones, la Tabla 4-7 resume de forma operativa las métricas empleadas en la evaluación del agarre, incluyendo sus definiciones, umbrales y reglas de cómputo.

En particular, se utiliza el criterio de éxito tipo Cornell, que combina solapamiento geométrico (IoU) y consistencia angular ($\Delta\theta$) entre el rectángulo predicho y las anotaciones GT. La Tabla 4-7 recoge, de forma resumida, las definiciones, umbrales y reglas de cómputo empleadas en este trabajo, incluyendo el caso de rectángulos rotados, y que se aplican de manera uniforme en todos los experimentos.

Métrica	Símbolo	Definición	Cómo se computa (rectángulos rotados)	Umbral / uso	Observaciones
Solapamiento	IoU	Medida de solapamiento entre predicción y GT	1) Convertir cada grasp paramétrico (c_x, c_y, w, h, θ) a un polígono (rectángulo) con 4 vértices. 2) Calcular el área de intersección entre polígonos (clipping/intersección). 3) Calcular: $IoU = \frac{A_n}{A_p + A_{gt} - A_n}$	Parte del criterio Cornell	Sensible a desplazamientos pequeños y a tamaño/apertura
Error angular	$\Delta\theta$	Diferencia angular mínima entre predicción y GT	Normalizar el error para considerar periodicidad del rectángulo (simetría a 180°): $\Delta\theta = \min(\delta, \pi - \delta)$ con $\delta = \theta_p - \theta_{gt} $	Parte del criterio Cornell.	Usar grados. La simetría a 180° evita penalizar orientaciones equivalentes.
Éxito tipo Cornell	Success	Predicción correcta si cumple simultáneamente IoU y $\Delta\theta$ con alguna GT válida.	Para cada predicción, se compara con todas las anotaciones GT válidas de la imagen. Se asigna Success = 1 si existe al menos una GT tal que se cumplan simultáneamente $IoU \geq 0,25$ y $\Delta\theta \leq 30^\circ$; en caso contrario, se asigna Success = 0.	Umbrales (Cornell). $IoU \geq 0,25$ y $\Delta\theta \leq 30^\circ$.	Criterio exigente: una predicción “razonable” puede fallar por poco solape o ángulo
Éxito medio en validación	val_success	Media de Success en el conjunto de validación	Proporción media de imágenes correctas en validación, según el criterio Cornell	Métrica principal reportada	Si se expresa en porcentaje: $val_success (\%) = 100 \cdot val_success$.
IoU medio en validación	val_iou	Solapamiento medio (aunque no supere umbral)	Promedio de IoU por muestra (con emparejamiento a GT)	Complementaria	Útil para ver “casi aciertos”
Error angular medio	val_angle_deg	Error angular medio (grados)	Promedio de $\Delta\theta$ (en grados) en validación.	Complementaria	Complementa a IoU para diagnosticar fallos

Tabla 4-7. Definición de métricas de evaluación (Cornell): IoU, $\Delta\theta$ y grasp success

En presencia de múltiples anotaciones GT por imagen, la evaluación tipo Cornell aplica un criterio “best-match”: para cada predicción se escoge la GT que maximiza IoU o minimiza $\Delta\theta$ (según corresponda) y se evalúa el cumplimiento de umbrales, evitando penalizar al modelo cuando existen varios agarres correctos posibles

Una vez definidas formalmente las métricas, la Ilustración 4-4 muestra un ejemplo de evaluación por imagen, donde se superpone el rectángulo predicho (rojo) y la anotación GT (verde), y se reportan IoU y $\Delta\theta$. El objetivo es ilustrar visualmente cómo un agarre puede fallar por solape insuficiente o por error angular, aun cuando el rectángulo resulte razonable a simple vista.

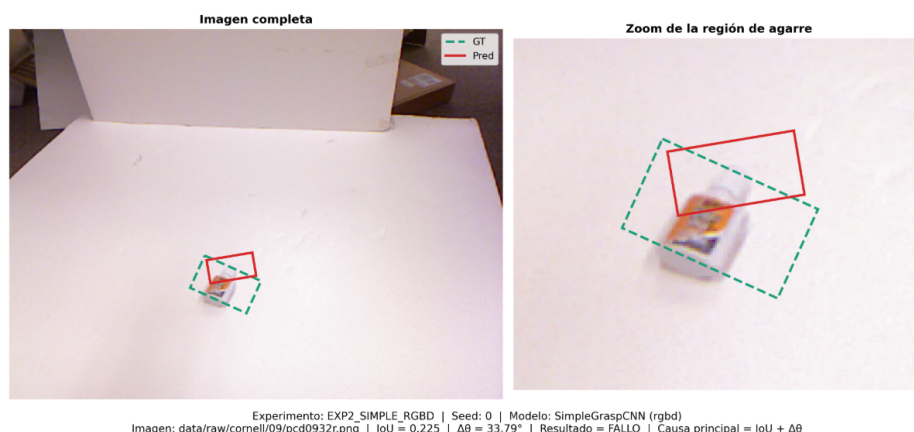


Ilustración 4-4. Ejemplo de evaluación tipo Cornell con fallo por incumplimiento de umbrales de IoU y/o $\Delta\theta$.

4.8.2. Evaluación cualitativa

Se generan visualizaciones superponiendo el rectángulo predicho sobre la imagen, incluyendo ejemplos de:

- aciertos representativos,
- fallos representativos (p. ej., error angular alto, IoU bajo, confusión por fondo/oclusiones).

Estas figuras se incluyen en el capítulo de resultados como apoyo a la interpretación de las métricas agregadas y al análisis de los modos de fallo observados.

Además de las métricas agregadas, se presenta una galería cualitativa con ejemplos representativos para visualizar el comportamiento del modelo sobre la tarea Cornell. En cada caso (véase la Ilustración 4-5) se muestran superpuestos el rectángulo GT (línea discontinua) y la predicción Pred (línea continua), y se etiqueta el resultado como acierto o fallo, indicando la causa principal del error (p. ej., $\Delta\theta$ alto por mala orientación, IoU bajo por desplazamiento, confusión al seleccionar un objeto vecino o falsos positivos en fondo).

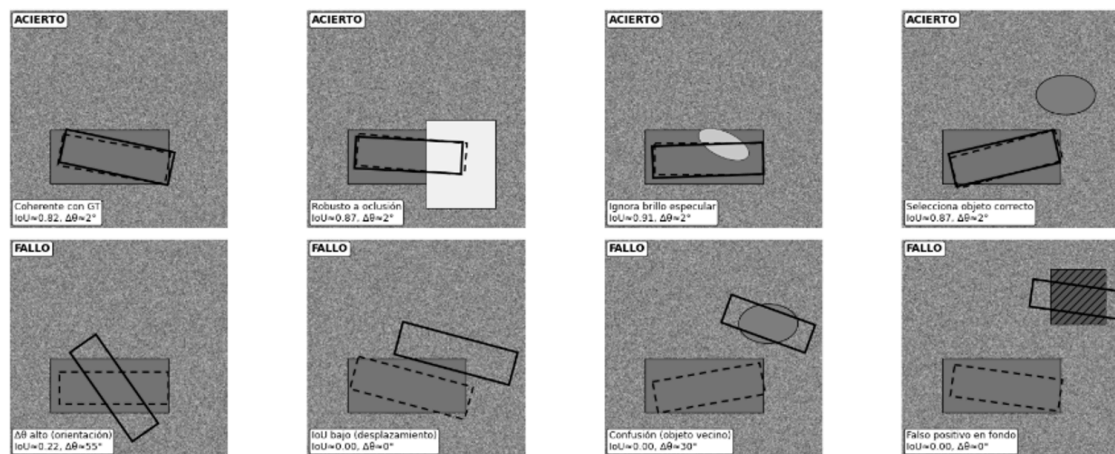


Ilustración 4-5. Galería cualitativa: 4 aciertos + 4 fallos con explicación del tipo de fallo.

Panel 2×4 con superposición de rectángulos de agarre: GT (línea discontinua) y Pred (línea continua). En los aciertos se observa alto solape y baja discrepancia angular; en los fallos se ilustran patrones típicos: $\Delta\theta$ elevado (orientación incorrecta), IoU bajo (desplazamiento/escala), confusión de instancia (agarre en objeto vecino) y falso positivo sobre fondo. Se muestran valores aproximados de IoU y $\Delta\theta$ como guía interpretativa. (Elaboración propia).

4.8.3. Métricas de eficiencia computacional

Se reportan indicadores de eficiencia:

- número de parámetros y tamaño del modelo,
- latencia de inferencia en CPU (y GPU cuando proceda),
- procedimiento reproducible de medida (calentamiento, repeticiones, sincronización si aplica) (Morrison et al., 2020; Platt, 2023).

La latencia se midió en inferencia con $batch = 1$, un escenario representativo de uso robótico en tiempo real, aplicando un periodo de *warm-up* para estabilizar cachés y compilación interna, y repitiendo la medición un número fijo de veces. Se reportan la media y la desviación estándar en milisegundos, así como percentiles para capturar la cola de latencia. En GPU, cada medición se sincroniza con `torch.cuda.synchronize()` para evitar subestimar tiempos debidos a la ejecución asíncrona. Este protocolo permite comparar ResNet18Grasp y SimpleGraspCNN, así como las modalidades ste protocolo permite comparar modelos A/B y modalidades RGB/RGB-D de forma reproducible, independientemente del valor absoluto de rendimiento.

La Tabla 4-8 resume el protocolo de medición de latencia empleado en este trabajo, incluyendo la unidad de medida, el número de iteraciones de *warm-up*, el número de repeticiones registradas, el modo de inferencia y los criterios de reporte (media $\pm$ std, p95 y FPS). Asimismo, explicita los controles aplicados —como la sincronización en GPU y la desactivación de *benchmarking* cuando se busca estabilidad— para garantizar comparabilidad entre ejecuciones.

Elemento	Configuración
Unidad de medida	Latencia por muestra (ms), batch=1
Entrada	Tensor ya preprocesado (mismo tamaño que evaluación)
Warm-up	5 iteraciones (sin registrar)
Repeticiones (N)	20 (registradas)
Sincronización	GPU: <code>torch.cuda.synchronize()</code> antes y después de cada iteración
Modo inferencia	<code>model.eval() + torch.inference_mode()</code>
Criterio de reporte	media $\pm$ std (ms), p95 (ms) y FPS (= 1000 / media)
Control	Fijar <code>torch.backends.cudnn.benchmark=False</code> y modo determinista si se busca estabilidad

Tabla 4-8. Protocolo de medición de latencia.

Los resultados numéricos de latencia (media $\pm$ std, p95 y FPS) se reportan en el Capítulo 5 (Tabla 5-3), manteniéndose aquí únicamente la definición del protocolo reproducible y la trazabilidad al artefacto `reports/tables/cap5/Tabla_5-3_medicion_de_latencia_de_inferencia_por_experimento_y_dispositivo.csv`.

4.9. Diseño de integración robótica (ROS 2 + Gazebo)

Se define una arquitectura de alto nivel basada en ROS 2 y Gazebo para validar la integración entre percepción visual, estimación de agarre y consumo de resultados en un entorno robótico simulado. En este escenario, un brazo robótico de 6 grados de libertad opera sobre una mesa con objetos, cuya disposición es observada mediante una cámara RGB-D simulada. A partir de estas imágenes, el sistema estima una hipótesis de agarre que posteriormente puede ser consumida por módulos de supervisión, visualización y control.

La arquitectura general del entorno simulado y el flujo de percepción–publicación–consumo se resumen en la Ilustración 4-6.

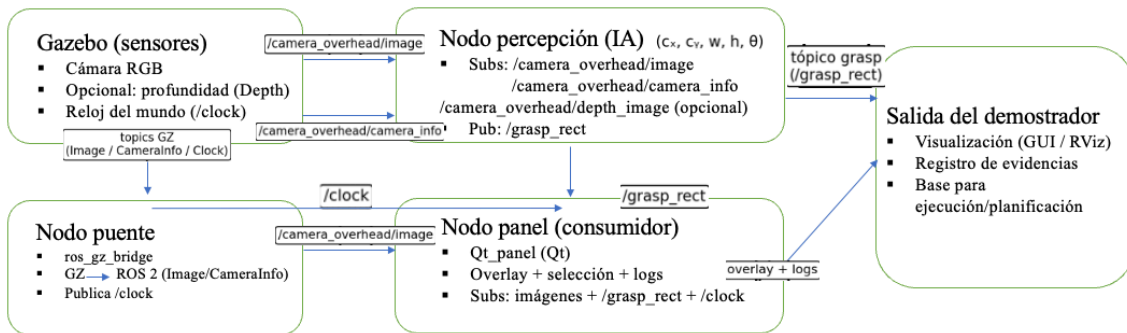


Ilustración 4-6. Arquitectura ROS 2 del entorno simulado (nodos y tópicos).

La validación se plantea inicialmente en simulación con Gazebo, sobre un escenario con mesa, objetos y cámara RGB-D simulada, permitiendo inspección cualitativa del flujo percepción → agarre propuesto. Se resume aquí el flujo observable y verificable de nodos y tópicos: Gazebo genera escena y sensores; *ros_gz_bridge* expone los datos en ROS 2; el nodo de percepción se suscribe a imágenes (RGB y, si aplica, profundidad/*camera_info*) y publica el grasp en forma de rectángulo tipo Cornell (C_x, C_y, w, h, θ); finalmente, un nodo consumidor (panel Qt) se suscribe a imágenes y a */grasp_rect* para superposición (overlay) y registro de evidencias.

La verificación de la integración se realiza mediante comandos estándar de ROS 2: *ros2 node list*, *ros2 topic list* y la inspección de la salida del detector/publicador con *ros2 topic echo /grasp_rect* (y, si aplica, *ros2 topic echo /clock*).

4.9.1. Interfaz de percepción: entradas y salidas

El nodo de percepción define una interfaz mínima y estable para desacoplar el modelo del resto del sistema robótico. Como entradas consume imágenes RGB (y profundidad cuando aplica) junto con la información intrínseca de la cámara (*camera_info*). Como salida publica una hipótesis de agarre en el plano imagen como rectángulo orientado (C_x, C_y, w, h, θ), manteniendo el mismo convenio geométrico que el protocolo tipo Cornell. Esta separación permite:

- sustituir el modelo A/B sin afectar a los consumidores,
- registrar evidencias (imágenes + predicciones) de forma consistente,
- y reutilizar la misma salida para evaluación offline y para ejecución robótica.

En esta arquitectura, */grasp_rect* representa la hipótesis visual en el plano imagen, mientras que */desired_grasp* constituye el canal principal de ejecución hacia MoveIt. Además, */desired_grasp/result* devuelve el estado del procesamiento y */desired_grasp_cartesian* se reserva para variantes cartesianas de descenso o retirada.

4.9.2. Sincronización temporal y reproducibilidad en ROS 2

Para asegurar coherencia entre simulación y procesamiento, el entorno simulado opera bajo reloj simulado (/clock) y mantiene trazabilidad mediante logs y artefactos persistentes. La reproducibilidad del flujo se apoya en: (i) la partición reproducible object-wise materializada en train.csv y val.csv, (ii) configuración por YAML versionada, (iii) ejecución con semillas fijadas, y (iv) registro explícito de configuración de entorno cuando se activa determinismo en GPU.

4.9.3. Proyección imagen→escena: calibración y transformación geométrica

La hipótesis de agarre en coordenadas de imagen requiere proyectarse a un objetivo físico en la escena. En este trabajo, la conversión se plantea combinando: (i) los parámetros intrínsecos de la cámara (tópico *camera_info*), (ii) la profundidad cuando está disponible para estimar escala/distancia y (iii) una relación geométrica entre el plano imagen y el plano de trabajo (mesa) que permite expresar el objetivo en el marco del robot. Este paso es crítico para conectar la hipótesis en imagen con un objetivo coherente en el entorno.

Bajo el supuesto de un escenario *table-top* con superficie de trabajo aproximadamente horizontal, la transformación imagen→mesa se modela mediante una calibración previa que relaciona píxeles con coordenadas sobre el plano de la mesa. Cuando existe profundidad fiable, se utiliza para obtener una estimación métrica local (escala) en torno al punto (C_x, C_y) del rectángulo predicho; cuando no se dispone de profundidad o esta no es estable, la conversión se apoya en la relación plano-plano calibrada (homografía) para mantener consistencia en la proyección. En ambos casos, el objetivo es obtener una posición objetivo estable y repetible en el marco del robot a partir de una hipótesis 2D/2.5D.

La validación de esta etapa se realiza de forma verificable, comprobando: (i) consistencia visual mediante la superposición (*overlay*) del rectángulo predicho sobre la imagen, (ii) estabilidad de la selección del objetivo ante variaciones moderadas de escena y (iii) coherencia geométrica de la posición objetivo al representarla en el escenario de simulación.

Alcance de la validación. La integración descrita tiene un propósito demostrativo y verificable del flujo percepción → publicación → consumo en ROS 2/Gazebo a partir de una hipótesis de agarre planar (2D/2.5D) representada como rectángulo orientado. En particular, no se pretende resolver el cierre completo 6D (estimación de pose completa del efector final y planificación de trayectorias en SE(3) con restricciones de contacto), sino validar la consistencia del *pipeline* y su uso como hipótesis de agarre en un escenario *table-top*. Esta decisión es coherente con el alcance definido en este trabajo (Tabla 2-2) y con el uso de Cornell como *benchmark* 2D/2.5D, priorizando trazabilidad e integración sobre una evaluación física completa del agarre.

Como evidencia visual, la Ilustración 4-7 muestra un ejemplo representativo del proceso, donde se observa simultáneamente la superposición (*overlay*) del rectángulo predicho en la imagen y el consumo de la hipótesis de agarre dentro del flujo percepción → publicación → consumo en el entorno simulado ROS 2 + Gazebo.

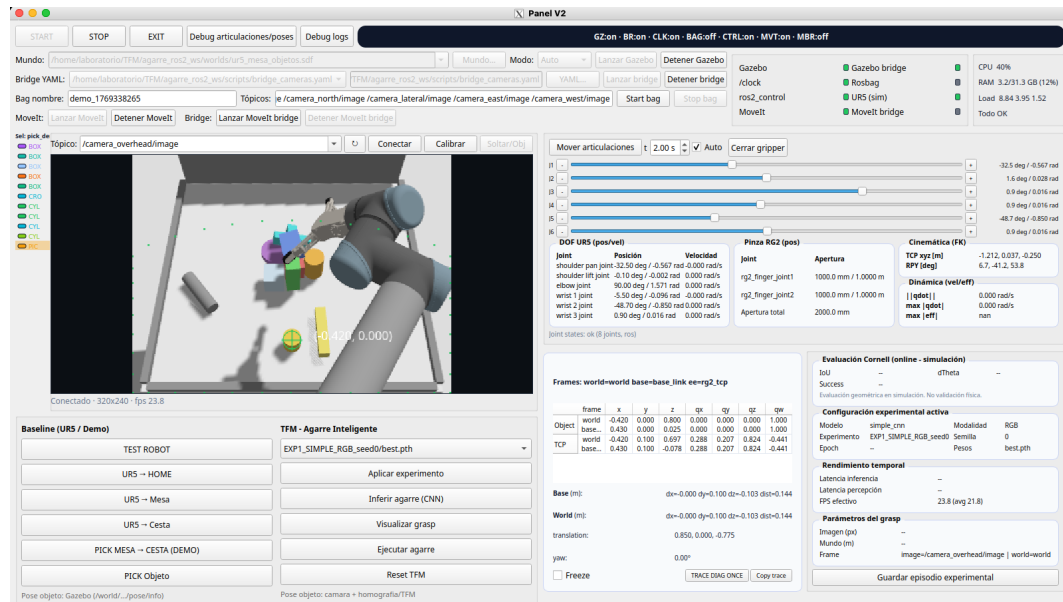


Ilustración 4-7. Entorno de emulación ROS 2/Gazebo: ejemplo de consistencia visual (*overlay*) y consumo de la hipótesis de agarre en la escena table-top.

4.9.4. Planificación y ejecución

Una vez estimada la pose objetivo, en el entorno simulado se contempla un módulo de planificación (p. ej., MoveIt 2), y un módulo de ejecución basado en *ros2_control*. Esta parte se describe a nivel de arquitectura y queda fuera del alcance de la evaluación experimental del presente trabajo. Aunque el foco experimental del trabajo se concentra en percepción, la arquitectura deja definida la cadena completa para que el lector entienda qué se publica, quién lo consume y cómo se valida el funcionamiento extremo a extremo.

4.9.5. Evidencias verificables en el entorno simulado

La integración se considera verificada cuando se dispone de evidencias observables tales como:

- logs del puente ROS 2 ↔ simulación mostrando tópicos activos (imágenes, *camera_info* y *clock*),
- capturas del panel con *overlays* de GT vs predicción, selección y trazas,
- y una ejecución demostrativa reproducible (misma escena, mismo *pipeline*, misma interfaz de mensajes).

4.10. Herramientas software y hardware

4.10.1. Entorno software

- Sistema operativo: Ubuntu 24.04 LTS.
- Lenguaje: Python 3.12 (entorno virtual).
- Framework DL: PyTorch 2.x (Paszke et al., 2019).
- Librerías auxiliares: NumPy, OpenCV/PIL, Matplotlib.
- Control de versiones: Git.
- Middleware robótico: ROS 2 Jazzy.
- Simulación: Gazebo.

4.10.2. Infraestructura hardware

Se distingue entre:

- plataforma objetivo de despliegue/eficiencia (mini-PC basado en Intel N300),
- plataforma de entrenamiento (cuando se emplea GPU para acelerar experimentación).

Los resultados de latencia y eficiencia deben interpretarse en el contexto de la plataforma en la que se miden, por lo que se documenta el hardware exacto utilizado en el trabajo (Platt, 2023). La Tabla 4-9 recoge las especificaciones de las plataformas empleadas, diferenciando el equipo destinado al entorno simulado (ejecución/validación en *edge*) y el equipo utilizado para entrenamiento y evaluación acelerada por GPU. Se incluyen CPU, RAM, GPU y, cuando aplica, versiones relevantes del *stack software* (SO, ROS 2, Gazebo y bibliotecas CUDA/cuDNN) para asegurar trazabilidad y reproducibilidad.

Plataforma	Uso principal	CPU	RAM	GPU	CUDA	cuDNN	Sistema / Software clave
MeLE Over-clock 4C N300	Entorno simulado ROS 2 + Gazebo / validación <i>edge</i>	Intel N300 (4C/4T)	32 GB	iGPU Intel UHD (integrada)	N/A	N/A	Ubuntu 24.04 LTS, ROS 2 Jazzy, Gazebo Harmonic, Python 3.12
PC RTX 4070	Entrenamiento/evaluación DL	Ryzen 7 5800X	32 GB	NVIDIA RTX 4070	12.1	9.1.0	Ubuntu 24.04, Python (venv), PyTorch + CUDA

Tabla 4-9. Especificaciones de hardware utilizadas.

Esta distinción permite interpretar correctamente los resultados de latencia y eficiencia, separando el contexto de entrenamiento/evaluación acelerada del contexto de ejecución/validación en *edge*.

4.10.3. Artefactos generados y estructura reproducible

La metodología se apoya en artefactos persistentes que permiten reproducir el estudio sin ambigüedades. En particular:

- Particiones reproducibles de entrenamiento y validación (train.csv y val.csv), junto con artefactos de auditoría y control de calidad del dataset.

- Configuraciones YAML por experimento (modalidad RGB vs RGB-D, augmen-
tación, hiperparámetros y rutas de partición train.csv/val.csv).
- Logs y métricas por época (metrics.csv) y checkpoints (best/last) para comparar
modelos de forma homogénea.
- Resúmenes y figuras (curvas y galerías cualitativas) que documentan convergen-
cia y patrones de fallo.
- Evidencias en el entorno simulado (logs del puente, capturas del panel/UI, y trazas
de ejecución), que conectan resultados de percepción con validación robótica.

Esta organización materializa la trazabilidad del *pipeline* (Ilustración 4-1) y evita que decisiones críticas queden implícitas o no verificables.

4.11. Resumen del capítulo y garantías de reproducibilidad

Este capítulo ha definido un *pipeline* metodológico completo, trazable y orientado a la reproducibilidad, articulado en seis bloques: (i) selección y caracterización del *Cornell Grasping Dataset*, (ii) auditoría y control de calidad de las anotaciones, (iii) preparación del conjunto experimental reproducible mediante las particiones train.csv y val.csv, (iv) preprocesado y aumento de datos controlados, (v) entrenamiento supervisado y evalua-
ción cuantitativa, cualitativa y de eficiencia, y (vi) validación de la integración robótica en un entorno de emulación basado en ROS 2 y Gazebo.

La metodología adoptada mantiene la coherencia entre el plano perceptivo y el plano experimental. Por una parte, define de forma explícita cómo se representan las hipótesis de agarre, cómo se preparan las entradas RGB y RGB-D y cómo se comparan los modelos bajo una misma salida paramétrica. Por otra, fija un protocolo de evaluación homogéneo basado en el criterio tipo Cornell, incluyendo el tratamiento correcto del caso en que una misma imagen dispone de múltiples anotaciones GT válidas, evaluando cada predicción frente al conjunto completo de referencias asociadas a dicha imagen.

La reproducibilidad del estudio se garantiza porque:

- el conjunto experimental queda definido explícitamente mediante particiones re-
producibles y versionables (train.csv y val.csv), evitando particionados implícitos;
- la auditoría del dataset se conserva como evidencia de control de calidad previo
sobre las anotaciones y la integridad del dato;
- la configuración experimental se fija mediante archivos YAML versionados y se-
millas controladas;
- la selección del mejor modelo se realiza con un criterio determinista basado en
métricas de validación;
- la evaluación utiliza definiciones formales y uniformes de IoU, error angular $\Delta\theta$
y *grasp success*;
- la comparación entre configuraciones mantiene constante la parametrización de
salida y el protocolo de medida;
- y la integración robótica se documenta como un flujo verificable por tópicos, logs,
capturas y artefactos del entorno simulado.

Además, el capítulo deja establecidos los elementos necesarios para interpretar correctamente los resultados del capítulo siguiente: qué configuraciones se comparan, bajo qué condiciones se entrenan, qué métricas se reportan y cómo deben leerse conjuntamente el rendimiento predictivo, la evidencia cualitativa y la eficiencia computacional.

Con ello, el Capítulo 5 puede presentar resultados comparativos entre configuraciones de forma consistente, apoyándose en evidencias cuantitativas, cualitativas y de eficiencia, y manteniendo coherencia entre entrenamiento offline y validación en entorno de emulación.

5. Resultados y discusión

En este capítulo se presentan y analizan los resultados obtenidos con el sistema de detección de poses de agarre desarrollado en este trabajo. En primer lugar, se estudia la evolución del entrenamiento a partir de las curvas registradas por el *pipeline* experimental. A continuación, se reportan los resultados cuantitativos agregados en validación bajo particionado object-wise, utilizando como métrica principal el criterio de éxito tipo Cornell y complementándolo con medidas de solapamiento geométrico, error angular y pérdida de validación. Posteriormente, estos resultados se interpretan junto con evidencias cualitativas de aciertos y fallos, así como con indicadores de eficiencia computacional basados en latencia de inferencia y tamaño de modelo. Finalmente, se discuten los hallazgos principales del estudio, poniendo el foco en las diferencias entre arquitecturas, modalidades de entrada y configuraciones experimentales dentro de un marco metodológico reproducible.

5.1. Resultados de entrenamiento

En este capítulo se analizan los resultados de entrenamiento obtenidos para las cuatro configuraciones experimentales consideradas (EXP1–EXP4). El objetivo es estudiar la dinámica de convergencia de cada modelo, comparar su estabilidad entre semillas y contextualizar la evolución de las métricas de validación a lo largo de las épocas. Para ello, se examinan las curvas de pérdida, éxito de agarre, solapamiento geométrico y error angular registradas por el *pipeline*. Posteriormente, se resumen los resultados finales agregados en validación, seleccionando para cada ejecución la mejor época según el máximo de *val_success*. Esta lectura permite distinguir el comportamiento del modelo ligero y del modelo residual, así como valorar el efecto de la modalidad de entrada y de la configuración experimental sobre el rendimiento final.

5.1.1. Curvas de pérdida y métricas de validación

En este subapartado se analizan las curvas de entrenamiento registradas por el *pipeline* para las cuatro configuraciones experimentales evaluadas (EXP1–EXP4), con el fin de estudiar la convergencia de los modelos y la evolución de las métricas de validación a lo largo de las épocas.

Las Ilustraciones 5-1 a 5-4 presentan, respectivamente, las curvas de pérdida de validación y entrenamiento (*train_loss* y *val_loss*) para EXP1 (SimpleGraspCNN, RGB), EXP2 (SimpleGraspCNN, RGB-D con aumento moderado), EXP3 (ResNet18Grasp, RGB con aumento) y EXP4 (ResNet18Grasp, RGB-D). En cada caso se muestran las ejecuciones correspondientes a distintas semillas, lo que permite evaluar la estabilidad del entrenamiento y apreciar tanto la velocidad de convergencia como la consistencia entre ejecuciones.

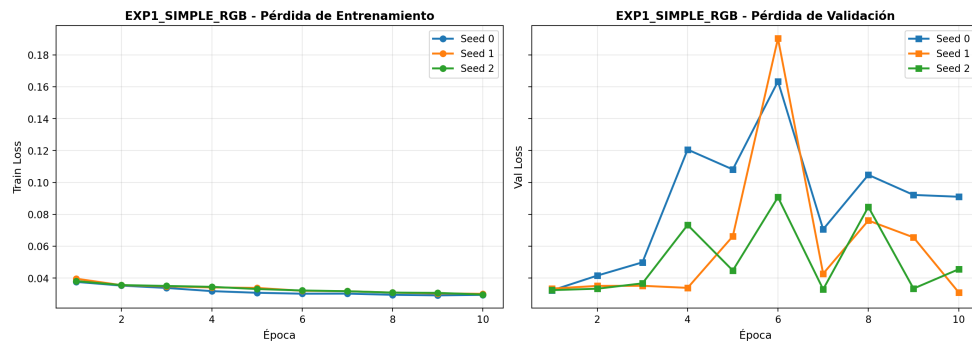


Ilustración 5-1. Curvas de pérdida (train_loss y val_loss) para EXP1 (SimpleGraspCNN, RGB).

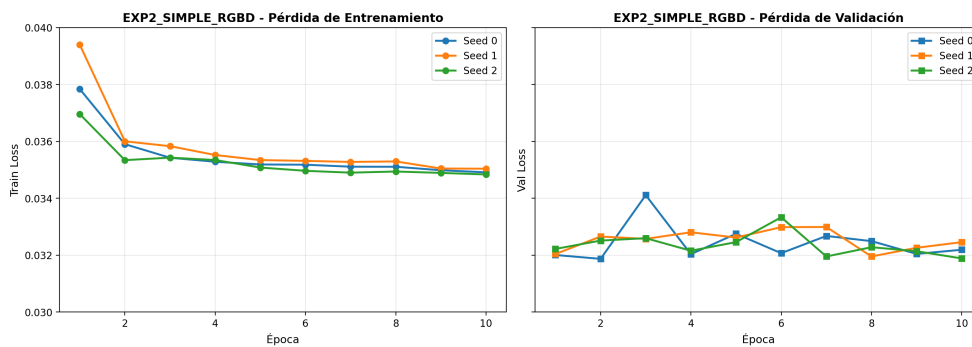


Ilustración 5-2. Curvas de pérdida (train_loss y val_loss) para EXP2 (SimpleGraspCNN, RGB-D con aumento moderado).

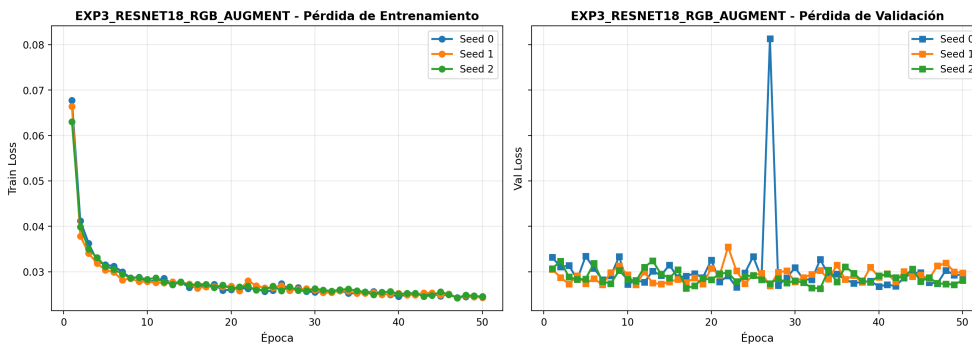


Ilustración 5-3. Curvas de pérdida (train_loss y val_loss) para EXP3 (ResNet18Grasp, RGB + augmentation).

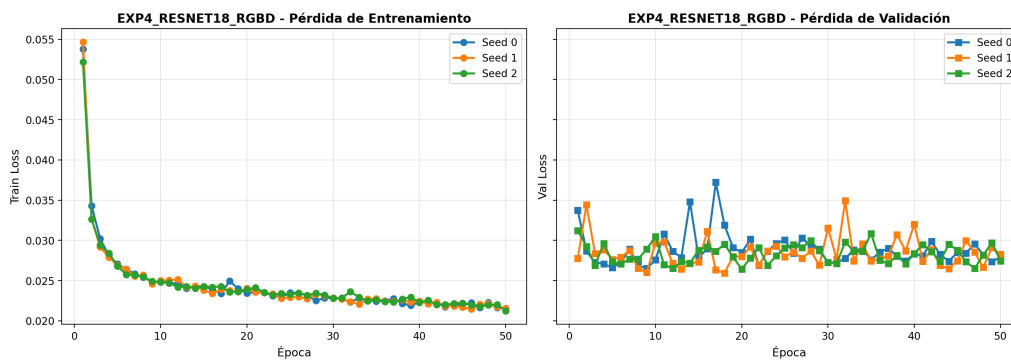


Ilustración 5-4. Curvas de pérdida (train_loss y val_loss) para EXP4 (ResNet18Grasp, RGB-D).

En todos los casos se observa un descenso acusado de la pérdida durante las primeras iteraciones, seguido de una fase de estabilización, aunque con comportamientos diferenciados entre el modelo ligero (SimpleGraspCNN; EXP1–EXP2) y el modelo residual (ResNet18Grasp; EXP3–EXP4). En particular, EXP1 y EXP2, entrenados durante 10 épocas, muestran una convergencia rápida y un margen de mejora más limitado, mientras que EXP3 y EXP4, entrenados durante 50 épocas, presentan una caída inicial más intensa y una evolución posterior más estable, compatible con la mayor capacidad representacional del *backbone* residual. Por su parte, la pérdida de validación (*val_loss*) reproduce de forma global esta pauta de descenso y estabilización, si bien presenta una variabilidad interépoca algo mayor, más visible en EXP3 y EXP4, sin que ello contradiga la tendencia general de convergencia observada.

Además de la pérdida de entrenamiento y validación (*train_loss* y *val_loss*), se consideran las curvas de tres métricas de validación: éxito de agarre (*val_success*; Ilustración 5-5), IoU medio (Ilustración 5-6) y error angular medio ($\Delta\theta$; Ilustración 5-7), que permiten interpretar no solo la convergencia del optimizador, sino también la evolución de la calidad geométrica de las predicciones bajo el criterio tipo Cornell. En lugar de mostrar simultáneamente todas las ejecuciones individuales, estas figuras resumen la tendencia de cada configuración experimental mediante una curva media por experimento, acompañada de una banda de variabilidad entre semillas.

En conjunto, estas curvas muestran que las configuraciones basadas en ResNet18Grasp alcanzan niveles de rendimiento superiores a las configuraciones basadas en SimpleGraspCNN, mientras que el efecto de la modalidad RGB frente a RGB-D no es uniforme, sino dependiente de la arquitectura y de la configuración experimental considerada.

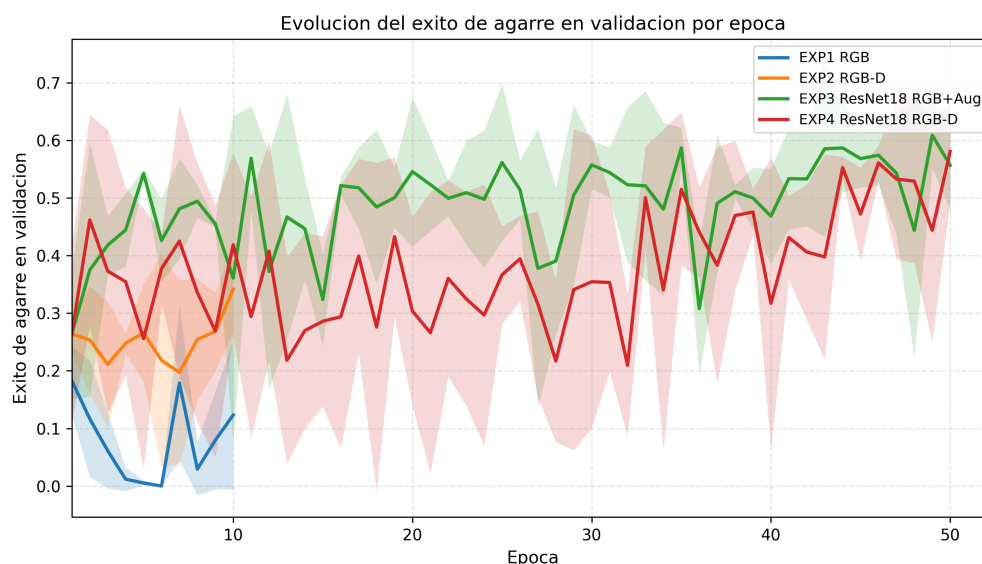


Ilustración 5-5. Evolución del éxito de agarre en validación (*val_success*) por época para las cuatro configuraciones experimentales.

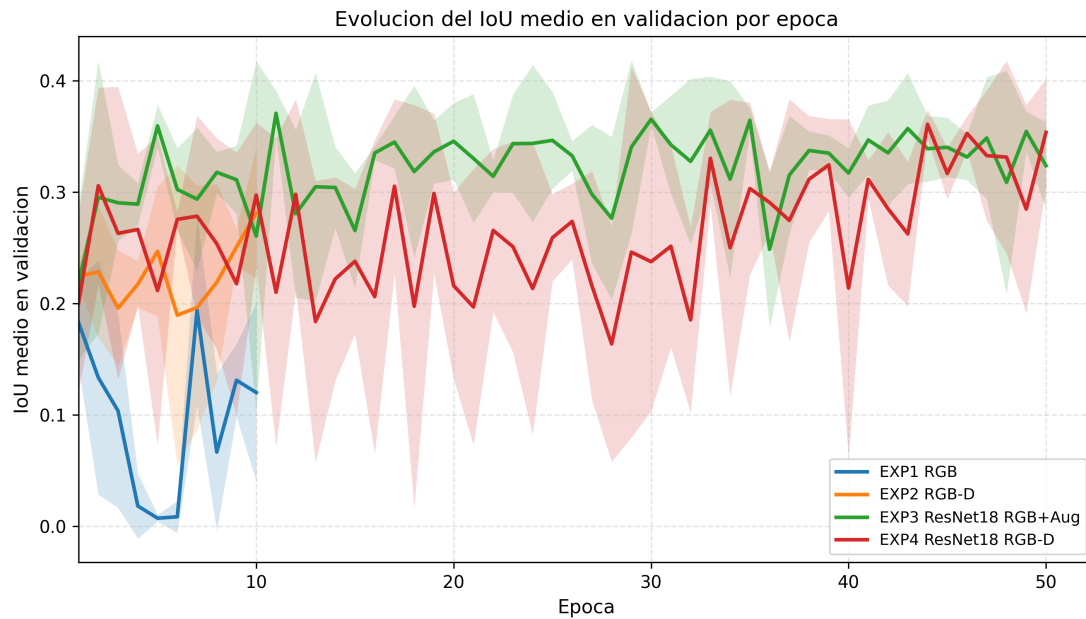


Ilustración 5-6. Evolución del IoU medio en validación por época para las cuatro configuraciones experimentales.

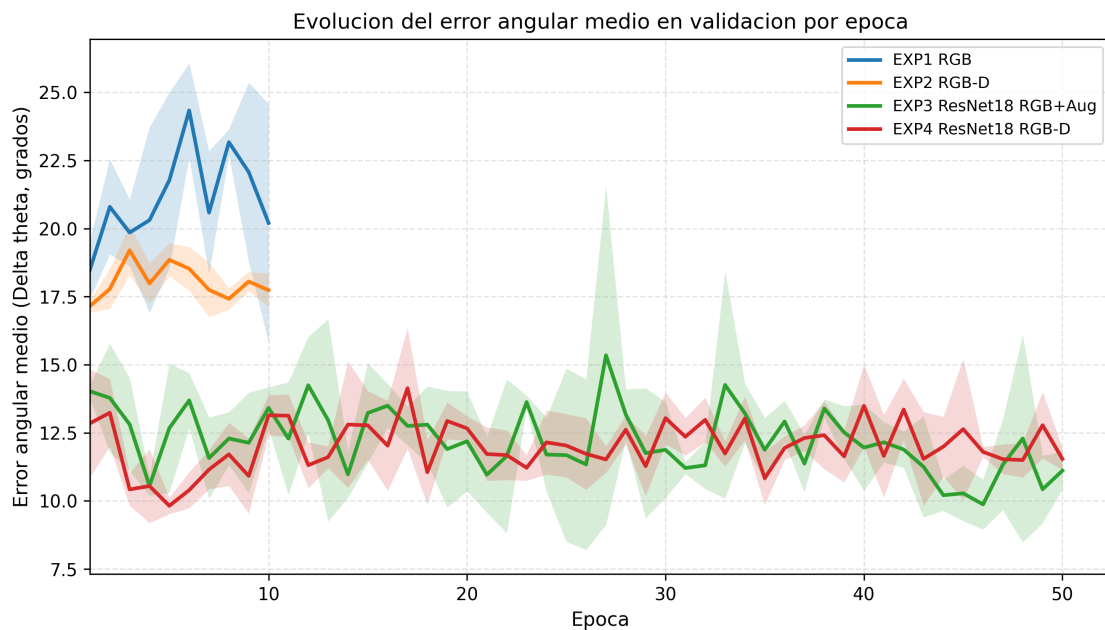


Ilustración 5-7. Evolución del error angular medio ($\Delta\theta$, en grados) en validación por época para las cuatro configuraciones experimentales.

En conjunto, las Ilustraciones 5-5 a 5-7 muestran que las configuraciones basadas en ResNet18Grasp mantienen, a lo largo del entrenamiento, una evolución más favorable en las métricas de validación que las configuraciones basadas en SimpleGraspCNN. En particular, las curvas medias de *val_success* e IoU medio alcanzan valores superiores en los experimentos residuales, mientras que el error angular medio se mantiene en niveles inferiores. Esta lectura refuerza la interpretación obtenida previamente a partir de las curvas de pérdida.

Luego, se seleccionaron las configuraciones experimentales con mejor época de entrenamiento. Para evitar que posibles épocas inválidas distorsionen la interpretación, la selección de la mejor época se realiza únicamente sobre registros consistentes de validación, aplicando el criterio determinista definido en el Capítulo 4. De este modo, los resultados agregados que se presentan a continuación reflejan, para cada ejecución, el mejor check-point seleccionado por máximo *val_success*, garantizando una comparabilidad homogénea entre experimentos y semillas.

A continuación, las Ilustraciones 5-8 y 5-9 permiten complementar la lectura de las curvas medias por época mostrando la variabilidad entre ejecuciones individuales en la mejor época de validación de cada seed. En la Ilustración 5-8, correspondiente a *val_success*, se observa una separación clara entre las configuraciones basadas en **SimpleGraspCNN** (EXP1 y EXP2) y las basadas en **ResNet18Grasp** (EXP3 y EXP4). En términos generales, las ejecuciones de EXP3 y EXP4 mantienen valores de éxito notablemente superiores, lo que refuerza la interpretación de que la arquitectura residual aporta una mejora consistente en la métrica principal del trabajo.

Además, la dispersión entre seeds es moderada en los cuatro experimentos, aunque con patrones diferenciados. En **EXP1**, la variabilidad relativa es más acusada, lo que sugiere una mayor sensibilidad del modelo ligero RGB a las condiciones iniciales del entrenamiento. Por otra parte, **EXP2** muestra una mejora clara respecto a EXP1 y una estabilidad entre seeds algo mayor, lo que resulta coherente con el efecto beneficioso de incorporar profundidad en la arquitectura ligera. Por su parte, **EXP3** y **EXP4** no solo alcanzan los valores más altos de éxito, sino que además mantienen una dispersión contenida, lo que indica un comportamiento más robusto y estable entre ejecuciones.

La Ilustración 5-9, correspondiente a *val_loss* en *best_epoch*, aporta una lectura complementaria. En este caso, las diferencias entre experimentos son más reducidas que en la tasa de éxito, lo que confirma que la pérdida de validación no siempre discrimina con la misma claridad que *val_success* la calidad final del agarre predicho. Aun así, puede observarse que las configuraciones residuales tienden a situarse en niveles de pérdida ligeramente inferiores, con **EXP4** mostrando en conjunto los valores más bajos, lo que resulta coherente con su buena estabilidad geométrica y con el comportamiento ya observado en las métricas agregadas.

En conjunto, ambas figuras muestran que la superioridad de **EXP3** y **EXP4** no depende de una única ejecución aislada, sino que se reproduce de manera razonablemente consistente en las distintas semillas. Asimismo, ponen de manifiesto que *val_success* ofrece una señal más informativa que *val_loss* para comparar configuraciones en este problema, al estar directamente alineada con el criterio de evaluación tipo Cornell empleado en el trabajo.

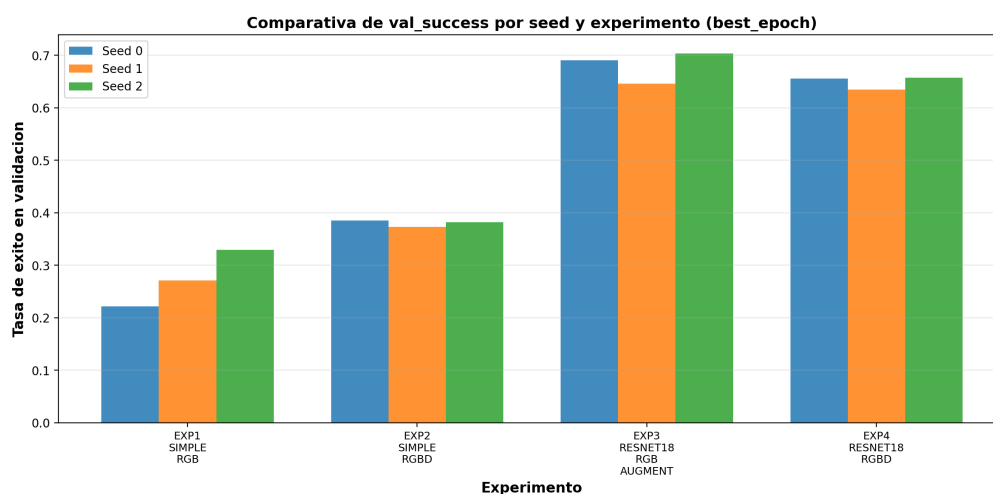


Ilustración 5-8. Comparativa de val_success por ejecución (seed) y experimento, tomando el valor en la mejor época (best_epoch) seleccionada por máximo val_success.

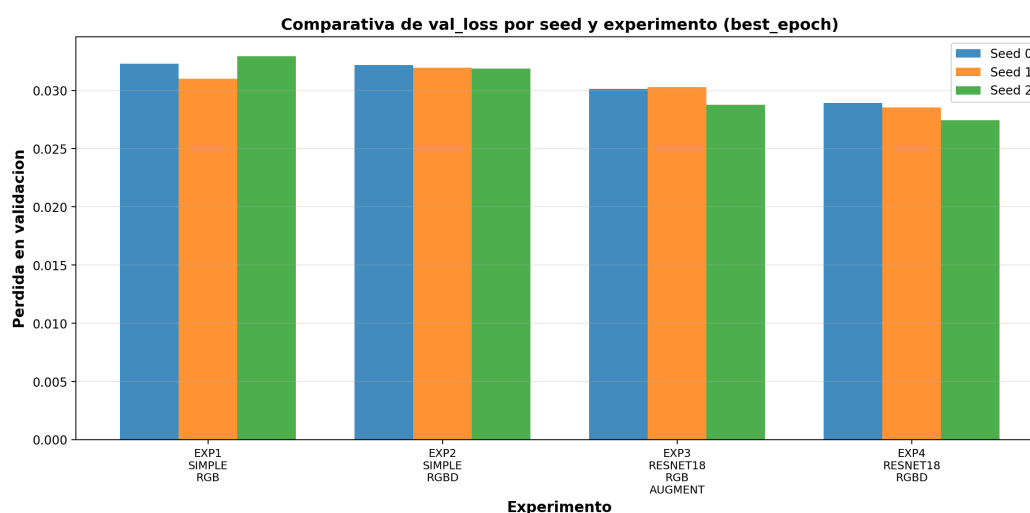


Ilustración 5-9. Comparativa de val_loss por ejecución (seed) y experimento, evaluado en best_epoch (seleccionado por máximo val_success).

A partir de la evolución por época analizada previamente, los resultados cuantitativos finales se presentan a continuación mediante métricas agregadas sobre validación, obtenidas del resumen consolidado del *pipeline* y reportadas bajo el particionado *object-wise* para los cuatro experimentos. En cada ejecución, la época seleccionada se corresponde con aquella que maximiza *val_success*, de acuerdo con el criterio de *best_epoch* definido en el Capítulo 4. Sobre esta base, las métricas finales se agregan considerando $n = 3$ semillas.

La Tabla 5-1 resume la métrica principal del trabajo, *grasp success (%)*, calculada sobre validación bajo criterio tipo Cornell. Por su parte, la Tabla 5-2 recoge métricas complementarias asociadas a la mejor época de cada ejecución, incluyendo el solapamiento geométrico medio (IoU), el error angular medio $\Delta\theta$ y la pérdida de validación (*val_loss*).

Experimento	Modelo	Modalidad	Grasp success (%)
EXP1_SIMPLE_RGB	SimpleGraspCNN	RGB	27,41 $\pm$ 4,40
EXP2_SIMPLE_RGBD	SimpleGraspCNN	RGB-D	38,05 $\pm$ 0,51
EXP3_RESNET18_RGB_AUGMENT	ResNet18Grasp	RGB	68,01 $\pm$ 2,45
EXP4_RESNET18_RGBD	ResNet18Grasp	RGB-D	64,92 $\pm$ 1,02

Tabla 5-1. Resultados agregados en validación (*best_epoch* por ejecución) bajo *split object-wise*.

En conjunto, los resultados muestran una separación clara entre las dos familias de modelos evaluadas. Las configuraciones basadas en ResNet18Grasp alcanzan niveles de rendimiento sustancialmente superiores a las configuraciones basadas en SimpleGraspCNN, lo que respalda la utilidad de una arquitectura residual preentrenada dentro del protocolo experimental adoptado. Asimismo, el efecto de la modalidad de entrada no es uniforme entre arquitecturas: en el modelo ligero, la incorporación de profundidad produce una mejora apreciable, mientras que en el modelo residual las configuraciones RGB con augmentation y RGB-D presentan resultados muy próximos en la métrica principal. Por ello, esta comparación debe matizarse a la luz de las métricas complementarias de solapamiento geométrico, error angular y pérdida de validación.

La Tabla 5-2 complementa la métrica principal de éxito incorporando tres indicadores adicionales de interés: el solapamiento geométrico medio (IoU), el error angular medio $\Delta\theta$ y la pérdida de validación asociada a la mejor época de cada ejecución. Estas métricas permiten matizar la comparación entre configuraciones más allá de la tasa de acierto final y ofrecen una visión más completa de la calidad geométrica y de la estabilidad del modelo seleccionado.

Experimento	Modelo	Modalidad	Grasp success (%)	IoU medio	$\Delta\theta$ (grados)	val_loss
EXP1_SIMPLE_RGB	SimpleGraspCNN	RGB	27,41 $\pm$ 4,40	0,2254 $\pm$ 0,0496	17,19 $\pm$ 1,37	0,0321 $\pm$ 0,0008
EXP2_SIMPLE_RGBD	SimpleGraspCNN	RGB-D	38,05 $\pm$ 0,51	0,3169 $\pm$ 0,0033	17,87 $\pm$ 0,36	0,0320 $\pm$ 0,0001
EXP3_RESNET18_RGB_AUGMENT	ResNet18Grasp	RGB	68,01 $\pm$ 2,45	0,4010 $\pm$ 0,0046	10,61 $\pm$ 1,18	0,0297 $\pm$ 0,0007
EXP4_RESNET18_RGBD	ResNet18Grasp	RGB-D	64,92 $\pm$ 1,02	0,3879 $\pm$ 0,0148	11,43 $\pm$ 0,53	0,0283 $\pm$ 0,0006

Tabla 5-2. Resumen de métricas finales por experimento en validación (*media $\pm$ desviación estándar sobre $n = 3$ semillas*).

Con el fin de sintetizar visualmente los resultados recogidos en las Tablas 5-1 y 5-2, se presentan a continuación tres representaciones (Ilustraciones 5-10 a 5-12) agregadas por experimento correspondientes a la métrica principal de éxito, al IoU medio y al error angular medio. En todos los casos se muestran los valores medios sobre tres semillas, calculados en la mejor época de cada ejecución según el criterio de *best_epoch*.

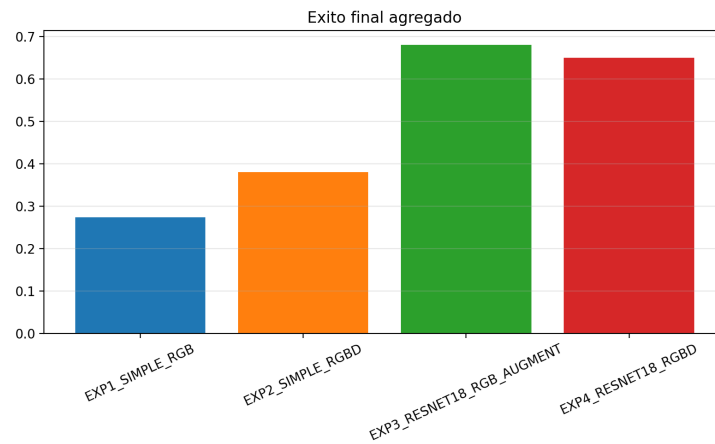


Ilustración 5-10. Éxito final de agarre en validación ($val_success$), agregado por experimento (media $\pm$ desviación estándar sobre semillas), seleccionando en cada ejecución $best_epoch$ por máximo $val_success$.

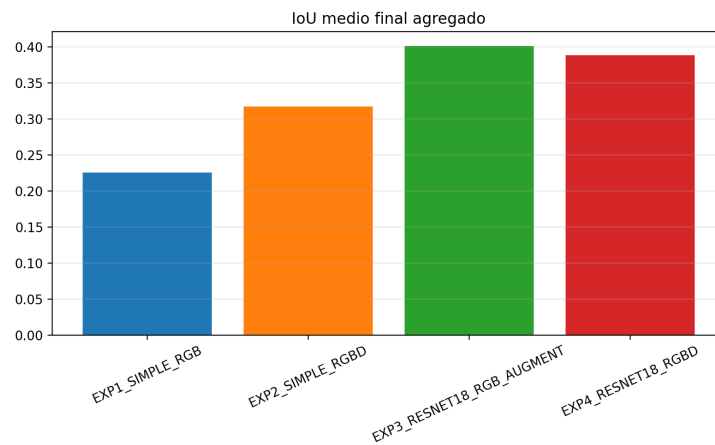


Ilustración 5-11. IoU medio final en validación, agregado por experimento (media $\pm$ desviación estándar sobre semillas) bajo el criterio de selección de $best_epoch$.

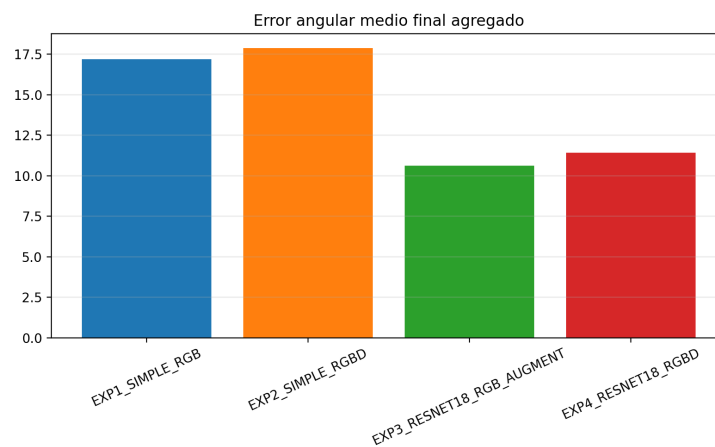


Ilustración 5-12. Error angular medio final ($\Delta\theta$ en grados) en validación, agregado por experimento (media $\pm$ desviación estándar sobre semillas), bajo el criterio de $best_epoch$.

A partir de las Tablas 5-1 y 5-2, y las ilustraciones 5-10 a 5-12, se derivan tres observaciones principales:

1. **Superioridad clara de la arquitectura residual.** Las dos configuraciones basadas en ResNet18Grasp (EXP3 y EXP4) superan ampliamente a las configuraciones ligeras basadas en SimpleGraspCNN (EXP1 y EXP2). Esta diferencia confirma que, bajo el protocolo experimental adoptado, la mayor capacidad representacional del *backbone* residual resulta determinante para mejorar la detección de agarres en validación object-wise.
2. **Efecto de la modalidad de entrada dependiente de la arquitectura.** En SimpleGraspCNN, la incorporación de profundidad produce una mejora sustancial al pasar de EXP1 (RGB) a EXP2 (RGB-D), elevando de forma clara la tasa de éxito en validación. En cambio, en ResNet18Grasp la comparación entre EXP3 (RGB con aumento) y EXP4 (RGB-D) muestra resultados muy próximos, por lo que el efecto de incorporar profundidad no puede interpretarse de forma aislada, sino en interacción con la arquitectura y con la configuración experimental.
3. **Mejor configuración según la métrica principal del trabajo.** Tomando como referencia el criterio de éxito tipo Cornell (grasp success), la mejor configuración observada es EXP3_RESNET18_RGB_AUGMENT, seguida muy de cerca por EXP4_RESNET18_RGBD. No obstante, cuando se incorporan las métricas complementarias de la Tabla 5-2, se observa que EXP3 presenta el mejor IoU medio y el menor error angular medio, mientras que EXP4 mantiene la menor val_loss. En consecuencia, ambas configuraciones residuales constituyen las alternativas más sólidas del estudio, con EXP3 como mejor opción en la métrica principal y en la calidad geométrica media, y EXP4 como configuración especialmente competitiva desde el punto de vista de la estabilidad de validación.

5.2. Análisis cualitativo (aciertos y fallos)

Dado que la métrica tipo Cornell penaliza fuertemente desviaciones relativamente pequeñas en orientación y solapamiento, el análisis cualitativo resulta útil para interpretar mejor los resultados cuantitativos. Para ello, se presenta una selección de ejemplos de validación con superposición del rectángulo anotado y el rectángulo predicho, con el fin de visualizar tanto aciertos plausibles como fallos representativos.

5.2.1. Tipos de acierto observados

En los casos de acierto observados, las predicciones correctas suelen compartir tres rasgos principales:

- localizan adecuadamente la región funcional del objeto;
- aproximan de forma coherente la orientación principal del agarre;
- generan aperturas compatibles con la geometría visible del objeto.

Desde un punto de vista cualitativo, estos aciertos corresponden a ejemplos en los que la predicción no solo resulta visualmente razonable, sino que además satisface simultáneamente los umbrales de IoU y $\Delta\theta$ exigidos por el criterio tipo Cornell. La Ilustración 5-13 muestra ejemplos representativos en los que la superposición entre el rectángulo anotado (ground truth, GT, en verde discontinuo) y el rectángulo predicho (Pred, en rojo continuo) resulta coherente tanto en posición como en orientación, cumpliéndose así las condiciones geométricas del criterio de éxito empleado.

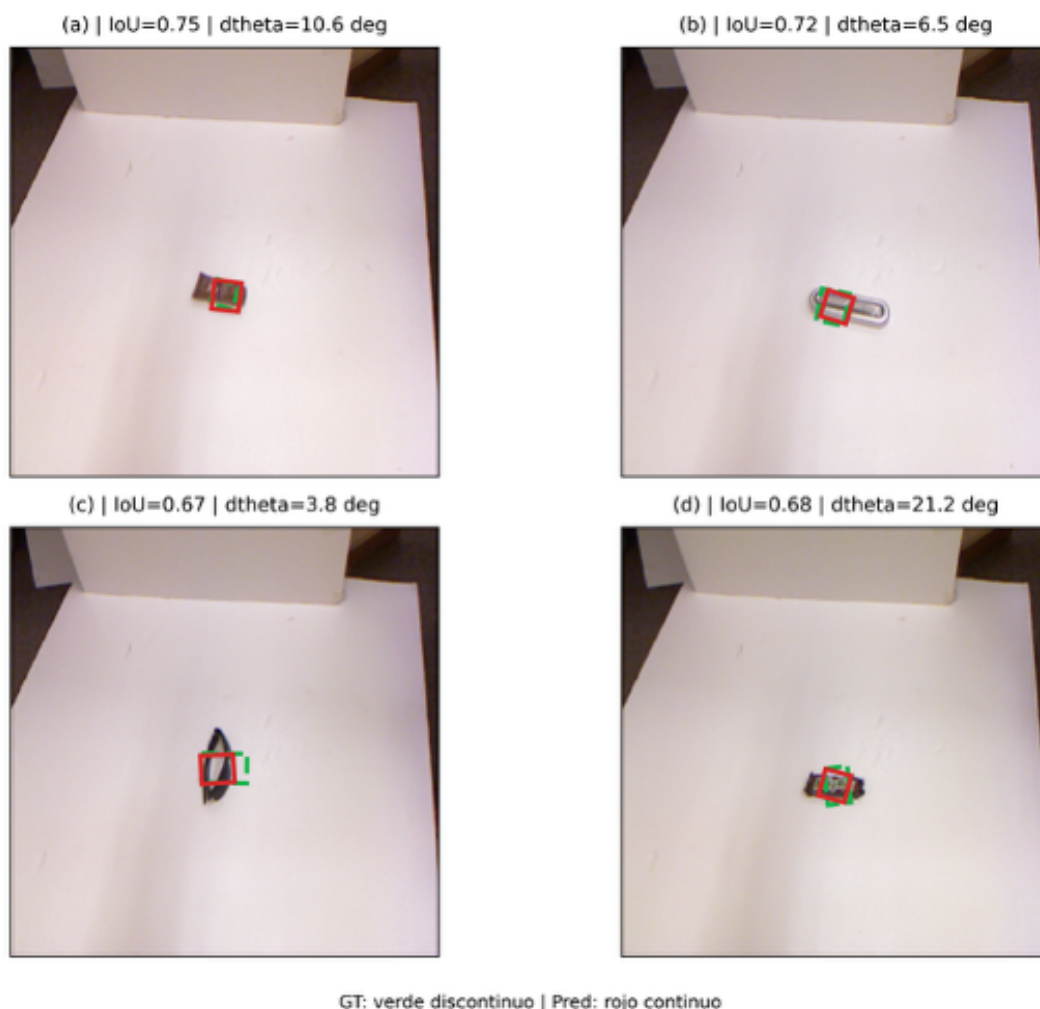


Ilustración 5-13. Aciertos representativos en validación para el modelo EXP3_RESNET18_RGB_AUGMENT (seed 0), con superposición entre el GT seleccionado (verde discontinuo) y la predicción (rojo continuo).

5.2.2. Tipos de fallo (tipología)

Los fallos se agrupan en patrones repetidos:

- E1 — **Error angular**: el rectángulo predicho está cerca del objeto, pero $\Delta\theta > 30^\circ$.

- E2 — **Bajo solapamiento (IoU)**: el rectángulo apunta a la zona correcta, pero queda desplazado o mal dimensionado.
- E3 — **Tamaño/apertura no plausible**: el rectángulo presenta dimensiones o apertura poco plausibles para el objeto.
- E4 — **Confusión por fondo u oclusión**: el centro se predice sobre el fondo o sobre una región que no constituye una zona de agarre accesible.

La Ilustración 5-14 muestra una galería cualitativa de fallos en validación, organizada explícitamente según esta tipología E1–E4. En cada ejemplo se representa la superposición entre el rectángulo anotado (GT, en verde discontinuo) y el rectángulo predicho (Pred, en rojo continuo), junto con los valores de IoU y error angular $\Delta\theta$. Esta separación respecto a la figura de aciertos permite distinguir con mayor claridad los casos en los que la predicción resulta geoméricamente plausible de aquellos en los que falla por orientación, solape, dimensión o confusión con el entorno.

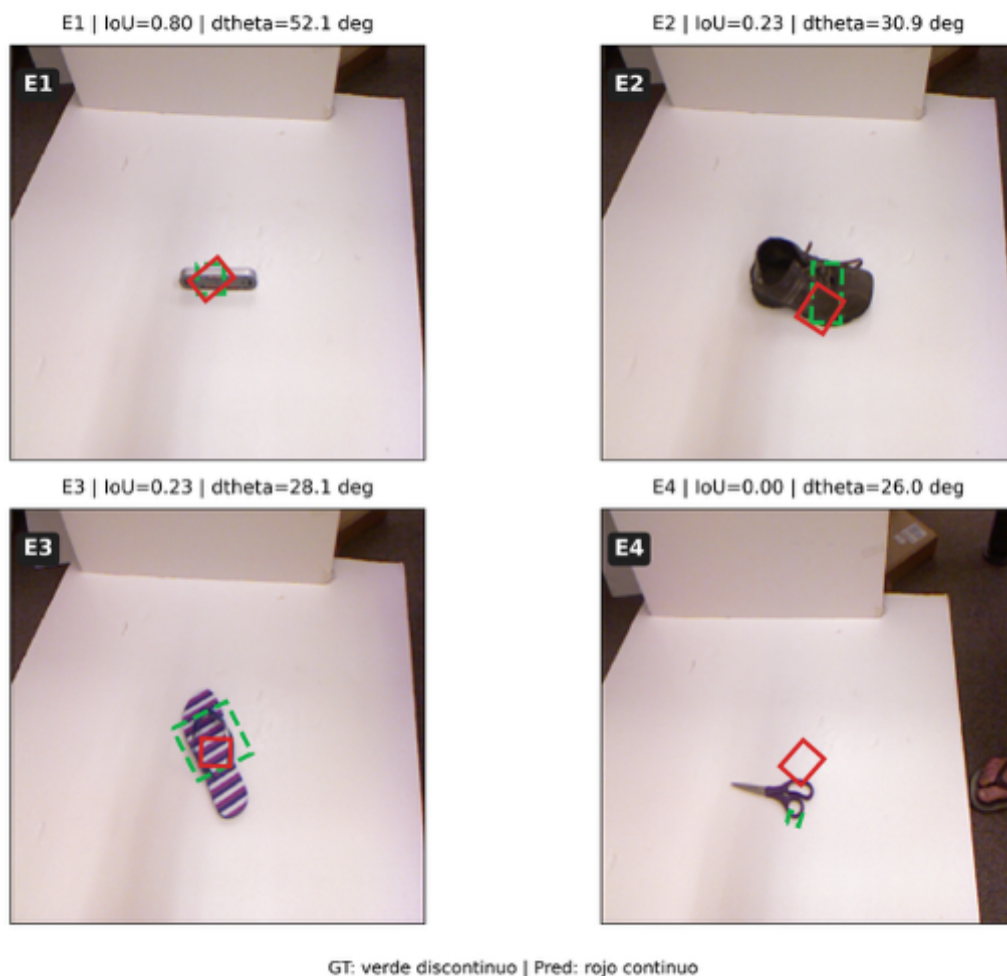


Ilustración 5-14. Fallos cualitativos representativos en validación, organizados por tipología: E1 (error angular), E2 (bajo solapamiento), E3 (tamaño o apertura no plausibles) y E4 (confusión por fondo o región no funcional). En todos los casos se muestra la superposición entre el GT de referencia (verde discontinuo) y la predicción (rojo continuo), junto con los valores de IoU y discrepancia angular del ejemplo.

Desde un punto de vista interpretativo, estos ejemplos muestran que no todos los errores responden a una mala localización global del objeto. En algunos casos, la red identifica una región próxima a la correcta, pero incurre en una desviación angular excesiva o en una apertura incompatible con la geometría del objeto. En otros, el fallo aparece por desplazamiento del rectángulo respecto a la zona funcional de agarre o por interferencia del fondo, lo que impide satisfacer simultáneamente los umbrales del criterio Cornell. En conjunto, esta tipología ayuda a entender que el error puede residir tanto en la precisión geométrica fina de la predicción como en la selección misma de la región de agarre.

Esta galería no pretende constituir una evaluación cuantitativa adicional, sino ofrecer una lectura visual complementaria del comportamiento de los modelos bajo el criterio Cornell. En particular, permite observar que una predicción puede resultar visualmente plausible y, sin embargo, no alcanzar simultáneamente los umbrales requeridos de orientación y solape.

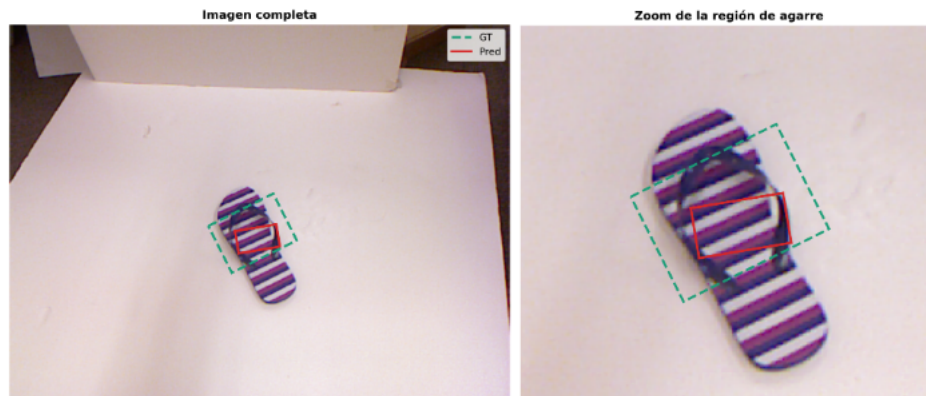
5.2.3. Comparación cualitativa entre modelos

Desde el punto de vista cualitativo, ResNet18Grasp muestra una mayor capacidad para localizar regiones plausibles del objeto y proponer orientaciones más consistentes, en línea con su mejor rendimiento cuantitativo. No obstante, siguen apareciendo falsos negativos con coincidencia visualmente razonable, cuyos detalles ayudan a explorar los matices del solapamiento insuficiente que invalida el acierto pese a que la predicción resulte visualmente razonable.

En SimpleGraspCNN también aparecen propuestas plausibles, pero con mayor frecuencia para parte relevante de ellas no supera simultáneamente los umbrales de IoU y $\Delta\theta$. Esta observación ayuda a explicar su menor tasa de éxito global y sugiere que la limitación no reside tanto en la localización gruesa del objeto, como en la precisión geométrica fina requerida por el criterio empleado.

Un caso especialmente ilustrativo es aquel en el que la predicción resulta visualmente plausible, pero no alcanza el criterio de éxito debido a una desviación angular moderada o a un solapamiento insuficiente con la anotación de referencia. Este tipo de situaciones pone de manifiesto que la evaluación no depende únicamente de la plausibilidad visual del agarre, sino del cumplimiento simultáneo de restricciones geométricas concretas sobre orientación y solapamiento.

La Ilustración 5-15 recoge un caso límite de falso negativo visualmente plausible. En este ejemplo, la predicción se aproxima de forma razonable a la región de agarre de referencia desde un punto de vista cualitativo, pero no supera el criterio de éxito tipo Cornell al incumplir, por un margen reducido, los umbrales geométricos exigidos. En concreto, el caso corresponde al experimento EXP1_SIMPLE_RGB, seed 2, sobre la muestra data/raw/cornell/02/pcd0271r.png, con $\text{IoU} = 0,248$ y $\Delta\theta = 17.18$ grados, por lo que queda clasificado como fallo. Este ejemplo resulta especialmente ilustrativo porque muestra que una predicción visualmente plausible puede ser invalidada por pequeñas desviaciones respecto a los umbrales de evaluación, lo que refuerza la exigencia geométrica del protocolo Cornell.



Experimento: EXP2_SIMPLE_RGBD | Seed: 0 | Modelo: SimpleGraspCNN (rgbd)
Imagen: data/raw/cornell/06/pcd0633r.png | IoU = 0,248 | $\Delta\theta = 17,18^\circ$ | Resultado = FALLO | Causa principal = solapamiento insuficiente

Ilustración 5-15. Caso límite de falso negativo por IoU insuficiente (EXP2_SIMPLE_RGBD, seed 0). La predicción mantiene una orientación compatible con la referencia ($\Delta\theta = 17,18^\circ$), pero no alcanza el umbral Cornell de solapamiento ($\text{IoU} = 0,248 < 0,25$), por lo que se clasifica como fallo

En conjunto, estos ejemplos cualitativos refuerzan la interpretación de los resultados cuantitativos y permiten identificar con mayor claridad el tipo de errores que separa una predicción visualmente razonable de un acierto efectivo bajo el protocolo de evaluación adoptado.

5.3. Eficiencia computacional

En robótica aplicada, la comparación entre modelos no debe limitarse al rendimiento predictivo; también debe considerar el coste computacional asociado al despliegue, especialmente cuando se opera con restricciones de CPU/GPU, memoria y latencia. En este trabajo, la comparación se centra en dos arquitecturas con la misma parametrización de salida (C_x, C_y, w, h, θ), pero con capacidades y costes distintos.

En despliegue, el factor más relevante para control en tiempo real es la latencia de inferencia por muestra (batch=1). Por ello, en este capítulo se comparan CPU y GPU mediante un protocolo reproducible, reportado en la Tabla 5-3. Además del rendimiento predictivo presentado en el capítulo 5.1, el análisis de eficiencia permite valorar la viabilidad del entorno de emulación en hardware limitado, donde un modelo ligero puede ser preferible si el coste computacional constituye un requisito dominante.

Con estos objetivos, la evidencia de eficiencia se apoya en medidas de latencia de inferencia obtenidas bajo un protocolo reproducible (batch=1, warm-up=5, repeticiones=20), diferenciando CPU y GPU cuando procede. La Tabla 5-3 reporta, para cada experimento y dispositivo, la media, la desviación estándar, el percentil p95, el FPS aproximado y el tiempo mínimo observado. En GPU, la medición se realiza con sincronización explícita cuando corresponde, para evitar infraestimar los tiempos reales de inferencia.

Experimento	Dispositivo	Batch	Warm-up	Rep	Media (ms)	Desv. (ms)	p95 (ms)	FPS	Mín. (ms)
EXP1_SIMPLE_RGB	CPU	1	5	20	4,66	0,21	5,12	214	4,22
EXP1_SIMPLE_RGB	GPU (CUDA)	1	5	20	0,33	0,01	0,36	3029	0,31
EXP2_SIMPLE_RGBD	CPU	1	5	20	5,01	0,22	5,54	199	4,61
EXP2_SIMPLE_RGBD	GPU (CUDA)	1	5	20	0,33	0,00	0,33	3030	0,33
EXP3_RESNET18_RGB_AUGMENT	CPU	1	5	20	7,18	0,07	7,34	139	7,00
EXP3_RESNET18_RGB_AUGMENT	GPU (CUDA)	1	5	20	1,30	0,01	1,33	768	1,27
EXP4_RESNET18_RGBD	CPU	1	5	20	7,34	0,21	7,82	136	6,88
EXP4_RESNET18_RGBD	GPU (CUDA)	1	5	20	1,32	0,01	1,35	760	1,29

Tabla 5-3. Medición de latencia de inferencia por experimento y dispositivo, incluyendo parámetros del protocolo y métricas agregadas de tiempo de ejecución.

Los resultados muestran que ambas familias de modelos pueden ejecutarse con latencias bajas en el hardware considerado, aunque la GPU reduce de forma muy notable el tiempo de inferencia. En particular, los experimentos basados en SimpleGraspCNN (EXP1 y EXP2) presentan latencias menores que los basados en ResNet18Grasp (EXP3 y EXP4), tanto en CPU como en GPU, lo que confirma su menor coste computacional.

En CPU, EXP1 y EXP2 alcanzan medias de 4,66 ms y 5,01 ms por inferencia, respectivamente, frente a 7,18 ms y 7,34 ms en EXP3 y EXP4. Esto implica que los modelos ligeros ofrecen un ahorro temporal apreciable, manteniendo además FPS aproximados de 214 y 199, frente a 139 y 136 en los experimentos residuales. En GPU, la aceleración es todavía más marcada: EXP1 y EXP2 descienden hasta 0,33 ms, mientras que EXP3 y EXP4 se sitúan en 1,30 ms y 1,32 ms. Aunque en todos los casos los tiempos siguen siendo muy bajos, el patrón relativo se mantiene: las configuraciones residuales continúan siendo más costosas que las ligeras.

Desde una perspectiva comparativa, la aceleración obtenida al pasar de CPU a GPU es del orden de 14–15× en los modelos simples (EXP1 y EXP2) y de aproximadamente 5–6× en los modelos basados en ResNet18 (EXP3 y EXP4). Esta diferencia sugiere que los modelos ligeros aprovechan especialmente bien el entorno de inferencia en GPU, aunque incluso las configuraciones residuales mantienen tiempos compatibles con un uso interactivo o cuasi-tiempo real en el marco experimental considerado.

Estos resultados deben interpretarse conjuntamente con el análisis predictivo del capítulo 5.1. EXP1 y EXP2 son más eficientes computacionalmente, pero a costa de un rendimiento notablemente inferior en la métrica principal de éxito y en las métricas geométricas complementarias. Por el contrario, EXP3 y EXP4 incrementan la latencia, aunque conservan tiempos de inferencia suficientemente bajos como para resultar viables en simulación y en un *pipeline* robótico de percepción, a cambio de una mejora sustancial del rendimiento.

En consecuencia, el compromiso rendimiento–eficiencia observado en este trabajo favorece a las configuraciones basadas en ResNet18Grasp cuando la prioridad es maximizar

la calidad de la detección de agarres bajo el protocolo evaluado. No obstante, las configuraciones ligeras siguen siendo relevantes en escenarios con hardware restringido o cuando el coste computacional constituya un criterio dominante de diseño. Esta lectura refuerza la idea de que la selección final del modelo no debe basarse únicamente en la tasa de éxito, sino en el equilibrio entre precisión, latencia y contexto de despliegue.

5.4. Resultados de integración en ROS 2 + Gazebo

Con el fin de complementar la evaluación del detector más allá del análisis offline sobre Cornell, se validó su integración en un entorno robótico simulado basado en ROS 2 Jazzy y Gazebo Harmonic (Gazebo Sim 8.10.0), ejecutado sobre Ubuntu 24.04.4 LTS. La validación se realizó en el workspace `/home/laboratorio/TFM/agarre_ros2_ws`, seleccionado como entorno principal frente a otras versiones históricas por ser el que concentra los logs recientes, las capturas y el flujo de arranque documentado del sistema. Desde el punto de vista hardware, la integración fue probada tanto en la plataforma objetivo MeLE Overclock 4C N300, definida en la metodología como entorno simulado/edge, como en la estación de trabajo con AMD Ryzen 7 5800X y NVIDIA RTX 4070, empleada para depuración, verificación acelerada y pruebas apoyadas en GPU. En consecuencia, esta subsección debe interpretarse como una validación funcional del pipeline en ambas plataformas, aunque las evidencias principales aquí mostradas proceden del entorno de trabajo sobre PC.

5.4.1. Escena simulada y adquisición de imagen

La validación se llevó a cabo sobre el mundo simulado *ur5_mesa_objetos.sdf*, que representa una escena de trabajo tipo *table-top* con un brazo UR5 y objetos dispuestos sobre una mesa. La adquisición de imagen para el detector se realizó mediante una cámara cenital simulada, publicada en el tópico `/camera_overhead/image`, con una resolución de 320×240 píxeles. La conexión entre Gazebo y ROS 2 se efectuó mediante el puente principal *ros_gz_bridge_main*, configurado a través del archivo *scripts/bridge_cameras.yaml*. La Ilustración 5-16 muestra una vista representativa del entorno simulado empleado en la validación de integración en ROS 2 + Gazebo.

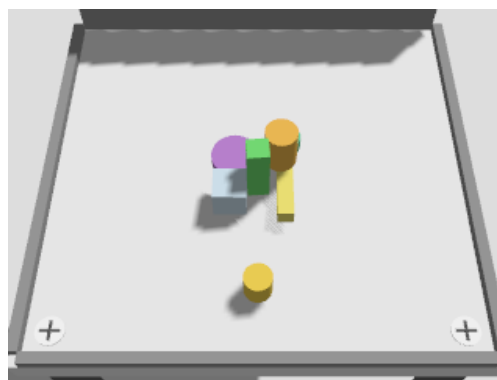


Ilustración 5-16. Evidencia funcional del pipeline percepción–publicación–consumo en ROS 2, donde la hipótesis de agarre generada por el detector se transforma en una pose ejecutable y es procesada por el subsistema de planificación a través de la interfaz definida para la integración con MoveIt.

Esta configuración permitió disponer de una imagen RGB consistente con la escena simulada y usarla como entrada directa del pipeline de percepción. Desde el punto de vista metodológico, el interés de esta fase no reside únicamente en la disponibilidad de la imagen, sino en que dicha imagen procede del entorno simulado real de la integración y no de un ejemplo desacoplado del detector.

5.4.2. Modelo integrado y generación de la hipótesis de agarre

Para este caso principal de validación se empleó el modelo EXP3_RESNET18_RGB_AUGMENT, concretamente la ejecución seed_0, cargando el checkpoint best.pth. Esta configuración corresponde a una arquitectura ResNet18 en modalidad RGB, con in_channels = 3 y tamaño de entrada 224×224 , y fue seleccionada por presentar el compromiso más favorable entre rendimiento predictivo y coste computacional entre las configuraciones evaluadas en este trabajo.

A partir de la imagen adquirida por la cámara simulada, el panel de integración ejecutó la inferencia y generó inicialmente una hipótesis de pinza 2D expresada en píxeles mediante los parámetros (C_x, C_y, w, h, θ) . Posteriormente, esta salida se transformó a una representación en el sistema de referencia del robot, denominada *grasp_base*, con los parámetros (x, y, z, yaw) . De este modo, la predicción del modelo dejó de ser únicamente una salida geométrica en la imagen para convertirse en una hipótesis de agarre consumible por el pipeline robótico. La Ilustración 5-17 presenta un ejemplo representativo de esta fase, mostrando el resultado de inferencia del modelo sobre la imagen simulada y su representación visual en el panel de integración.

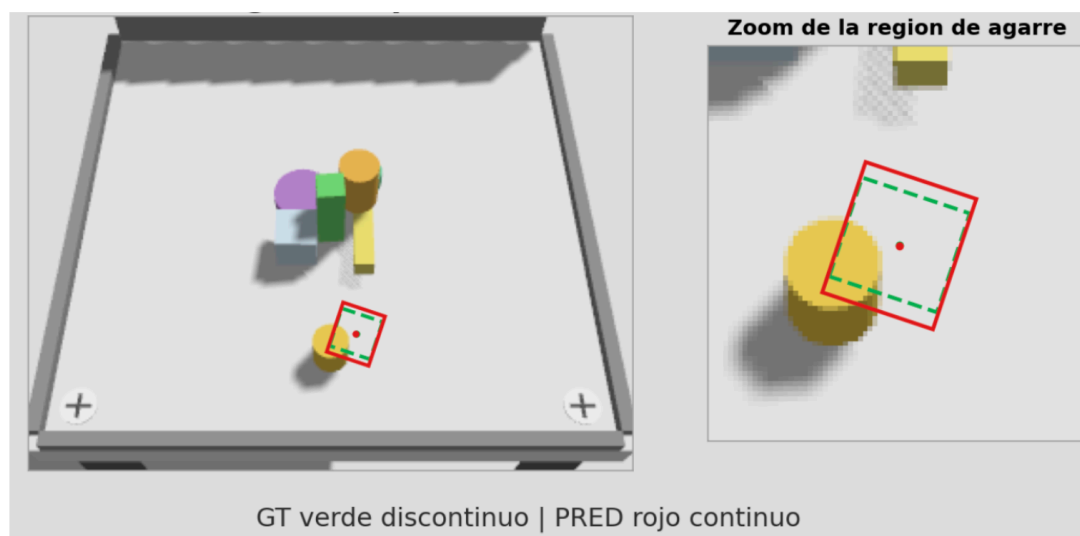


Ilustración 5-17. Resultado de inferencia del modelo EXP3_RESNET18_RGB_AUGMENT sobre una imagen simulada, mostrando la hipótesis de agarre estimada y su representación visual en el panel de integración. En este ejemplo, la predicción se sitúa próxima a la referencia de agarre, lo que permite visualizar de forma cualitativa el comportamiento del sistema.

La evidencia disponible confirma que, en el caso principal seleccionado, la inferencia finalizó correctamente y que la hipótesis de agarre quedó efectivamente disponible para su publicación y posterior consumo por el resto del sistema.

5.4.3. Publicación en ROS 2 y consumo por MoveIt

Una vez obtenida la hipótesis de agarre, el sistema distingue entre su representación visual en imagen y su representación ejecutable para planificación. En concreto, la inferencia produce una hipótesis visual en `/grasp_rect`, empleada como overlay sobre la imagen, mientras que la pose 3D normalizada que activa la planificación se publica en `/desired_grasp`. Esta última actúa como interfaz de entrada para el subsistema de MoveIt, y su resultado se devuelve a través de `/desired_grasp/result`.

Desde el punto de vista funcional, este encadenamiento confirma que la salida del detector no queda limitada a una visualización local en el panel, sino que entra en el middleware robótico mediante una interfaz explícita y es interpretada por el componente de planificación del sistema. En otras palabras, se validó el flujo completo percepción → publicación → consumo, que constituye la evidencia central de esta subsección. La Ilustración 5-18 muestra una evidencia funcional representativa de este flujo dentro de ROS 2, donde la hipótesis de agarre generada por el detector se publica para su procesamiento por el subsistema de planificación.

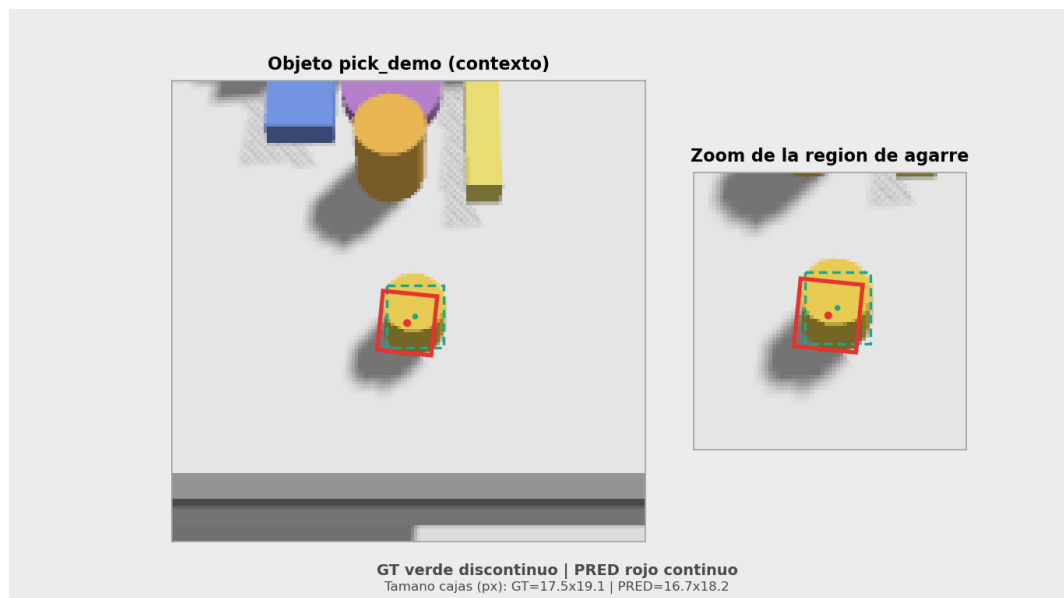


Ilustración 5-18. Evidencia funcional del pipeline percepción–publicación–consumo en ROS 2, en el que la hipótesis de agarre generada por el detector se publica para su procesamiento por el subsistema de planificación, verificando la coherencia de la integración entre percepción, publicación y consumo dentro del entorno simulado.

La evidencia técnica más sólida asociada a este caso incluye registros de ejecución con eventos del tipo `apply_experiment OK`, `infer_end status=OK`, disponibilidad de `grasp_base` y varias trazas `execute OK mode=moveit_sequence source=infer_model`. En conjunto, estos elementos muestran que la integración entre detector, publicación ROS 2 y consumidor MoveIt funcionó de manera consistente dentro del escenario seleccionado.

5.4.4. Alcance de la validación y nivel de evidencia

A partir del conjunto de evidencias revisadas, esta subsección permite sostener, con un grado de evidencia funcional suficiente, que el sistema implementa de forma coherente la cadena percepción → publicación → consumo dentro del entorno ROS 2 + Gazebo. No obstante, este resultado no puede presentarse como una demostración cerrada de agarre autónomo completo, guiado extremo a extremo por la red hasta el estado final del objeto.

Esta distinción es importante desde el punto de vista académico. El resultado obtenido acredita que la hipótesis de agarre producida por el modelo puede insertarse en una arquitectura robótica reproducible y ser utilizada por el subsistema de planificación, lo cual constituye una evidencia relevante de aplicabilidad. Sin embargo, la documentación revisada no permite vincular de manera inequívoca, en este mismo caso inferido, la predicción del detector con una secuencia final de manipulación cerrada y verificada sobre el objeto. Por ello, esta sección debe interpretarse como una demostración funcional de integración y no como una campaña estadística de éxito manipulativo autónomo.

5.4.5. Discusión

Los resultados obtenidos refuerzan la idea de que la contribución de este trabajo no se limita a la comparación *offline* de modelos sobre Cornell, sino que avanza hacia una integración operativa en un pipeline robótico simulado. En particular, se ha comprobado que el modelo seleccionado puede recibir una imagen procedente de una cámara simulada, generar una hipótesis de agarre consistente, transformarla al marco del robot y publicarla en ROS 2 para que sea consumida por un componente basado en MoveIt.

Esta validación aporta una evidencia de aplicabilidad práctica del sistema y conecta los resultados cuantitativos del detector con un escenario de despliegue más próximo a una arquitectura robótica real. No obstante, el alcance de la prueba sigue siendo funcional y demostrativo, por lo que sus resultados deben interpretarse como una validación de integración y no como una evaluación manipulativa autónoma cerrada. A partir de este marco, resulta pertinente analizar seguidamente cómo se sitúan los resultados obtenidos respecto al estado del arte, teniendo en cuenta tanto las diferencias metodológicas del protocolo experimental como el desplazamiento reciente del área hacia escenarios más complejos de agarre.

5.5. Comparación con el estado del arte

Una vez presentados los resultados cuantitativos, cualitativos y de integración funcional del sistema, se analiza a continuación su relación con trabajos representativos del estado del arte. Esta comparación debe interpretarse con cautela metodológica, ya que los valores obtenidos en este trabajo proceden de un protocolo reproducible específico, basado en validación bajo *split object-wise* y selección de *best_epoch* por máximo *val_success*. En consecuencia, los resultados aquí reportados deben entenderse como una línea base reproducible y no como un rendimiento final

directamente comparable con estudios publicados bajo particionados, métricas o configuraciones experimentales diferentes.

La literatura sobre Cornell presenta resultados competitivos con enfoques diversos, incluyendo trabajos clásicos de referencia como Lenz et al. (2015) y propuestas posteriores basadas en redes convolucionales más profundas. Sin embargo, la investigación reciente en agarre robótico ha desplazado parte del foco hacia escenarios más realistas de *clutter* y agarre 6-DoF, apoyándose en *datasets* y representaciones de mayor complejidad. Ejemplos representativos son Contact-GraspNet (Sundermeyer et al., 2021), orientado a generación eficiente de agarres 6-DoF en escenas desordenadas, NeuGraspNet (Jauhri et al., 2024), centrado en agarre 6-DoF en clutter desde una única vista, y propuestas más recientes como Efficient Heatmap-Guided 6-DoF Grasp Detection in Cluttered Scenes (Chen et al., 2023), que buscan mejorar simultáneamente precisión y eficiencia en escenas complejas.

En este contexto, la comparación directa exige explicitar al menos tres elementos: el particionado utilizado, el criterio exacto de evaluación ($\text{IoU}/\Delta\theta$ o métricas equivalentes) y el régimen de entrenamiento. Por ello, el contraste con el estado del arte debe interpretarse con cautela metodológica. El mejor resultado de este trabajo se obtiene bajo un *split object-wise*, con selección de *best_epoch* por máximo *val_success* y sin conjunto de prueba independiente, por lo que constituye una línea base reproducible más que un rendimiento final directamente comparable con resultados publicados bajo protocolos distintos.

Además, la evolución reciente del área sugiere que el *benchmark* Cornell sigue siendo útil para estudiar detección de agarres 2D/2.5D bajo un protocolo controlado, pero ya no agota el problema del agarre en entornos no estructurados. En esa dirección, trabajos recientes y nuevos recursos como *GraspClutter6D* insisten en escenas con mayor densidad de objetos, más oclusión y configuraciones de captura más próximas a aplicaciones reales. Desde esta perspectiva, el presente trabajo debe situarse como una línea base reproducible en 2D/2.5D bajo protocolo Cornell, y no como una comparación directa con métodos actuales de 6-DoF en *clutter*.

En consecuencia, no resulta metodológicamente correcto comparar de forma directa porcentajes reportados si no coinciden el tipo de particionado, el procedimiento exacto de split, los umbrales de evaluación y el presupuesto experimental. Además, dado que este trabajo no emplea un conjunto de prueba independiente, los valores obtenidos deben interpretarse como resultados en validación bajo el protocolo reproducible definido, y no como rendimiento final absoluto. En ese marco, el valor principal de este trabajo no reside en competir con el estado del arte, sino en construir una línea base reproducible, comparar arquitecturas bajo un protocolo controlado e integrarlas en un *pipeline* robótico demostrativo basado en ROS 2 y Gazebo.

5.6. Discusión: causas probables del rendimiento modesto y plan de mejora

Los resultados indican que el *pipeline* es funcional y reproducible, pero que el rendimiento alcanzado sigue siendo modesto respecto al potencial esperado. A partir de las curvas de entrenamiento, de las métricas agregadas y del análisis cualitativo, las causas más plausibles son las siguientes:

1. **Sensibilidad geométrica del criterio Cornell.** Pequeños errores en orientación o solapamiento pueden invalidar el agarre, incluso cuando la predicción resulta visualmente razonable. Esto hace que la métrica de éxito sea especialmente exigente y amplifique diferencias geométricas relativamente pequeñas.
2. **Formulación de salida restrictiva.** El modelo predice un único rectángulo por imagen. En escenas donde existen varios candidatos plausibles de agarre, esta formulación puede penalizar casos en los que la propuesta es razonable pero no coincide con la anotación seleccionada como referencia.
3. **Exploración limitada de hiperparámetros.** Aunque la comparación A/B se ha mantenido controlada y reproducible, no se ha llevado a cabo una búsqueda amplia de hiperparámetros, scheduler o variantes de pérdida, lo que limita la optimización fina del entrenamiento.
4. **Presupuesto experimental acotado.** Más que un problema de número de épocas de forma aislada, la principal limitación parece residir en el presupuesto global de experimentación: combinaciones de scheduler, regularización, parametrización angular y estrategias de entrenamiento no han sido exploradas de forma exhaustiva.

Como plan de mejora priorizado, se proponen las siguientes líneas:

- refinar el régimen de entrenamiento, incorporando scheduler y ajustando el criterio de parada o selección en función de métricas geométricas además de *val_success*;
- realizar una búsqueda acotada de hiperparámetros;
- revisar la parametrización del ángulo mediante formulaciones más estables;
- evaluar salidas más expresivas, como formulaciones multi-candidato o predicción densa;
- reforzar la estabilidad del entrenamiento mediante clipping, validación defensiva y control de predicciones degeneradas;
- estudiar transferencia con datasets de mayor escala, como Jacquard, si el diseño experimental lo permite.

5.7. Conclusiones parciales del capítulo

A partir de los resultados presentados, se concluye que:

- El enfoque 2D/2.5D sobre RGB(-D) es técnicamente viable y el pipeline experimental resulta reproducible;

- las configuraciones basadas en ResNet18Grasp superan claramente a las basadas en SimpleGraspCNN en la métrica principal de éxito y también en las métricas geométricas complementarias;
- la mejor configuración según *val_success* medio en validación es EXP3_RESNET18_RGB_AUGMENT, con $68,01 \pm 2,45$ %, seguida muy de cerca por EXP4_RESNET18_RGBD, con $64,92 \pm 1,02$ %;
- en el modelo ligero, la incorporación de profundidad mejora de forma apreciable el rendimiento al pasar de EXP1 (RGB) a EXP2 (RGB-D), mientras que en el modelo residual la comparación entre EXP3 y EXP4 debe interpretarse con cautela, al no aislarse de forma pura el efecto de la modalidad frente al de augmentation;
- si se consideran conjuntamente las métricas complementarias, EXP3 presenta el mejor IoU medio y el menor error angular medio, mientras que EXP4 mantiene la menor *val_loss*;
- la mejora de capacidad del modelo constituye el factor dominante del rendimiento, aunque debe seguir valorándose junto con las restricciones de despliegue y latencia;
- persiste un margen amplio de mejora, especialmente mediante ajuste fino del entrenamiento, exploración acotada de hiperparámetros y formulaciones de salida más expresivas.

Para facilitar la interpretación del efecto de la arquitectura bajo cada modalidad de entrada, la Tabla 5-4 presenta una comparativa por modalidad entre SimpleGraspCNN y ResNet18Grasp, derivada de los resultados agregados de la Tabla 5-1. En cada fila se muestra la tasa media de *val_success* para ambos modelos en una misma modalidad, así como la diferencia absoluta en puntos porcentuales.

Debe tenerse en cuenta que, en la rama RGB, la comparación entre EXP1 y EXP3 incorpora también el efecto de augmentation, por lo que no representa un contraste arquitectónico puro en sentido estricto.

Modalidad	SimpleGraspCNN <i>val_success</i> (%)	ResNet18Grasp <i>val_success</i> (%)	Δ absoluto (pp)	Interpretación
RGB	27,41	68,01	+40,60	Fuerte ventaja del backbone residual y del esquema de entrenamiento asociado
RGB-D	38,05	64,92	+26,87	Ventaja clara de ResNet18 también en modalidad RGB-D

Tabla 5-4. Comparativa por modalidad entre SimpleGraspCNN y ResNet18Grasp derivada de la Tabla 5-1.

La comparación A/B por modalidad muestra que el cambio de arquitectura produce una mejora muy marcada en la métrica principal de éxito. En RGB, ResNet18Grasp supera a SimpleGraspCNN en 40,60 puntos porcentuales, mientras que en RGB-D la ventaja es de 26,87 puntos. Estos resultados sugieren que el factor dominante es la capacidad representacional del modelo. No obstante, en la rama RGB esta comparación incorpora también el efecto de augmentation, por lo que su interpretación estricta como contraste arquitectónico puro debe hacerse con cautela.

6. Conclusiones y trabajos futuros

En este capítulo se presentan las conclusiones del trabajo, estructuradas en torno a los objetivos específicos definidos al inicio del documento. Asimismo, se sintetizan las aportaciones principales, se discuten las limitaciones observadas y se proponen líneas de trabajo futuro priorizadas para reforzar el rendimiento, la validez experimental y la consolidación de la integración robótica.

6.1. Síntesis del trabajo realizado

Este trabajo ha abordado el diseño, implementación y evaluación de un sistema de detección de poses de agarre 2D/2.5D a partir de imágenes RGB-D mediante aprendizaje profundo. La formulación adoptada representa el agarre como un rectángulo orientado en el plano imagen y emplea una métrica de *grasp success* tipo Cornell basada en criterios geométricos (IoU y diferencia angular), definida sobre el *Cornell Grasping Dataset* y utilizada en trabajos posteriores de aprendizaje profundo. Sobre esta base, se ha construido un *pipeline* experimental reproducible que cubre desde la preparación del *Cornell Grasping Dataset* hasta el entrenamiento, la evaluación y la generación automatizada de tablas y figuras de resultados.

Los resultados se reportan en validación bajo *split object-wise*. Al no disponerse de un conjunto de prueba independiente, las cifras deben interpretarse como rendimiento en validación dentro del régimen evaluado, con selección por *best_epoch*, y no como rendimiento final de generalización. Con el objetivo de estudiar el compromiso entre rendimiento y coste computacional, se han desarrollado y comparado dos modelos que comparten la misma representación de salida:

- Modelo A (ResNet18Grasp): arquitectura de referencia basada en ResNet-18 preentrenado, de complejidad moderada.
- Modelo B (SimpleGraspCNN): CNN ligera diseñada desde cero, orientada a entornos con recursos de cómputo limitados.

Los experimentos se han organizado en configuraciones base (EXP1–EXP4). El resumen cuantitativo principal, expresado como media $\pm$ desviación estándar sobre tres ejecuciones por experimento ($n = 3$), se presenta en el Capítulo 5, mientras que el detalle por semilla y las comparativas derivadas se incluyen también en ese capítulo. En conjunto, los resultados constituyen una línea base reproducible que permite: (i) validar el *pipeline* de extremo a extremo, (ii) identificar limitaciones de convergencia y estabilidad bajo el *split object-wise*, más exigente que otros particionados habituales, y (iii) establecer un punto de partida trazable para plantear mejoras incrementales y su evaluación controlada.

6.2. Consideración sobre el uso de un manipulador 6-DoF

En este trabajo, la representación 2D/2.5D se adopta como aproximación adecuada para tareas *table-top* (manipulación sobre mesa) bajo el supuesto operativo de aproximación casi perpendicular o con restricciones cinemáticas que reducen el problema a un subconjunto efectivo del espacio 6-DoF. Por tanto:

- La contribución experimental principal se limita a la percepción 2D/2.5D y su evaluación cuantitativa y cualitativa.
- La integración con 6-DoF se entiende en este trabajo como arquitectura y demostrador de integración, no como resolución completa del problema de agarre espacial 6-DoF.

Como extensión natural, se considera pertinente evaluar en el futuro la integración con manipuladores más acordes con un problema planar (p. ej., SCARA o manipuladores con fuerte componente 2D en el espacio de trabajo) o bien migrar progresivamente a formulaciones 6-DoF basadas en nubes de puntos y datasets a gran escala.

6.3. Conclusiones por objetivo específico

Para cerrar el capítulo de conclusiones, se presenta una síntesis del grado de cumplimiento de los objetivos específicos definidos en el Capítulo 2. La Tabla 6-1 recoge, para cada objetivo, el estado de cumplimiento y la evidencia concreta donde se demuestra. A partir de esta trazabilidad, se desarrolla a continuación una conclusión específica por objetivo, no solo indicando su cumplimiento, sino también el resultado principal alcanzado y su aportación al conjunto del trabajo.

Objetivo	Evidencia (dónde se demuestra)	Estado
OE1. Analizar el problema y encaje 2D/2.5D	Cap. 1 (motivación y alcance) + Cap. 3 (estado del arte y marco teórico)	Cumplido
OE2. Preparar Cornell y pipeline reproducible	Cap. 4 (dataset, preprocesado, split object-wise, reproducibilidad) + reports/cornell_audit (mencionado en texto)	Cumplido
OE3. Evaluar ResNet18Grasp (RGB/RGB-D)	Cap. 4 (configuración experimental) + Cap. 5 (resultados cuantitativos)	Cumplido
OE4. Desarrollar SimpleGraspCNN (alternativa ligera)	Cap. 4 (arquitectura/entrenamiento) + Cap. 5 (comparativa y discusión)	Cumplido
OE5. Definir protocolo reproducible (semillas, métricas, registros)	Cap. 4 (protocolo) + tablas/figuras de Cap. 5 ($n=3$, media $\pm$ desv, best epoch)	Cumplido
OE6. Evaluar modelos en rendimiento (IoU, $\Delta\theta$) y eficiencia (tamaño/latencia/coste)	Cap. 4 (métricas + protocolo latencia/eficiencia) + Cap. 5 (tabla(s) de latencia/tamaño)	Cumplido
OE7. Integrar el detector en un demostrador ROS 2 + Gazebo	Cap. 4 (integración ROS 2 + Gazebo): arquitectura, tópicos y flujo percepción→publicación→consumo por planificación/control; evidencias (diagramas/logs)	Cumplido

Tabla 6-1. Síntesis de cumplimiento de objetivos.

Tabla de síntesis elaborada por el autor a partir de la trazabilidad entre objetivos, implementaciones y evidencias generadas.

- **Objetivo 1 — Estado del arte.** Este objetivo se considera cumplido, ya que la revisión bibliográfica permitió situar el trabajo en la línea de detección de agarres 2D/2.5D sobre RGB-D, identificando tendencias relevantes como la predicción densa, los enfoques multi-candidato, las formulaciones generativas y la transición hacia agarres 6-DoF. Como resultado de esta revisión, se justificó la elección de un enfoque convolucional de complejidad moderada como primera línea base reproducible, coherente con el alcance del trabajo y con la posterior integración robótica en simulación.
- **Objetivo 2 — Preparación de Cornell y *pipeline* de datos.** Este objetivo se considera cumplido. Se ha preparado el *Cornell Grasping Dataset* con particionado *object-wise* y un flujo de preprocesamiento coherente con el problema abordado. Además, en el *pipeline* se incorporó una auditoría reproducible y el filtrado de casos problemáticos, lo que permitió estabilizar el entrenamiento, reducir incidencias debidas a etiquetas no finitas y reforzar la trazabilidad experimental del conjunto de datos utilizado.
- **Objetivo 3 — Modelo A (ResNet18Grasp).** Este objetivo se considera cumplido. Se ha diseñado, implementado e integrado en el *pipeline* común el modelo de referencia ResNet18Grasp. En el régimen experimental evaluado, este modelo aporta el mejor rendimiento del estudio en su variante RGB con aumento de datos, alcanzando el mayor *val success* medio en validación y confirmando la ventaja de una arquitectura residual con mayor capacidad representacional bajo un protocolo reproducible.
- **Objetivo 4 — Modelo B (SimpleGraspCNN).** Este objetivo se considera cumplido. Se ha implementado un modelo ligero propio, SimpleGraspCNN, que constituye una línea base estable y computacionalmente eficiente. Aunque su rendimiento es inferior al modelo residual en las condiciones evaluadas, su implementación ha permitido disponer de una comparación controlada entre capacidad predictiva y coste computacional, aportando una referencia útil para escenarios con restricciones de hardware.
- **Objetivo 5 — Protocolo experimental y métricas.** Este objetivo se considera cumplido. Se ha definido y aplicado un protocolo experimental reproducible basado en la métrica principal tipo Cornell y en métricas complementarias como IoU medio, error angular medio y pérdida de validación. Además, la instrumentación mediante CSV, scripts de análisis y selección explícita de *best_epoch* ha permitido reproducir tablas, figuras y decisiones metodológicas clave, reforzando la auditabilidad del trabajo.
- **Objetivo 6 — Evaluación de rendimiento y eficiencia.** Este objetivo se considera cumplido. Se ha realizado una evaluación comparativa controlada de las configuraciones principales bajo *split object-wise*, reportando tanto métricas geométricas de validación como medidas de eficiencia basadas en latencia de inferencia y FPS. La conclusión obtenida es que la arquitectura ResNet18 en modalidad RGB con aumento de datos fue la que ofreció el mejor compromiso global entre rendimiento predictivo y coste computacional dentro de las configuraciones analizadas, al alcanzar los mejores resultados de validación sin que la latencia de inferencia dejara de ser compatible con un uso práctico en un pipeline robótico. En cambio, los modelos basados en

SimpleGraspCNN presentaron menor coste computacional, pero también una capacidad predictiva claramente inferior, mientras que la incorporación de profundidad no aportó en este estudio una mejora suficiente como para justificar por sí sola el incremento de complejidad. Por tanto, este objetivo no solo permitió comparar cuantitativamente las configuraciones propuestas, sino también identificar cuál de ellas resulta más eficiente en términos de compromiso precisión-latencia y concluir que los modelos más competitivos podrían integrarse en un sistema robótico real o simulado con restricciones temporales moderadas, siempre que su despliegue final se complete con la validación conjunta del resto del pipeline, incluyendo percepción, planificación y control.

- **Objetivo 7 — Integración ROS 2 + Gazebo.** Este objetivo se considera cumplido. Se ha planteado, implementado y documentado la integración del detector como nodo de percepción dentro de un pipeline robótico basado en ROS 2 y Gazebo, con una interfaz explícita de publicación y consumo de hipótesis de agarrar. Esta integración no queda sustentada únicamente por la arquitectura descrita en el Capítulo 4, sino también por los resultados funcionales presentados en el subapartado 5.4, donde se muestra la adquisición de imagen simulada, la inferencia del modelo, la transformación de la hipótesis visual en una pose ejecutable y su consumo por el subsistema de planificación. En consecuencia, el trabajo no solo define una arquitectura compatible con integración robótica, sino que aporta además evidencia funcional de que el flujo percepción → publicación → consumo opera de forma coherente dentro del entorno ROS 2 + Gazebo, aun cuando la validación se interprete como demostración funcional de integración y no como una prueba cerrada de manipulación autónoma completa.

6.4. Discusión crítica de resultados y limitaciones

Los resultados de *grasp success* obtenidos indican margen de mejora respecto a enfoques especializados reportados en la literatura. No obstante, el valor principal de este trabajo es metodológico: se ha construido una base reproducible y auditable que cubre de forma integrada datos, entrenamiento, evaluación e integración, y se han identificado de forma explícita los factores que condicionan el rendimiento bajo el protocolo tipo Cornell.

Las limitaciones más relevantes son:

1. **Ausencia de conjunto de prueba independiente:** la validación se emplea tanto para selección de época como para estimación de rendimiento, lo que limita la inferencia sobre generalización.
2. **Presupuesto de entrenamiento e hiperparámetros:** Aunque las curvas tienden a estabilizarse en las primeras épocas, esto no implica necesariamente convergencia óptima, sino que puede reflejar un régimen de optimización subóptimo (p. ej., tasa de aprendizaje inadecuada, ausencia de scheduler, regularización o estrategia de ajuste fino). En este contexto, incrementar el número de épocas por sí solo puede aportar ganancias marginales; las mejoras esperables requieren ajustar el régimen de entrenamiento, por ejemplo: incorporar un planificador de tasa de

- aprendizaje (*cosine/step*), reducir LR en el *backbone* y usar LR mayor en la cabeza, aplicar *warm-up* o *early stopping*, y/o revisar la función de pérdida y la parametrización angular para estabilizar la regresión geométrica (C_x, C_y, w, h, θ).
3. **Sensibilidad del criterio Cornell:** pequeñas desviaciones en solape u orientación pueden convertir un caso en fallo, incluso cuando el agarre es visualmente razonable.
 4. **Restricción de salida (un único rectángulo por imagen):** esta formulación limita la capacidad de representar múltiples hipótesis válidas en escenas ambiguas o con oclusiones, frente a alternativas de predicción densa o multi-candidato.
 5. **Brecha de dominio hacia entornos no estructurados:** la presencia de *clutter*, oclusiones y variabilidad de fondo dificulta la generalización y exige una evaluación más amplia.

En conjunto, estas limitaciones no invalidan el *pipeline* propuesto, pero sí motivan un refuerzo del régimen experimental y de la formulación del modelo para aproximarse a rendimientos competitivos en escenarios no estructurados

6.5. Aportaciones principales

De forma sintética, las aportaciones más relevantes de este trabajo son:

1. *Pipeline* reproducible y trazable para detección de agarres 2D/2.5D en RGB-D, cubriendo preparación de datos, entrenamiento, evaluación y generación automatizada de resultados.
2. Depuración del *dataset* y “Subset estricto” mediante auditoría de integridad y generación de índices limpios, reduciendo fallos por etiquetas no finitas y mejorando la estabilidad del entrenamiento.
3. Implementación y comparación controlada A/B de dos variantes (modelo base vs modelo ligero) bajo un protocolo común (mismas particiones, métricas y criterios de selección), permitiendo análisis comparativo sin introducir sesgos de configuración.
4. Protocolo de evaluación tipo Cornell con métricas geométricas ($IoU, \Delta\theta$) y *grasp success*, complementado con evidencia cualitativa mediante *overlays* de predicción vs GT para interpretar errores típicos.
5. Demostrador de integración robótica en simulación (ROS 2 + Gazebo) donde el detector actúa como nodo de percepción y se valida el flujo extremo a extremo (sensores → inferencia → publicación → consumo/visualización), apoyado por evidencias verificables en logs y panel de operación.

Estas aportaciones dejan una base sólida para ampliar la validez externa del sistema (escenarios más complejos y despliegue progresivo) manteniendo reproducibilidad y trazabilidad.

6.6. Líneas de trabajo futuro

Las extensiones propuestas se organizan de forma progresiva, alineadas con las limitaciones observadas y con el objetivo de aumentar la validez y aplicabilidad del sistema en entornos no estructurados:

- i. **Consolidación metodológica en Cornell.** Reforzar el régimen de entrenamiento, la robustez numérica y el análisis de fallos, manteniendo el protocolo tipo Cornell para preservar comparabilidad.
- ii. **Mejora de la formulación de salida.** Explorar salidas más expresivas, como enfoques multi-candidato o predicción densa, con el fin de manejar mejor ambigüedad, oclusiones y múltiples hipótesis válidas de agarre.
- iii. **Evolución arquitectónica del modelo.** Evaluar variantes con mejor capacidad representacional o mejor compromiso precisión-coste, incluyendo backbones más modernos, como EfficientNet o ConvNeXt, que podrían ofrecer una extracción de características más robusta que ResNet18 sin un incremento desproporcionado del coste computacional, así como estrategias de transferencia desde datasets de mayor escala y formulaciones específicas para *grasp detection*.
- iv. **Robustez y escenarios complejos.** Ampliar la evaluación hacia escenas con clutter, oclusiones y mayor variabilidad de fondo, así como estudiar transferencia a configuraciones de datos más exigentes preservando trazabilidad y comparabilidad.
- v. **Extensión hacia integración robótica avanzada.** Consolidar la ejecución pick-and-place en simulación bajo condiciones controladas y, como extensión posterior, validar el sistema en hardware real manteniendo evidencias verificables del flujo extremo a extremo.
- vi. **Transición hacia agarre espacial.** Como línea de mayor alcance, estudiar la migración desde la formulación 2D/2.5D actual hacia enfoques 6-DoF basados en nubes de puntos o representaciones tridimensionales, en coherencia con la evolución del estado del arte y con la integración futura en manipuladores con mayor libertad cinemática.

6.7. Cierre del capítulo

En conjunto, el plan de trabajo futuro prioriza primero la estabilidad metodológica y la validez experimental en el marco Cornell, después la mejora de la expresividad del modelo y su robustez en escenarios no estructurados, y finalmente la consolidación de la integración robótica completa. Esta secuencia preserva la trazabilidad del *pipeline* y permite ampliar progresivamente el alcance del sistema sin perder reproducibilidad.

7. Referencias bibliográficas

Cao, B., Zhou, X., Guo, C., Zhang, B., Liu, Y., & Tan, Q. (2023). NBMOD: Find it and grasp it in noisy background [Preprint]. arXiv. <https://arxiv.org/abs/2306.10265>

Depierre, A., Dellandréa, E., & Chen, L. (2018). Jacquard: A large-scale dataset for robotic grasp detection [Preprint]. arXiv. <https://arxiv.org/abs/1803.11469>

Eppner, C., Mousavian, A., & Fox, D. (2020). ACRONYM: A large-scale grasp dataset based on simulation [Preprint]. arXiv. <https://arxiv.org/abs/2011.09584>

Fang, H.-S., Wang, C., Gou, M., & Lu, C. (2020). GraspNet-1Billion: A large-scale benchmark for general object grasping. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 11444–11453). https://openaccess.thecvf.com/content_CVPR_2020/html/Fang_GraspNet-1Billion_A_Large-Scale_Benchmark_for_General_Object_Grasping_CVPR_2020_paper.html

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition [Preprint]. arXiv. <https://arxiv.org/abs/1512.03385>

Hou, Z., Zhao, Y., Jin, Y., Yang, C., He, Z., & Chen, X. (2025). Hierarchical information-guided robotic grasp detection. Scientific Reports, 15, Article 18821. <https://doi.org/10.1038/s41598-025-03313-z>

Huang, B., Yu, J., & Jain, S. (2023). EARL: Eye-on-hand reinforcement learner for dynamic grasping with active pose estimation [Preprint]. arXiv. <https://arxiv.org/abs/2310.06751>

Jiang, Y., Moseson, S., & Saxena, A. (2011). Efficient grasping from RGBD images: Learning using a new rectangle representation. In 2011 IEEE International Conference on Robotics and Automation (ICRA) (pp. 3304–3311). IEEE. <https://doi.org/10.1109/ICRA.2011.5980145>

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In Proceedings of the 3rd International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/1412.6980>

Kleeberger, K., Bormann, R., Kraus, W., & Huber, M. F. (2020). A survey on learning-based robotic grasping. Current Robotics Reports, 1(4), 239–249. <https://doi.org/10.1007/s43154-020-00021-6>

Kumra, S., & Kanan, C. (2016). Robotic grasp detection using deep convolutional neural networks [Preprint]. arXiv. <https://arxiv.org/abs/1611.08036>

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>

Lenz, I., Lee, H., & Saxena, A. (2015). Deep learning for detecting robotic grasps. *The International Journal of Robotics Research*, 34(4–5), 705–724. <https://doi.org/10.1177/0278364914549607>

Morrison, D., Corke, P., & Leitner, J. (2018). Closing the loop for robotic grasping: A real-time, generative grasp synthesis approach. In *Robotics: Science and Systems XIV*. <https://doi.org/10.15607/RSS.2018.XIV.021>

Morrison, D., Corke, P., & Leitner, J. (2020). Learning robust, real-time, reactive robotic grasping. *The International Journal of Robotics Research*, 39(2–3), 183–201. <https://doi.org/10.1177/0278364919859066>

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems* (Vol. 32, pp. 8024–8035). Curran Associates.

Platt, R. (2023). Grasp learning: Models, methods, and performance. *Annual Review of Control, Robotics, and Autonomous Systems*, 6, 363–389. <https://doi.org/10.1146/anurev-control-062122-025215>

Ren, D., Ren, X., & Wang, X. (2021). Fast-learning grasping and pre-grasping via clutter quantization and Q-map masking [Preprint]. *arXiv*. <https://arxiv.org/abs/2107.02452>

Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Song, Y., Sun, P., Jin, P., Ren, Y., Zheng, Y., Li, Z., Chu, X., Zhang, Y., Li, T., & Gu, J. (2023). Learning 6-DoF fine-grained grasp detection based on part affordance grounding [Preprint]. *arXiv*. <https://arxiv.org/abs/2301.11564>

Sundermeyer, M., Mousavian, A., Triebel, R., & Fox, D. (2021). Contact-GraspNet: Efficient 6-DoF grasp generation in cluttered scenes [Preprint]. *arXiv*. <https://arxiv.org/abs/2103.14127>

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Xie, Z., Liang, X., & Canale, R. (2023). Learning-based robotic grasping: A review. *Frontiers in Robotics and AI*, 10, Article 1038658. <https://doi.org/10.3389/frobt.2023.1038658>

ANEXO A. Repositorio y trazabilidad experimental

Este anexo recoge la información mínima necesaria para identificar la versión del código y los artefactos empleados en los experimentos de este trabajo, con fines de reproducibilidad.

Repositorio de referencia: https://github.com/JLRRRC/TFM_VIU_09MIAR.git

Rama utilizada: main

Versión exacta del código: a50e1c776dfdeb3e6caa1569eb683690a22cbdbd

Fecha de referencia: 18 de marzo de 2026

Rutas principales utilizadas:

- `agarre_inteligente/config/`
- `agarre_inteligente/src/`
- `agarre_inteligente/experiments/`
- `reports/`
- `agarre_inteligente/data/processed/cornell/splits/object_wise/train.csv`
- `agarre_inteligente/data/processed/cornell/splits/object_wise/val.csv`

Artefactos clave para reproducibilidad:

- **Configuraciones experimentales:**
 - `agarre_inteligente/config/exp1_simple_rgb.yaml`
 - `agarre_inteligente/config/exp2_simple_rgb.yaml`
 - `agarre_inteligente/config/exp3_resnet18_rgb_augment.yaml`
 - `agarre_inteligente/config/exp4_resnet18_rgb.yaml`
- **Resultados experimentales:**
 - `agarre_inteligente/experiments/EXP*/best_epoch_summary.csv`
 - `agarre_inteligente/experiments/EXP*/seed_*/metrics.csv`
- **Manifiesto de fuentes de resultados:**
 - `reports/logs/reproducibility/report_source_manifest.tsv`
- **Métricas validadas del capítulo de resultados:**
 - `reports/metrics/validated/chapter5_experiment_summary_validated.csv`
- **Tablas e ilustraciones finales:**
 - `reports/tables/cap5/`
 - `reports/figures/cap5/`

Los resultados presentados en este trabajo corresponden a la versión del repositorio identificada por la rama y el commit indicados anteriormente. Los archivos de auditoría del dataset se conservan como soporte de trazabilidad metodológica, mientras que el mecanismo activo de carga empleado en los experimentos finales del Capítulo 5 se basa en los archivos `train.csv` y `val.csv` del split object-wise.

ANEXO B. Modelo de Declaración del uso de sistemas de IA

En la realización de este trabajo, se ha empleado inteligencia artificial de manera responsable en distintas fases del proceso.

Para el análisis de la literatura, se ha utilizado ChatGPT (GPT-5.4 Thinking) como asistente en la identificación de artículos, garantizando siempre la verificación de fuentes.

En la fase de experimentación, se ha empleado Claude Code (Claude Sonnet 4.5) para la generación de código principalmente la parte de desarrollo del panel QT e implementación de la planificación del brazo robot para el agarre del objeto, que posteriormente fue revisado y modificado manualmente.

Finalmente, en la redacción del documento, se ha utilizado ChatGPT (GPT-5.4 Thinking) para la corrección gramatical y de estilo.

En todo momento, se ha mantenido el criterio académico y se ha supervisado que el uso de estas herramientas no comprometiera la originalidad ni la calidad del trabajo.